

Universität Leipzig
Institut für Informatik

Untersuchungen zu einer hierarchischen Variante der
ACO-Metaheuristik zur Funktionsoptimierung

Masterarbeit

Leipzig, Dezember, 2025

vorgelegt von

Felix Cäsar Madorskiy
Studiengang Informatik M.Sc.

Betreuender Hochschullehrer: Prof. Dr. Martin Middendorf
Fakultät für Mathematik und Informatik/Institut für Informatik/
Schwarmintelligenz und Komplexe Systeme

Inhaltsverzeichnis

Abkürzungsverzeichnis

1. Einleitung	1
2. Grundlagen	2
2.1. Einführung in Ameisenalgorithmen	2
2.1.1. Biologische Beschreibung der Nahrungssuche von Ameisen	2
2.1.2. Mathematische Beschreibung der Nahrungssuche von Ameisen	2
2.1.3. Permutationsprobleme	4
2.1.4. Traveling Salesperson Problem (TSP)	5
2.2. ACO-Metaheuristik nach <i>Dorigo et al.</i> [11]	5
2.3. Weiterentwicklung der ACO-Metaheuristik zu der PB-ACO-Metaheuristik	6
2.4. Alternative Verwaltung für ein Lösungsarchiv	8
2.5. Ameisenalgorithmen mit einem stetigen Lösungsarchiv	8
2.6. Ameisenalgorithmen mit einem gemischten Lösungsarchiv	8
2.7. Ameisenalgorithmen mit gemischten Lösungsarchiven in einer hierarchischen Struktur	9
2.8. Optimierungsprobleme	9
2.9. $ACO_{\mathbb{R}}$	10
2.10. $ACO_{\mathbb{R}}$ -simple	12
2.11. $ACO_{\mathbb{R}}$ -very-simple	13
2.12. Erweiterung von $ACO_{\mathbb{R}}$ -very-simple zu $ACO_{\mathbb{R}}$ -very-simple-mixed-variable für Probleme mit diskreten Variablen	14
2.13. Erweiterung von $ACO_{\mathbb{R}}$ zu ACO_{MV} für Probleme mit diskreten Variablen	14
2.14. Vergleichsalgorithmen	15
2.14.1. Algorithmen, die probabilistisches Lernen einsetzen	15
2.14.1.1. $(1 + 1)$ -ES	15
2.14.1.2. CSA-ES	15
2.14.1.3. CMA-ES	15
2.14.1.4. IDEA	16
2.14.1.5. MBOA	16
2.14.2. Algorithmen, die verwandt zu Ameisenalgorithmen sind	16
2.14.2.1. CACO	16
2.14.2.2. API	16
2.14.2.3. CIAC	17
2.14.3. Metaheuristiken, die für stetige Testfunktionen adaptiert wurden	17
2.14.3.1. CGA	17
2.14.3.2. ECTS	17
2.14.3.3. ESA	18
2.14.3.4. DE	18

2.14.4. Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden	19
2.14.4.1. NLIDP	19
2.14.4.2. GA	19
3. SPPBO-Schema	20
4. HSPPBO-Schema	21
5. Kombinieren des HSPPBO-Schemas mit $ACO_{\mathbb{R}}$-very-simple-mixed-variable	22
5.1. Hom-HPB- $ACO_{\mathbb{R}}$ -VS-MV	22
5.2. Het-HPB- $ACO_{\mathbb{R}}$ -VS-MV	23
5.3. Het-HSPPB- $ACO_{\mathbb{R}}$ -VS-MV	24
6. Testfunktionen	24
6.1. Stetige Testfunktionen	26
6.1.1. Testfunktion - Paraboloid	32
6.1.2. Testfunktion - Easom	33
6.1.3. Testfunktion - Griewangk	34
6.1.4. Testfunktion - Rosenbrock	34
6.1.5. Testfunktion - Hartmann	35
6.1.6. Testfunktion - Shekel	36
6.2. Testfunktionen mit gemischten Variablen	36
6.2.1. Konstruierte Testfunktionen	36
6.2.2. Coil Spring Design Problem (CSD)	37
6.2.2.1. Vollständiger Raum als Initialisierungsintervall	40
6.2.2.2. Gefilterter Raum als Initialisierungsintervall	41
6.2.2.3. Resultierende Testkombinationen für das Coil Spring Design Problem (CSD Problem)	42
7. Vergleich von $ACO_{\mathbb{R}}$, ACO_{MV} und den Varianten von $ACO_{\mathbb{R}}$-very-simple-mixed-variable	43
7.1. Vergleich bzgl. stetiger Testfunktionen	43
7.2. Vergleich bzgl. Testfunktionen mit gemischten Variablen	50
7.2.1. Vergleich bzgl. konstruierter Testfunktionen mit gemischten Variablen	50
7.2.2. Vergleich bzgl. dem Coil Spring Design Problem (CSD)	52
8. Kurzzusammenfassung	57
Literaturverzeichnis	59
Tabellenverzeichnis	62
Abbildungsverzeichnis	62

Anlagen

I

A. Vergleich der pre-, add- und realen Funktionsauswertungen

I

Abkürzungsverzeichnis

ACO_ℝ-VS ACO_ℝ-very-simple

ACO_ℝ-VS-MV ACO_ℝ-very-simple-mixed-variable

CSD Problem Coil Spring Design Problem

SCE solution creating entities

TSP Traveling Salesperson Problem

1. Einleitung

Ein wichtiger Bereich der Informatik ist das Kreieren von Algorithmen zum Lösen von Problemen, besonders für das angenäherte Lösen von Problemen, für die eine analytische Lösung entweder nicht möglich ist, aufgrund der Beschaffenheit des Problems, oder für die eine analytische Lösung machbar ist aber diese zu berechnen Jahrhunderte brauchen kann, was zu lange dauert. Es wird also nach Algorithmen gesucht, die hinreichend gute Lösungen liefern und das in absehbarer Zeit.

Viele dieser Algorithmen sind natürlichen Vorgängen oder der Natur selbst direkt nachgeahmt, wie das Simulated Annealing aus *Kirkpatrick et al.* [25], die Evolution Strategies aus *Rechenberg* [40] oder die ACO-Metaheuristik aus *Dorigo et al.* [11]. Der letzte Algorithmus steht für Ant Colony Optimization und ist der Nahrungssuche von Ameisen nachempfunden, weswegen dort von künstlichen Ameisen gesprochen wird. ACO war erfolgreich im Lösen von Permutationsproblemen und diskreten Problemen, wie dem Traveling Salesperson Problem [46].

Einer der nächsten Evolutionsschritte dieser Algorithmen war das Erweitern diskreter Algorithmen zu Stetigen, so entstanden aus der ACO-Metaheuristik, CACO vorgestellt von *Bilchev und Parmee* [1], API vorgestellt von *Monmarché et al.* [35], CIAC vorgestellt von *Dréo and Siarry* [13] und $ACO_{\mathbb{R}}$ vorgestellt von *Socha und Dorigo* [44]. Der letzte Algorithmen bestach dadurch, dass er bei vielen Testfunktionen der beste war und, dass er bei Testfunktionen, wenn er nicht der beste war, er trotzdem gut war. Auch hier wird die Metapher von künstlichen Ameisen beibehalten. Zudem wird in *Socha und Dorigo* [44] ausgesagt, dass $ACO_{\mathbb{R}}$ von allen Erweiterungen der ACO-Metaheuristik, die von ACO im Geiste treueste Erweiterung ist.

Um Probleme der nächsten Klasse zu lösen, Probleme, bei denen eine Menge ihrer Variablen nur diskrete Werte annehmen kann, während eine andere Menge nur Stetige, also Probleme mit gemischten Variablen, wurden die bis dato bekannten Algorithmen wieder erweitert. Die beiden Erweiterung, die in dieser Arbeit betrachtet werden, sind ACO_{MV} von *Socha* [43] und die Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable ($ACO_{\mathbb{R}}$ -VS-MV), einem Algorithmus, der in dieser Arbeit eingeführt wird. Während ACO_{MV} eine direkte Erweiterung von $ACO_{\mathbb{R}}$ ist, werden die Varianten von $ACO_{\mathbb{R}}$ -VS-MV durch das Kombinierten von $ACO_{\mathbb{R}}$ -VS-MV mit dem HSPPBO-Schema gewonnen. Das HSPPBO-Schema diktiert wie Ameisen in einer hierarchischen Struktur verwaltet werden. Somit wird in dieser Arbeit die Güte der Funktionsoptimierung von der Kombination des HSPPBO-Schema mit der $ACO_{\mathbb{R}}$ -VS-MV-Metaheuristik untersucht.

2. Grundlagen

2.1. Einführung in Ameisenalgorithmen

Wie in meiner Bachelorarbeit [31] wird zunächst mit der biologischen und mathematischen Beschreibung der Nahrungssuche von Ameisen begonnen.

2.1.1. Biologische Beschreibung der Nahrungssuche von Ameisen

Wie *Bonabeau, Dorigo und Theraulaz* [3] und *Blum und Merkle* [9] beschrieben haben, hinterlassen Ameisen, bei der Nahrungssuche, wenn sie sich zwischen Nest und Futter bewegen, eine Pheromonspur. Die Menge des hinterlassenen Pheromons hängt von der Quantität und Qualität des gefundenen Futters ab. Wenn Ameisen Nest oder Futter verlassen, haben die Wege auf denen Pheromon liegt, eine höhere Wahrscheinlichkeit wieder begangen zu werden, je höher die Menge des hinterlassenen Pheromons auf einem Weg, desto höher die Wahrscheinlichkeit, dass der Weg von Ameisen begangen wird. Wenn eine Ameise also den optimalen, also kürzesten Weg zwischen Nest und Futter gefunden hat, aggregiert sich Pheromon auf diesem Weg schneller, da Ameisen, die sich für den kürzesten Weg entschieden haben, da der Weg der kürzeste ist, vom Futter schon zurückkommen, während andere Ameisen noch unterwegs sind. Zusätzlich verdunstet Pheromon über Zeit, was dazu führt, dass seltener begangene Wege zunehmend weniger Pheromon aufweisen und damit unwahrscheinlicher begangen werden. Diese beiden Mechanismen zusammen sorgen dafür, dass sich die kürzesten Wege über Zeit herauskristallisieren.

2.1.2. Mathematische Beschreibung der Nahrungssuche von Ameisen

In diesem Abschnitt wird ein ideales Futter betrachtet, also ein Futter, dass in Quantität und Qualität konstant bleibt, und ideale Ameisen, also Ameisen, die mit konstanter Geschwindigkeit sich geradlinig bewegen. Diese Annahmen müssen getroffen werden, da, wie im Abschnitt 2.1.1 beschrieben, die Menge des auf einem Weg hinterlassenen Pheromons von der Quantität und Qualität des Futters abhängt und wie in *Camazine et al.* [5] beschrieben, reale Ameisen auf einem Weg umdrehen können und wieder zurücklaufen von woher sie kamen. Ferner, in Anlehnung an *Dorigo und Stützle* [12] und *Engelbrecht* [14], wird eine ideale Experimentaufstellung in Form eines Tupels $(\Sigma, \mathcal{E}, \Delta, \mathcal{M}, f, \mathcal{I}, \mathcal{S})$ betrachtet.

Dabei ist Σ der Suchraum, welcher hier der ungerichtete Graph $G(V, E)$ ist, der aus drei Orten/Knoten, an denen sich Ameisen aufhalten können und drei Wegen zwischen diesen Orten, besteht. Der Graph befindet sich in der euklidischen Ebene. Die Orte sind Nest (N), Food (F) und Umweg/Detour (D), also gilt $V = \{N, F, D\}$. Die Wege verbinden die Orte, wie Abbildung 1 zu sehen. Die Wege haben alle die Länge 1 Längeneinheit (LE) und können in beide Richtungen begangen werden, also gilt $E = \{\{NF\}, \{ND\}, \{DF\}\}$. Zusammengefasst gilt $G = (\{N, F, D\}, \{\{NF\}, \{ND\}, \{DF\}\})$. Die Menge der idealen Ameisen bzw. der solution creating entities (SCE) ist $\mathcal{E}$. Hier gilt

$|\mathcal{E}| = 1$, es wird also jedem Durchgang eine ideale Ameise in diesem Experiment betrachtet. Die Menge des Pheromons, welches die Ameise auf die von ihr gewählten Wege hinterlässt, ist Δ . In unserem Experiment, angelehnt an [3] und [9], gilt $\Delta = 1$. Die Menge des Pheromons ρ auf allen Wegen, also die Pheromoninformation, wird in der Pheromonmatrix $\mathcal{M}$ gespeichert. Dabei steht jeder Matrixeintrag für eine Kante im Graph G und der Wert des Eintrages steht für die Menge von Pheromon, welches auf der Kante liegt. Die Zielfunktion ist f . Die Zielfunktion gibt die Summe der Weglängen der Wege an, zwischen N und F, welche die Ameisen gegangen sind. In unserer idealen Experimentaufstellung gilt also $f(x) \in \{1, 2\}$, da die Ameisen entweder den direkten Weg $\{NF\}$ gehen oder den Umweg $\{ND, DF\}$ gehen kann und damit gilt $f(\{NF\}) = 1$ oder $f(\{ND, DF\}) = 2$. Das Begehen eines Weges verbraucht exakt eine Zeiteinheit, also verbraucht der direkte Weg $\{NF\}$ eine Zeiteinheit und der Weg über den Umweg $\{ND, DF\}$ Zwei. Das Ergebnis des in diesem Abschnitt beschriebenen Vorgehens, und das Ziel für zukünftige Problemstellungen, ist das finden eines globalen Minimums der Zielfunktion. Bevor die idealen Ameisen mit dem Experiment anfangen können, wird $\mathcal{M}$ mit der Initialisierungsmethode $\mathcal{I}$ initialisiert. Hier wird jeder Matrixeintrag $(m_{ij})_{i,j=1,2,3}$ der Pheromonmatrix $\mathcal{M}$ auf Δ gesetzt, damit in den Gleichungen 1 und 2 nicht durch 0 geteilt wird. Das Experiment wird beendet, wenn das Stoppkriterium $\mathcal{S}$ erreicht ist. Das Stoppkriterium $\mathcal{S}$ ist hier, dass das Experiment, nach dem Verstreichen von 200 Zeiteinheiten beendet wird.

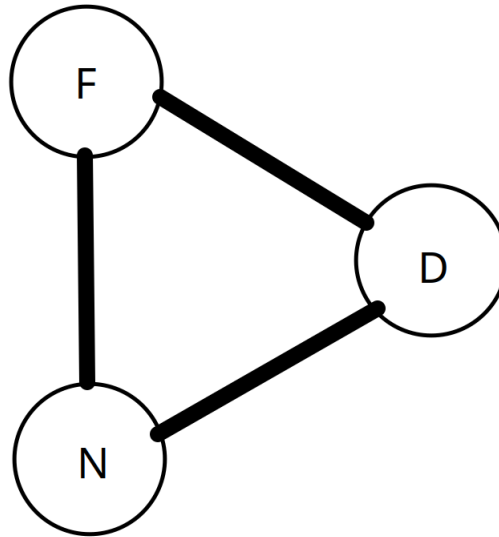


Abbildung 1: Experimentaufstellung
in Anlehnung an *Engelbrecht* [14]

Weiter, in Anlehnung an *Dorigo und Stützle* [12] und wie in meiner Bachelorarbeit [31] beschrieben, ergibt sich die Wahrscheinlichkeit, dass sich eine Ameise für das Begehen der Weges zwischen N und F entscheidet, aus der Gleichung 1 und die Wahrscheinlichkeit,

dass eine Ameise sich für das Begehen des Weges zwischen N und D entscheidet, aus Gleichung 2. Gleichung 3 beschreibt die Verdunstung des bereits vorhandenen Pheromons auf einem Weg nach dem eine Zeiteinheit verstrichen ist. Dabei ist ν die Verdunstungskonstante, welche bestimmt, wie schnell Pheromon nach jedem Zeitschritt verdunstet, und α ein Parameter, der bestimmt, wie viel Einfluss das auf einem Weg liegende Pheromon hat. Für Werte für α , nach [12], gilt $\alpha \in [0, 2]$. Wie viel Pheromon auf einem Weg liegt, also die Pheromoninformation, wird nach jedem Zeitschritt aktualisiert. Auf den Kanten, für die sich die Ameise entschieden hat, wird Pheromon abgelegt gemäß Gleichung 4.

$$prob_{NF} = \frac{\rho_{NF}^\alpha}{\rho_{NF}^\alpha + \rho_{ND}^\alpha} \quad (1)$$

$$prob_{ND} = \frac{\rho_{ND}^\alpha}{\rho_{ND}^\alpha + \rho_{NF}^\alpha} \quad (2)$$

$$\rho_{XY} = (1 - \nu) \cdot \rho_{XY}, \quad \nu \in [0, 1), \quad XY \in \{NF, ND, DF\} \quad (3)$$

$$\rho_{XY} = \rho_{XY} + \Delta_{XY} \quad (4)$$

Wie zu sehen, konstruiert das oben beschriebene Vorgehen eine Belegung der Knoten $\{N, F, D\}$ so, dass die Kürzeste Verbindung $\{FN\}$ als Ergebnis entsteht, $\{D\}$ also nicht begangen wird. Wie in meiner Bachelorarbeit [31] beschrieben, kann beobachtet werden, dass die Orte N, F und D nur begangen und nicht begangen als Werte annehmen können. Damit ist das Problem die kürzeste Verbindung zu finden, ein diskretes Problem, da die Werte, welche die Orte, verallgemeinert Variablen, annehmen können, abzählbar sind. Für das bis jetzt beschriebene Vorgehen wichtige Teilmenge von diskreten Problemen, sind Permutationsprobleme.

2.1.3. Permutationsprobleme

In Anlehnung an meine Bachelorarbeit [31], kann ein Permutationsproblem definiert werden durch das Tupel $(\mathcal{G}, \pi, \Sigma, \mathcal{B}, f)$. Für den Grundraum $\mathcal{G}$ gilt hier $\mathcal{G} = \{a_1, \dots, a_n\}$, $n \in \mathbb{N}_{>0}$, wobei die a_i beliebige Objekte sein können, die man ordnen kann. Die Permutation $\pi : \Sigma \rightarrow \Sigma$ ordnet jedem Element $a_i \in \Sigma, i \in \{1, \dots, n\}$ das korrespondierende Element $a_{\pi(i)}$ zu. Für den Suchraum Σ gilt hier $\Sigma = \{a_{\pi(1)}, \dots, a_{\pi(n)}\}$, wobei $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ jede Permutation der Grundmenge $\{a_1, \dots, a_n\}$ ist. Die Menge an Beschränkungen, welche an die Ordnungen der $a_i, i \in \{1, \dots, n\}$ oder Werte von f gestellt werden, werden mit $\mathcal{B} = \{\mathcal{B}_1, \dots, \mathcal{B}_m\}$, $m \in \mathbb{N}_{>0}$ bezeichnet. Das Permutationsproblem ist unbeschränkt, wenn $\mathcal{B} = \emptyset$ gilt, sonst ist es beschränkt. Die Zielfunktion $f : \{a_{\pi(1)}, \dots, a_{\pi(n)}\} \rightarrow \mathbb{R}_{\geq 0}$ gibt die Güte einer Ordnung $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ der Objekte an. Eine Lösung $s \in \Sigma$ ist eine Ordnung aller Elemente aus Σ , welche konform ist mit den Beschränkungen $\mathcal{B}$. Eine Lösung s_{opt} ist ein globales Optimum, wenn $f(s_{opt}) \leq f(s) \forall s \in \Sigma$ gilt. Ein wichtiger Spezialfall eines Permutationsproblems ist das Traveling Salesperson Problem (TSP).

2.1.4. Traveling Salesperson Problem (TSP)

Das TSP, nach *Stützle et al.* [46], ist ein Permutationsproblem $(\mathcal{G}, \pi, \Sigma, d, \mathcal{B}, f)$ mit $\mathcal{G} = \{a_i, \dots, a_n\}, n \in \mathbb{N}_{>0}$, wobei beim TSP die a_i die Städte sind und $d : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ die Distanz zwischen zwei Städten ist. Da das TSP keine Beschränkungen hat, gilt $\mathcal{B} = \emptyset$. Die Permutation π gibt die Reihenfolge der Städte $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ und damit eine Rundreise an. Eine Rundreise ist eine Reise, bei der jede Stadt, außer der Startstadt, nur einmal besucht wird. Die Zielfunktion f gibt die Länge der Rundreise $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ an. Gesucht ist mindestens ein $s_{opt} \in \Sigma, s_{opt} = \{a_{\pi(1)}, \dots, a_{\pi(n)}\}$, sodass gilt $f(s_{opt}) \leq f(s) \forall s \in \Sigma$. Es ist also mindestens eine kürzeste Rundreise gesucht. Das TSP war eines der Probleme, dass sich angeboten hat, um mit der ACO-Metaheuristik gelöst zu werden.

2.2. ACO-Metaheuristik nach *Dorigo et al.* [11]

Die ACO-Metaheuristik, nach *Dorigo et al.* [11], steht für **Ant Colony Optimization**-Metaheuristik und kann als Erweiterung der Konstruktion aus Abschnitt 2.1.2 gesehen werden, da sie auch als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \Delta_c, \mathcal{M}_c, \eta, f, \mathcal{I}, \mathcal{S})$ angegeben werden kann. Dabei ist zwar $\Sigma = G(V, E)$ aber V und E sind beliebig, genauso wie die Beschränkungen $\mathcal{B}$. Weiter gilt $|\mathcal{E}| = m, m \in \mathbb{N}$, anstatt $|\mathcal{E}| = 1$. Die Menge des Pheromons Δ , welche die iterationsbeste Ameise auf die von ihr gewählten Kanten hinterlegt, wird problemabhängig gewählt. Die Pheromonmatrix $\mathcal{M}$ kann größer sein als im Abschnitt 2.1.2 aber sie ist genauso aufgebaut und auch ihre Funktionalität ist die selbe. Auch die Verdunstungsmechanik aus Gleichung 3 wird beibehalten. Die problemabhängige Heuristik ist η . Die Zielfunktion f kann auch beliebig formuliert werden, es bleibt jedoch gleich, dass mindestens eines ihrer globalen Minima gesucht wird. Das Initialisierungskriterium bleibt gleich, wie im Abschnitt 2.1.2. Das Stoppkriterium $\mathcal{S}$ kann frei gewählt werden.

Für die ACO-Metaheuristik angewandt auf das TSP kann $\mathcal{B} = \max_{v_i, v_j \in V} d(v_i, v_j) \leq D$ sein, wobei D die maximale erlaubte Distanz zwischen zwei Städten ist, wenn die maximale Tankfüllung eines Transporters oder andere Beschränkungen berücksichtigt werden müssen, sonst gilt $\mathcal{B} = \emptyset$. Die Anzahl der Ameisen ist $|\mathcal{E}| = m$. Die Menge des Pheromons Δ , dass von der iterationsbesten Ameise abgelegt wird, ist in der Gleichung 5 angegeben. Für das TSP wird η in der Gleichung 6 angegeben, wobei $d(v_i, v_j), v_i, v_j \in V$ die Distanz zwischen v_i und v_j ist.

$$\Delta_{ij} = \sum_{k=1}^m \frac{1}{f(s)}, (ij) \in s \quad (5)$$

Dabei ist f die im Abschnitt 2.1.4 definierte Zielfunktion, welche die Länge der Rundreise s wiedergibt. Wie zu sehen, ist die Menge des auf einer Kante (i, j) hinterlegten Pheromons größer, je kürzer die Rundreise s ist, in der die Kante (i, j) vorkommt.

$$\eta_{ij} = \frac{1}{d(v_i, v_j)} \text{ für zwei Knoten } v_i, v_j \in V \quad (6)$$

Wie in *Dorigo et al.* [11] beschrieben werden bei der Anwendung der ACO-Metaheuristik auf das TSP die Gleichungen 1 und 2 zuerst zur Gleichung 7 erweitert, wobei V' die Menge der Städte ist, die noch nicht besucht wurden, danach zur Gleichung 8. Dabei hat α die selbe Funktion, wie in den Gleichungen 1 und 2 und β ist ein Parameter, der bestimmt, wie viel Einfluss die Heuristik η hat. Die Gleichung 8 ist schließlich die Gleichung, welche beim Algorithmus 1 im Schritt „CreateSolution(ant)“ verwendet wird. In diesem Schritt wird eine Ameise zunächst auf einen zufällig ausgewählten Knoten gesetzt und von diesem Knoten aus wird dann mithilfe der Gleichung 8 der nächste Knoten gewählt. Dies wird solange wiederholt, bis $V' = \emptyset$ gilt.

$$prob_{ij} = \frac{\rho_{ij}}{\sum_{v \in V'} \rho_{iv}} \quad (7)$$

$$extend_prob_{ij} = \frac{\rho_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{v \in V'} \rho_{iv}^\alpha \cdot \eta_{iv}^\beta} \quad (8)$$

Als letztes fällt noch auf, dass „UpdatePheromoneInformation“, gegeben eine $n \times n$ -Matrix, da jeder Matrixeintrag aktualisiert werden muss, $\mathcal{O}(n^2)$ Schritte braucht, was bei großen Problemen viel Zeit verbraucht. Dies wurde durch *Guntzsch und Middendorf* [17], mit der Weiterentwicklung der ACO-Metaheuristik zu der PB-ACO-Metaheuristik, verbessert.

Algorithm 1 ACO-Metaheuristik angewandt auf das TSP

```

1: repeat
2:   for  $ant \in \mathcal{E}$  do
3:     CreateSolution(ant)
4:   UpdatePheromoneInformation( $\mathcal{M}$ )
5: until  $\mathcal{S}$  fulfilled

```

2.3. Weiterentwicklung der ACO-Metaheuristik zu der PB-ACO-Metaheuristik

Die folgende Beschreibung ist nach *Guntzsch und Middendorf* [17] und in Anlehnung an meine Bachelorarbeit [31].

PB-ACO steht für **P**opulation **B**ased-ACO-Metaheuristik. Die Unterschiede zu ACO sind, dass in PB-ACO die Pheromoninformation nicht mehr in einer stetigen $n \times n$ -Pheromonmatrix $\mathcal{M}$, sondern in einer diskreten $n \times n$ -Pheromonmatrix $\mathcal{M}_d$ gespeichert wird. Die Matrix $\mathcal{M}_d$ ist diskret, da ihre Elemente nur endlich viele Werte annehmen können. In PB-ACO wird auch zudem eine Liste $\mathcal{P}$ der Länge $k, k \in \mathbb{N}$, von Lösungen mitgeführt, wobei in [17] für das TSP gilt $k \in \{1, 5, 25\}$. Die in $\mathcal{P}$ gespeicherten Lösungen

werden als Population interpretiert, daher „PB-ACO“ als Name für diese Erweiterung von „ACO“. Demnach können wir, angelehnt an ACO, die PB-ACO-Metaheuristik als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \Delta_d, \mathcal{M}_d, \mathcal{P}, \eta, f, \mathcal{I}, \mathcal{S})$ angeben. Nach [17], gilt für das TSP, $|\mathcal{E}| = 10$. Dabei wird in *Guntsch und Middendorf* [17] die Liste der Populationen nach dem Prinzip „first in last out“ befüllt. Bei diesem Prinzip wird, nachdem die Liste $\mathcal{P}$ zunächst mit k Lösungen befüllt wurde, wenn eine neue Lösung generiert wird, die älteste Lösung entfernt, die verbleibenden Lösungen um eine Stelle nach hinten verschoben und die neue Lösung an den ersten Platz in der Liste $\mathcal{P}$ gespeichert. Ein anderes Prinzip $\mathcal{P}$ zu verwalten ist das „best in worst out“ Prinzip. Dieses Prinzip wird im Abschnitt 2.5 erläutert. Das Befüllen von $\mathcal{P}$ mit k Lösungen und dem folgenden Aufbau von $\mathcal{M}_d$ ist hier die Initialisierungsmethode $\mathcal{I}$. Ein weiterer Unterschied ist, dass die Pheromonmatrix $\mathcal{M}_d$ nicht mehr beliebig viel Pheromoninformation enthält, wie $\mathcal{M}$, sondern jedes Element nur $k + 1$ verschiedene Werte annehmen kann, wodurch $\mathcal{M}_d$ diskret ist. Ferner muss bei PB-ACO nicht mehr jedes Element der Pheromonmatrix $\mathcal{M}_d$ bei jeder Iteration betrachtet werden. Nur die Elemente von $\mathcal{M}_d$, welche in der neuen Lösung, welche in $\mathcal{P}$ aufgenommen wird, vorkommen und die in der alten Lösung vorkommen, welche aus $\mathcal{P}$ entfernt wird, und damit verändert werden, müssen betrachtet werden. Beides braucht $n, n \in \mathbb{N}$ Schritte, also insgesamt $2n$ Schritte. Damit braucht das Pheromonupdate statt $\mathcal{O}(n^2)$ Zeit, nur $\mathcal{O}(n)$, wie in *Guntsch und Middendorf* [17] beschrieben.

Genauer, nach [17] und [31], seien $\Delta > 0$, Δ konstant, $\rho_0 = 1/(n - 1)$. Dann werden die Elemente der Pheromonmatrix $\mathcal{M}_d$ gemäß Gleichung 9 initialisiert. Nach [17] gilt weiter für das TSP für die Gleichung 8, $\alpha = 1$ und $\beta = 5$. Die Elemente von $\mathcal{M}_d$ werden dann, wie in Gleichung 10 dargestellt, aktualisiert. Dabei darf, wie zuvor, nur die iterationsbeste Ameise Pheromon legen. Die Tatsache, dass jedes Element von $\mathcal{M}_d$ nur $k + 1$ verschiedene Werte annehmen kann, ergibt sich daraus, dass, auf der einen Seite, jedes Element mindestens aus dem Initialisierungswert ρ_0 besteht und, auf der anderen Seite, nur Vielfache von Δ hinzugefügt werden können und das nur maximal k -Mal, da die Menge an Pheromon, welches die iterationsbeste Ameise ablegen kann, konstant Δ ist und die Population der Ameisen $\mathcal{P}$, nach Initialisierung, konstant die Größe k hat. Somit kann eine Kante (i, j) nur maximal in k Lösungen vorkommen und damit ergibt sich, dass auf einer Kante nur $k + 1$ verschiedene Mengen an Pheromon liegen können. Dies wird nochmal in der Gleichung 11 verdeutlicht.

$$(m_{ij})_{i,j=1,\dots,n} = (\rho_{ij})_{i,j=1,\dots,n} = \rho_0; \quad (9)$$

$$\rho_{ij} = \rho_{ij} + \Delta \quad (10)$$

$$\rho_0 \leq \rho_{ij} \leq \rho_0 + k \cdot \Delta \quad (11)$$

Schließlich kann festhalten werden, dass die Population $\mathcal{P}$ als Lösungsarchiv interpretieren werden kann, mit dessen Hilfe neue Lösungen konstruiert werden können, da in $\mathcal{P}$ die Pheromoninformation implizit gegeben ist. Darauf aufbauend wird in den folgenden

Algorithmen in dieser Arbeit auf eine Pheromonmatrix verzichtet, da diese, gegeben $\mathcal{P}$, überflüssig in diesen Algorithmen ist.

2.4. Alternative Verwaltung für ein Lösungsarchiv

Ein anderes Prinzip ein Lösungsarchiv zu verwalten ist das „best in worst out“ Prinzip. Bei diesem Prinzip, wie in meiner Bachelorarbeit [31] beschrieben, wird eine Lösung nach ihrer Güte in das Lösungsarchiv einsortiert. Ist die Güte der neu generierte Lösung schlechter als die Güte aller Anderen im Archiv, wird sie ignoriert. Ist die Güte der neuen Lösung besser als bei einer Anderen, wird die letzte Lösung im Archiv gelöscht und alle Lösungen, die schlechter sind, als die neue Lösung, sofern diese existieren, wandern um eine Position im Archiv nach unten. Wird dieses Verwaltungsprinzip verwendet, legt es den Selektionsdruck von den Ameisen auf das Archiv, weswegen in den folgenden Algorithmen in dieser Arbeit gilt $|\mathcal{E}| = 2$, wie im Abschnitt 2.9 oder $|\mathcal{E}| = 1$, wie in den Abschnitten 2.10 und 2.11. Dieses Verwaltungsprinzip wird in allen folgenden Algorithmen in dieser Arbeit verwendet.

2.5. Ameisenalgorithmen mit einem stetigen Lösungsarchiv

Wie in meiner Bachelorarbeit [31] und im Abschnitt 2.3 beschrieben, kann man Pheromoninformation in einer diskreten Matrix speichern. Diese Matrix kann dann implizit als ein diskretes Lösungsarchiv interpretiert werden, was nur möglich ist, wenn Testfunktionen oder Probleme diskret sind. Um stetige Testfunktionen und Probleme zu lösen, bedarf es eines Archivs, welche stetigen Einträgen halten kann. Dies kann man erreichen, indem man ein Archiv erstellt, dass stetige Einträge halten kann, wie es vorher für diskrete Einträge gemacht wurde.

2.6. Ameisenalgorithmen mit einem gemischten Lösungsarchiv

Da in dieser Arbeit die Art der Algorithmen, die getestet werden und die Art der Testfunktionen, die zum Testen benutzt werden, erweitert wird, muss auch der Begriff des Lösungsarchivs erweitert werden. Gegeben eine Funktion mit gemischten Variablen, wie in den Abschnitten 6.2.1 und 6.2.2 und ein einfaches Lösungsarchiv mit $n \in \mathbb{N}$ Einträgen, wie es im Abschnitt 2.5 definiert wurde, dann ist die einfachste Erweiterung für ein Lösungsarchiv, mehrere Lösungsarchive zu einem zusammengesetzten Lösungsarchiv zusammenzufassen. Dabei bilden die Einträge der Unterarchive an der Stelle $i \in \{1, \dots, n\}$, $n \in \mathbb{N}$, eine Gesamtlösung. Zum Beispiel kann ein zusammengesetztes einfaches Lösungsarchiv $\{\{'diskret': [1], 'stetig': [1.0, 1.5]\}, \{'diskret': [2], 'stetig': [2.0, 3.0]}\}$ sein. Dann ist die 2.te Lösung $\{'diskret': [2], 'stetig': [2.0, 3.0]\}$.

2.7. Ameisenalgorithmen mit gemischten Lösungsarchiven in einer hierarchischen Struktur

Wie in [26] beschrieben, werden bei einem hierarchischen Algorithmus die Ameisen in einer Baumstruktur angeordnet und die Anzahl der Archive wird von insgesamt 1 auf 3 pro Ameise erhöht. Ein Archiv für die letzte gefundene Lösung der Ameise, Eines für die bis dato beste gefundene persönliche Lösung der Ameise und Eines für die bis dato beste gefundene Lösung der Elternameise. Zusätzlich gibt es noch 1 weiteres Archiv, welches die bis dato insgesamt beste Lösung hält. Die Größe der Lösungsarchive schrumpft aber, von $n = 100$ bei ACO_ℝ-VS [31], auf $n = 1$. Dies ist notwendig, damit Lösungen entlang dem Baum wandern können, wie z.B. im HSPBO-Schema, denn man kann Archive nur dann einfach vergleichen, wenn sie nur eine Lösung haben. Das Vergleichskriterium ist dann der Funktionswert der Fitnessfunktion.

Eine offene Forschungsfrage ist, wie mit Archiven in einer hierarchischen Struktur umgegangen werden kann, wenn sie mehr als 1 Lösung halten. Die einfachste Anordnung ist nach der Besten Lösung im Archiv zu ordnen. Die zweit einfachste Anordnung ist die Archive nach der Durchschnittsqualität der Lösungen zu sortieren. Eine komplexere Möglichkeit die Archive zu ordnen ist einen zusammengesetzten Term zu verwenden, wie in Gleichung 12 zu sehen. Dieser Term funktioniert sowohl für Archive, die Lösungen nach dem Prinzip „first in last out“ speichern als auch für Archive, die Lösungen nach dem Prinzip „best in worst out“ speichern. Im letzten Fall gilt $s_1 = s_{opt}$.

$$Güte = c_1 \cdot f(s_1) + c_2 \cdot \frac{\sum_{i=2}^n f(s_i)}{n-1} \quad (12)$$

Dabei sind $c_1, c_2 \in \mathbb{R}$ Konstanten, s_{opt} ist die bis dato gefundene beste Lösung und n die Anzahl der Lösungen in den Archiven, wobei alle Archive die selbe Anzahl an Lösungen halten. In diesem Fall kann auf eine persönliche und globale beste Lösung verzichtet werden. Als Ausgangspunkt für weitere Forschung eignet sich mit der Beziehung $c_1 + c_2 = 1$, $c_1, c_2 \in [0, 1]$ anzufangen. Später kann man allgemeiner verlangen, dass nur $c_1, c_2 \in \mathbb{R}$ gilt. Als letzte Forschungsfrage können verschiedene Archive verschieden viele Lösungen halten.

2.8. Optimierungsprobleme

Da wir, im Vergleich zu meiner Bachelorarbeit [31], in dieser Arbeit die Menge der betrachteten Testfunktionen und Probleme auf Testfunktionen und Probleme mit gemischten Variablen erweitern, bietet es sich an, eine abstraktere Definition von Optimierungsproblemen zu etablieren.

Wie an [30] angelehnt, wird ein Optimierungsproblem durch 4 Teile definiert. Diese sind der Suchraum $\Sigma = \{\Sigma_1, \dots, \Sigma_n\}, n \in \mathbb{N}$, die Menge der abstrakten Variablen $X = \{X_1, \dots, X_n\}, n \in \mathbb{N}$, die Beschränkungen an die Variablen $\mathcal{B} = \{\mathcal{B}_1, \dots, \mathcal{B}_n\}, n \in \mathbb{N}$ und eine Fitness- bzw. Zielfunktion $f : \Sigma \rightarrow \mathbb{R}^+$, die jeder Lösung $s \in \Sigma$ einen Wert $f(s) > 0$

zuordnet, wobei eine Lösung $s \in \Sigma$ das Belegen aller Variablen X_i mit den Werten aus Σ_i ist, sodass die Werte den Beschränkungen $\mathcal{B}_i$ genügen, $i \in \{1, \dots, n\}$. Damit kann ein Optimierungsproblem definiert werden als das Tupel $(\Sigma, X, \mathcal{B}, f)$. Wenn $\mathcal{B} = \emptyset$, dann ist das Problem unbeschränkt, sonst ist es beschränkt.

Eine Population $\mathcal{P}$ besteht aus mindestens einer Lösung $s \in \Sigma$. Alle Lösungen $s \in \Sigma$, die auch in einer Population $\mathcal{P}$ sind, werden durch $\Sigma|\mathcal{P}$ beschrieben. Darauf aufbauend, gegeben eine Population $\mathcal{P}$, beschreibt $\Sigma|\mathcal{P}_i, i \in \{1, \dots, n\}$, alle Werte in Σ an der Stelle i , die auch in $\mathcal{P}$ an der Stelle i auftreten.

Eine Lösung s_{opt} heißt Optimum, wenn gilt $f(s_{opt}) \leq f(s) \forall s \in \Sigma$. Ein Optimierungsproblem kann mehr als ein s_{opt} haben. Das Ziel eines Optimierungsproblems ist mindestens ein s_{opt} zu finden.

Mit dieser Definition kann beispielhaft der Suchraum und die Variablen für das Coil Spring Design Problem angegeben werden. Der Suchraum ist $\Sigma = \{\mathbb{R}, \mathbb{O}, \mathbb{N}_{>0}\}$ und die Variablen sind $X = \{D, d, N\}$. $\mathbb{O}$ ist die Menge aller rationalen ordinalen Zahlen. Es ist die Menge der rationalen Zahlen $\mathbb{Q}$ ohne eine Ordnungsrelation. D ist eine stetige Variable, d ist ordinal und N ist natürlich, wobei die Beschränkungen $D \in \{3 \cdot d_{min}, 3.0\}$, $d \in \{d_{min}, \dots, 0.5000\}$ und $N \in \{1, \dots, 70\}$ gelten. Die Beschränkung d_{min} wird im Abschnitt 6.2.2 zum CSD Problem angegeben.

2.9. ACO_ℝ

In Anlehnung an *Socha und Dorigo* [44] und wie in meiner Bachelorarbeit [31] beschrieben, baut der ACO_ℝ Algorithmus auf einem stetigen Lösungsarchiv $\mathcal{P}_c$ der Größe k auf. Auf eine Pheromonmatrix und damit auf Pheromon, sowie eine problemabhängige Heuristik wird verzichtet. Demnach kann ACO_ℝ als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \mathcal{P}_c, f, \mathcal{I}, \mathcal{S})$ angegeben werden. In [44] gilt $|\mathcal{E}| = 2$, $|\mathcal{P}_c| = k = 50$.

Die Initialisierungsmethode $\mathcal{I}$ ist auch hier das Füllen von $\mathcal{P}_c$ mit Lösungen, wie bei PB-ACO. Aber Das Stoppkriterium ist problemabhängig. So kann z.B. $|f - f^*| < \varepsilon, \varepsilon = 10^{-10}$ sein, wie in Abschnitt 6.1, wobei f^* das bekannte Optimum einer Testfunktion ist und f der Momentane Funktionswert, den der Lösungsalgorithmus gefunden hat.

Die Veränderung von PB-ACO zu ACO_ℝ, um stetige Testfunktionen und Probleme lösen zu können, transformiert die Lösungskonstruktion aus Gleichung 8 zur Gleichung 13. Und es wird nicht mehr betrachtet wie wahrscheinlich das wandern entlang einer Kante (i, j) ist, sondern es wird im i -ten Konstruktionsschritt ein Gauss-Kernel in der i -ten Dimension ausgewertet, wobei gilt $i \in \{1, \dots, n\}$ und n ist sowohl die Dimension der Testfunktion, als auch die Anzahl der Konstruktionsschritte, die jede Ameise pro Iteration macht.

..

$$G^i = \sum_{l=1}^k \omega_l g_l^i(x) = \sum_{l=1}^k \omega_l \frac{1}{\sigma_l^i \sqrt{2\pi}} e^{-\frac{(x-\mu_l^i)^2}{2\sigma_l^{i2}}}, \quad i \in \{1, \dots, n\} \quad (13)$$

Weiter, in Anlehnung an *Socha und Dorigo* [44] gilt für die Vektoren ω , μ und σ , dass ihre Kardinalität gleich der Kardinalität von $\mathcal{P}$ ist, also gilt $|\omega| = |\mu| = |\sigma| = |\mathcal{P}| = k = 50$. Die Elemente der Parametervektoren ω , μ und σ werden in den Gleichungen 14, 15 und 16 berechnet. Dabei gilt, dass n die Anzahl der Dimensionen der Testfunktion und i der Konstruktionsschritt ist. Weiter ist s_j^i die i -te Variable der j -ten Lösung. Der Parametervektor ω unterscheidet sich von μ und σ darin, dass er nicht zur Erzeugung der nächsten Lösung benutzt wird, sondern er jeder bereits vorhandenen Lösung im Lösungsarchiv $\mathcal{P}_c$ ein Gewicht ω_l zuordnen, welches dazu benutzt wird mithilfe der Gleichung 17 eine Wahrscheinlichkeitsverteilung aufzubauen. Unter Anwendung dieser Wahrscheinlichkeitsverteilung wird dann die l -te Gaußfunktion gewählt.

Der Term in Gleichung 16 wird in *Socha und Dorigo* [44] als Standardabweichung bezeichnet, auch wenn in diesem Term keine Wurzel vorhanden ist. Dies wird so verstanden, dass der Term zwar aus mathematischer Sicht keine Standardabweichung ist aber als Standardabweichung für den Algorithmus interpretiert wird, da in der Gleichung 13 nach einer Standardabweichung verlangt wird.

$$\mu = \{\mu_1^i, \dots, \mu_k^i\} = \{s_1^i, \dots, s_k^i\}, \quad i \in \{1, \dots, n\} \quad (14)$$

$$\omega_l = \frac{1}{qk\sqrt{2\pi}} e^{-\frac{(\ell-1)^2}{2q^2k^2}}, \quad l \in \{1, \dots, k\} \quad (15)$$

$$\sigma_l^i = \xi \sum_{e=1}^k \frac{|s_e^i - s_l^i|}{k-1}, \quad l \in \{1, \dots, k\}, \quad i \in \{1, \dots, n\} \quad (16)$$

$$p_l = \frac{\omega_l}{\sum_{r=1}^k \omega_r}, \quad l \in \{1, \dots, k\} \quad (17)$$

Nachdem die Parametervektoren berechnet wurden, wird im Konstruktionsschritt i die Gaußfunktion g_l^i ausgewertet, was durch einen Zufallszahlengenerator, der Zufallszahlen aus einer parametrisierten Normalverteilung erzeugt, passieren kann.

Der Parameter q steuert, wie stark bessere Lösungen im Archiv bei der Konstruktion einer neuen Lösung bevorzugt werden. Je kleiner q , desto stärker werden bessere Lösungen bevorzugt. Der Parameter ξ steuert, wie schnell der Algorithmus konvergiert. Je größer ξ , desto langsamer konvergiert der Algorithmus. In [44] gilt $q = 10^{-4}$, $\xi = 0.85$.

2.10. ACO_ℝ-simple

Wie in meiner Bachelorarbeit [31] beschrieben, entstand der ACO_ℝ-simple Algorithmus durch eine Vereinfachung des ACO_ℝ Algorithmus. Der ACO_ℝ-simple Algorithmus nutzt das Lösungsarchiv auch wie ACO_ℝ, jedoch setzt er die vorhandenen Lösungen auf eine einfachere Weise zu einer neuen Lösung zusammen – deswegen simple, also einfach.

Die vier Konzepte, die aus ACO_ℝ übernommen wurden, ist die Verwendung eines Lösungsarchivs $\mathcal{P}_c$, die Zuweisung jeder Lösung im Archiv eines Gewichts w_i , dass mit der Gleichung 18 berechnet und zum Aufbau einer Wahrscheinlichkeitsverteilung mithilfe der Gleichung 19 benutzt wird und somit wie eine Lösung s_i aus dem Archiv gewählt wird, die Art wie die Abweichung in Gleichung 20 berechnet wird, sowie die Verwendung der Parameter q und ξ . Bei der Berechnung der Abweichung in Gleichung 20 wird aber, aufgrund der Art und Weise wie die Abweichung späteren verwendet wird, explizit die lineare Abweichung gewollt.

Im Gegensatz zu ACO_ℝ gilt in ACO_ℝ-very-simple aber $|\mathcal{P}| = k = 100$, $q = 0.95$ und $\xi = 0.95$. Die Archivgröße wurde verdoppelt, da der Algorithmus viel einfacher ist als ACO_ℝ und damit eine größere Archivgröße ACO_ℝ-very-simple erlaubt auch vom momentanen Bestwert weiter entferntere Lösungen im Suchraum zu erforschen. Das ist auch der selbe Grund, wieso $q = 0.95$ gesetzt wurde. Auf der anderen Seite wurde $\xi = 0.95$ gesetzt, damit das Suchintervall in Gleichung 21 möglichst erhalten bleibt. Es gab auch die Möglichkeit $\xi = 1.0$ zu setzen aber im Sinne der Einfachheit wurde $q = \xi = 0.95$ gesetzt. Im Bestreben nach Einfachheit habe wurde auch $|\mathcal{E}| = 1$ gesetzt, da der Selektionsdruck, der durch das Verwalten des Lösungsarchivs nach dem „best in worst out“Prinzip entsteht, für ausreichend gehalten wurde. Demnach kann ACO_ℝ-simple als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \mathcal{P}_c, f, \mathcal{I}, \mathcal{S})$ angegeben werden. Der größte Unterschied zu ACO_ℝ liegt aber darin wie ACO_ℝ-simple eine neue Lösung erzeugt. Denn nachdem die lineare Abweichung in jeder Dimension $d, d \in \{1, \dots, n\}$ der n -dimensionalen Testfunktion/des n -dimensionalen Problems in Gleichung 20 berechnet wurde, wird nur noch ein Intervall in Gleichung 21 in jeder Dimension $d, d \in \{1, \dots, n\}$ der Testfunktion/des Problems mithilfe der linearen Abweichung aufgebaut und dann wird aus jedem der Intervalle ein zufälliger Punkt als Komponente der neuen Lösung gewählt.

$$\omega_i = \frac{1}{qk\sqrt{2\pi}} e^{-\frac{(i-1)^2}{2q^2k^2}}, \quad i \in \{1, \dots, k\} \quad (18)$$

$$p_i = \frac{w_i}{\sum_{j=1}^n w_j}, \quad i \in \{1, \dots, k\} \quad (19)$$

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j_d} - s_{i_d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\} \quad (20)$$

$$[s_d - \delta_d, s_d + \delta_d], \quad d \in \{1, \dots, n\} \quad (21)$$

Die Standardeinstellungen der Parameter sind, wie in Tabelle 1 zu sehen ist.

Tabelle 1: Parametereinstellungen ACO_ℝ-simple

Parameter	Symbol	Wert
Anzahl der Ameisen	m	1
Konvergenzgeschwindigkeit	ξ	0.95
Lokalität der Suche	q	0.95
Archivgröße	k	100

2.11. ACO_ℝ-very-simple

Wie in meiner Bachelorarbeit [31] beschrieben, baut der Algorithmus ACO_ℝ-very-simple (ACO_ℝ-VS) auf ACO_ℝ-simple auf, indem er ACO_ℝ-simple weiter vereinfacht – deswegen very-simple, also sehr einfach. Jedoch sind die Algorithmen sich noch hinreichend ähnlich, dass ACO_ℝ-VS als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \mathcal{P}_c, f, \mathcal{I}, \mathcal{S})$ angegeben werden kann. Die Vereinfachung besteht darin, dass, anstatt eine Lösung aus dem Archiv nach einer Wahrscheinlichkeitsverteilung auszuwählen, immer die erste und damit beste Lösung s_1 im Archiv zur Lösungsgenerierung gewählt wird. Damit entfallen die Pendants zu den Gleichungen 18 und 19 und es kann direkt die Gleichung 23 berechnet werden. Danach wird in ACO_ℝ-VS, genau wie in ACO_ℝ-simple, in jeder Dimension $d, d \in \{1, \dots, n\}$ der n -dimensionalen Testfunktion/des n -dimensionalen Problems, mithilfe der Gleichung 23, ein Intervall aufgebaut und dann wird aus jedem dieser Intervalle ein zufälliger Punkt als Komponente der Lösung gewählt.

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j_d} - s_{1_d}|}{k - 1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\} \quad (22)$$

$$[s_d - \delta_d, s_d + \delta_d], \quad d \in \{1, \dots, n\} \quad (23)$$

Die Standardeinstellungen der Parameter sind, wie in Tabelle 2 zu sehen ist.

Tabelle 2: Parametereinstellungen ACO_ℝ-VS

Parameter	Symbol	Wert
Anzahl der Ameisen	m	1
Konvergenzgeschwindigkeit	ξ	0.95
Archivgröße	k	100

2.12. Erweiterung von $\text{ACO}_{\mathbb{R}}$ -very-simple zu $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable für Probleme mit diskreten Variablen

Da $\text{ACO}_{\mathbb{R}}$ -VS ursprünglich für Funktionsoptimierung von stetigen Testfunktionen/Problemen entwickelt wurde, muss die $\text{ACO}_{\mathbb{R}}$ -VS-Metaheuristik für diskrete Testfunktionen/Probleme angepasst werden.

Um $\text{ACO}_{\mathbb{R}}$ -VS anzupassen werden nicht-stetige Testfunktionen/Probleme als stetige Testfunktionen/Probleme behandelt. Das heißt, es wird mit nicht-stetigen Werten gerechnet als wären sie stetig. Es wird also bis zur Gleichung 23 genauso verfahren, wie im Abschnitt 2.11, um einen neuen, stetigen Wert zu erhalten. Nachdem man einen neuen stetigen Wert gewonnen hat, betrachtet man die nicht-stetigen Werte zwischen denen der stetige Wert liegt und wählt dann als Ergebnis, den nicht-stetigen Wert, an dem der Stetige am nächsten liegt. Diese so gewonnen Erweiterung von $\text{ACO}_{\mathbb{R}}$ -VS wird $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable oder kurz $\text{ACO}_{\mathbb{R}}$ -VS-MV genannt.

Schließlich wird $\text{ACO}_{\mathbb{R}}$ -VS um eine Neustartstrategie erweitert. Der Algorithmus kann entweder eine fixe Anzahl an Neustarts ausführen oder adaptiv neu starten. Für die Neustartstrategie, bei welcher der Algorithmus eine fixe Anzahl neu startet, wird die Anzahl mit *fixed* = n angegeben. Es wurde $n = 0$ gesetzt, um einen Kontrast zu der Neustartstrategie *adaptiv* zu haben. Wenn die Neustartstrategie *adaptiv* gewählt wurde, startet der Algorithmus neu, wenn eine Periode von γ Lösungen hintereinander sich um weniger als ein Threshold ε_{th} unterscheiden. Im Sinne der Einfachheit wurde als Start $\gamma = 7$ und $\varepsilon_{th} = 10^{-10}$ gesetzt.

Tabelle 3: Parametereinstellungen $\text{ACO}_{\mathbb{R}}$ -VS-MV

Parameter	Symbol	Wert
n	n	0
Periode	γ	7
Threshold	ε_{th}	10^{-10}

2.13. Erweiterung von $\text{ACO}_{\mathbb{R}}$ zu ACO_{MV} für Probleme mit diskreten Variablen

Die Erweiterung von $\text{ACO}_{\mathbb{R}}$ zu ACO_{MV} ist ähnlich zu der Erweiterung von $\text{ACO}_{\mathbb{R}}$ -very-simple zu $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable. Wie in *Socha* [43] beschrieben, damit ACO_{MV} mit diskreten Variablen umgehen kann, rechnet ACO_{MV} mit den Indizes der diskreten Werte in Form von reellen Zahlen, anstatt mit den diskreten Werten selbst. Weiter rechnet ACO_{MV} wie $\text{ACO}_{\mathbb{R}}$. Am Ende werden dann die gewonnen Werte für die

diskreten Werte an die Indizes gerundet, die am nächsten an den neuen diskreten Werten liegen.

2.14. Vergleichsalgorithmen

Neben dem Algorithmus $\text{ACO}_{\mathbb{R}}$ selbst, gibt es noch drei Gruppen von Algorithmen, mit denen sich $\text{ACO}_{\mathbb{R}}$ -VS und die Iterationen von $\text{ACO}_{\mathbb{R}}$ -VS-MV vergleichen. Die Algorithmen dieser Gruppen sind in den Abschnitten 2.14.1, 2.14.2 und 2.14.3 vorgestellt.

2.14.1. Algorithmen, die probabilistisches Lernen einsetzen

2.14.1.1. $(1 + 1)$ -ES

Die folgende Beschreibung ist nach *Rechenberg* [40].

ES steht für **E**volution **S**trategies. $(1 + 1)$ -ES ist eines der ersten ES, die für Optimierung von stetigen Funktionen f vorgeschlagen wurden. In $(1 + 1)$ -ES wird mit einem Elter (parent) begonnen und in jeder Iteration gibt es nur ein Kind (offspring), deswegen $(1 + 1)$. Die zugrunde liegende Population besteht also, in jeder Generation $g, g \in \{0, 1, 2, \dots\}$, nur aus dem Elter und dem Kind. Nach der Initialisierung des Elter $x_{parent}^{(g)}$, wird das Kind $x_{offspring}^{(g+1)}$ in der Gleichung 24 gewonnen, wobei I die Einheitsmatrix ist und σ die Verteilungsvarianz ist und als „Step Size“ bezeichnet wird.

$$x_{offspring}^{(g+1)} = z, z \sim \mathcal{N}(x_{parent}^{(g)}, \sigma^{(g)^2} I) \quad (24)$$

Weiter wird mit der Gleichung 25 verfahren und das so gewonnene Kind wird zum neuen Elter.

$$x_{offspring}^{(g+1)} = \begin{cases} x_{offspring}^{(g+1)} & , \text{ wenn } f(x^{(g+1)}) \leq f(x_{parent}^{(g)}) \\ x_{parent}^{(g)} & , \text{ sonst.} \end{cases} \quad (25)$$

2.14.1.2. CSA-ES

Die folgende Beschreibung ist nach *Ostermeier et al.* [37].

CSA-ES steht für **E**volutionary **S**trategy with **C**umulative **S**tep **S**ize **A**daptation und ist eine Weiterentwicklung von $(1 + 1)$ -ES. Die Weiterentwicklung besteht darin, dass die Step Size σ adaptiv ist und in Abhängigkeit des Weges, welcher von den letzten $n, n \in \mathbb{N}$, Generationen gewählt wurde, geändert wird.

2.14.1.3. CMA-ES

Die folgende Beschreibung ist nach *Hansen und Ostermeier* [19].

CMA-ES steht für **E**volutionary **S**trategy with **C**ovariance **M**atrix **A**daptation. Es ist die Weiterentwicklung von CSA-ES, bei der σ von einer Kovarianzmatrix beeinflusst wird.

2.14.1.4. IDEA

Die folgende Beschreibung ist nach *Bosman und Thierens* [4].

IDEA steht für das IDEA-Framework, was wiederum für **I**terated **D**ensity Estimation **E**volutionaty **A**lgorithms-Framework steht. Algorithmen, die mit dem IDEA-Framework entworfen wurden, bauen probabilistische Modelle und schätzen Wahrscheinlichkeitsdichten, gegeben einiger Anfangspunkte.

2.14.1.5. MBOA

Die folgende Beschreibung ist nach *Očenášek und Schwarz* [36].

MBOA steht für **M**ixed **B**ayesian **O**ptimization **A**lgorithm. Es ist die Weiterentwicklung von hBOA, das wiederum für **h**ierarchical **B**ayesian **O**ptimization **A**lgorithm. Nach [38] ist hBOA der erste Estimation of Distribution Algorithm (EDA), der Bayessche Netze mit Entscheidungsbäumen kombinierte. MBOA ist eine Erweiterung von hBOA, die auch mit gemischten Variablen, diskret und stetig, umgehen kann.

2.14.2. Algorithmen, die verwandt zu Ameisenalgorithmen sind

2.14.2.1. CACO

Die folgende Beschreibung ist nach *Bilchev und Parmee* [1].

CACO steht für **C**ontinuous **A**CO. In CACO startet der Algorithmus mit einem zufälligen Punkt $\mathcal{N}$ im Suchraum. Dieser Punkt wird „Nest“ bezeichnet. Gute Lösungen werden als Vektoren, ausgehend vom Nest, gespeichert. Danach wird probabilistisch ein Vektor gewählt und sein Ende wird zum neuen Nest. Danach wird der Algorithmus wiederholt.

2.14.2.2. API

Die folgende Beschreibung ist nach *Monmarché et al.* [35].

API ist nach der „Pachycondyla apicalis“-Ameise benannt – „Pachycondyla **API**calis“ – und der Algorithmus API ist der Nahrungssuche dieser Ameise nachempfunden. Dabei

startet auch API mit einem zufälligen Punkt $\mathcal{N}$, der Nest bezeichnet wird, im Suchraum. Ausgehend von $\mathcal{N}$ etablieren n Ameisen Jagdgründe innerhalb eines Kreises um $\mathcal{N}$ mit dem Radius $R = 20 LE$, $LE = \text{Längeneinheit}$. In [35] wird $n = 20$ gesetzt. Jeder Jagdgrund hat selbst einen Grenzkreis von $r = 2 LE$. Jede Ameise kann sich p Jagdgründe merken. In [35] wird $p = 2$ gesetzt. Jede Ameise führt dann eine Suche in einem von ihr gemerkten Jagdgründe aus. Wenn in einem Jagdgrund P -mal keine bessere Lösung, als die von der jeweiligen Ameise bis dato gefundene Lösung, gefunden wurde, wird der Jagdgrund vergessen. In der Wildnis gilt für die echten Ameisen $1 LE = 1 \text{ Meter}$. Nach T Iterationen, wobei in [35] $T = 50$ gesetzt wird, wird das Nest $\mathcal{N}$ an den bis dato besten gefunden Punkt gesetzt. Es wurde in [35] $P = T$ gesetzt, damit ein Jagdgrund erst dann vergessen wird, wenn das Nest $\mathcal{N}$ bewegt wird.

2.14.2.3. CIAC

Die folgende Beschreibung ist nach *Dréo and Siarry* [13].

CIAC steht für **C**ontinuous **I**nteracting **A**nt **C**olony. In CIAC kommunizieren die Ameisen auf zwei Wegen, indirekt über Pheromon und direkt über Nachrichten. Nachrichten bestehen wiederum aus zwei Komponenten, die Position des Senders und dem Funktionswert der Zielfunktion an seiner Position.

2.14.3. Metaheuristiken, die für stetige Testfunktionen adaptiert wurden

2.14.3.1. CGA

Die folgende Beschreibung ist nach *Chelouah und Siarry* [7].

CGA steht für **C**ontinuous **G**enetic **A**lgorithm. Nach [21] und [20] wurden Genetic Algorithms zunächst für Permutationsprobleme entwickelt und zeichnen sich durch 3 Teile aus, nämlich Selektion, Crossover und Mutation. Selektion wählt aus, welche Agenten/-Ameisen in einer gegebenen Generation ausgewählt werden, sich vermehren dürfen und damit die nächste Generation erzeugen und Crossover, der Mechanismus wie sich Ameisen vermehren, generiert aus zwei Eltern zwei Nachkommen, die Eigenschaften beider Eltern vermischt ausweisen. Mutation wird auf einen Teil der Nachkommen angewandt und verändert bei jedem ausgewählten Nachkommen eine seiner Variablen. Wie groß die Änderung ist, ist algorithmusabhängig. Continuous Genetic Algorithm ist eine Erweiterung von GA, die auch mit stetigen Variablen umgehen kann.

2.14.3.2. ECTS

Die folgende Beschreibung ist nach *Chelouah und Siarry* [6].

ECTS steht für **E**nhanced **C**ontinuous **T**abu **S**earch. Nach [15] werden bei Tabu Search Tabulisten erzeugt, welche die letzten m Züge des Suchalgorithmus speichern, wobei m

eine Die so gespeicherten Züge sind verboten bzw. tabu. Dies soll das Zurückverfolgen von bereits gegangenen Schritten ausschließen. Wenn also ein Suchalgorithmus von 3 zu 4 gegangen ist, darf er für eine gewisse Zeit nicht mehr von 4 zu 3 gehen. Enhanced Continuous Tabu Search ist eine Erweiterung von Tabu Search, die auch mit stetigen Variablen umgehen kann.

2.14.3.3. ESA

Die folgende Beschreibung ist nach *Siarry et al.* [42].

ESA steht für **E**nanced **S**imulated **A**nnealing. Nach [25] wird bei Simulated Annealing eine Temperatur $T \in \mathbb{R}$ eingeführt. Die Temperatur/Variable T steuert, wie stark der Zufall in die Lösungssuche eingeht. Es wird zunächst mit einem hohen T begonnen, was einen hohen Einfluss von Zufall bedeutet. Danach wird T , bis zu einem Threshold, langsam verringert. Wenn der Threshold erreicht ist, wird T wieder erhöht aber nicht so hoch, wie beim Anfang. Dies sorgt dafür, dass ein gefundenes lokales Minima wieder verlassen werden kann, was für Simulated Annealing wichtig ist, da Simulated Annealing für hochdimensionale Probleme mit vielen lokalen Minima entwickelt wurde. Wie weit in einer Iteration, gegeben eines Startpunktes, gegangen werden kann, wird durch einen Step Vector geregelt. Enhanced Simulated Annealing zeichnet sich durch eine optimierte Temperaturänderungsstrategie und Step Vector-Kontrolle aus, sowie bessere Stopkriterien.

2.14.3.4. DE

Die folgende Beschreibung ist nach *Storn und Price* [45].

DE steht für **D**ifferential **E**volution. DE wendet Selektion, Crossover und Mutation, um ein Minimum einer gegebenen Zielfunktion zu finden. Diese Metaheuristik heißt differential, da in der mathematischen Definition des Schrittes „Mutation“ eine Differenz zentrale Bedeutung hat, wie in Gleichung 26 zu sehen.

$$v_{i,G+1} = x_{r_1,G} + F \cdot (x_{r_2,G} - x_{r_3,G}), \quad (26)$$

wobei die $x_{i,G}$, $i \in \{1, \dots, N\}$, $N \in \mathbb{N}$ Ausgangsparametervektoren sind, G die Generation ist, $r_1, r_2, r_3 \in \{1, \dots, N\}$ Indizes sind, die zufällig gewählt wurden, jedoch verschieden von i sind, $F \in [0, 2]$ ist ein konstanter Faktor, der den Einfluss der Differenz $(x_{r_2,G} - x_{r_3,G})$ steuert und $(x_{r_2,G} - x_{r_3,G})$ ist die Differenz nach der die Metaheuristik benannt ist.

2.14.4. Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden

2.14.4.1. NLIDP

Die folgende Beschreibung ist nach *Sandgren* [41].

NLIDP steht für **N**onlinear **I**nteger and **D**iscrete **P**rogramming. Nach [32] ist Linear Programming (LP) beschrieben durch als die Formulierung eines Problems, bei dem sowohl Zielfunktion als auch Beschränkung linear vorliegen. Abstrakt wird es formuliert in der Gleichung 27.

$$\begin{aligned} (LP) \quad & \min \sum_{i=1}^n a_i x_i, \quad i \in \{1, \dots, N\}, \quad N \in \mathbb{N} \\ & s.d. \sum_{i=1}^n b_{ji} x_i \geq c_j, \quad j \in \{1, \dots, m\}, \quad m \in \mathbb{N} \\ & x_i \in \mathbb{R}_{\geq 0}, \quad \forall j \in \{1, \dots, N\} \\ & a, b, c \in \mathbb{R} \end{aligned} \tag{27}$$

Nonlinear Integer and Discrete Programming ist Linear Programming bei der die Suchvariablen $x_i, i \in \{1, \dots, N\}, N \in \mathbb{N}$ nicht mehr linear vorkommen müssen aber dafür auf die natürlichen Zahlen und auf diskrete Werte beschränkt sind.

2.14.4.2. GA

Die folgende Beschreibung ist nach *Wu und Chow* [52].

GA steht für **G**enetic **A**lgorithm. In GA werden die Variablen durch Zeichenketten fester oder variabler Länge codiert, welche als Chromosome interpretiert werden. Jede Zeichenkette besteht aus Zeichen, welche als Gene interpretiert werden. Der Wert, den ein Gen kodiert, wird als Allel interpretiert. Jede Zeichenkette ist eine Lösung. In GAs wird eine konstante oder variable Menge an Lösungen in jedem Iterationsschritt geführt, welche als Population interpretiert werden. Die erste Population wird durch eine Initialisierungsmethode generiert und wird als Initialpopulation bezeichnet. Die Güte einer Population wird durch eine Fitnessfunktion gegeben, welche das bestimmen von Kennzahlen der Population ermöglicht, wie die Minimumfitness, Maximumfitness und durchschnittliche Fitness der Chromosome in einer Population, sowie die Summe der Fitness aller Chromosome. Diese Kennzahlen bilden die Basis für die Erzeugung neuer Chromosome durch genetische Operatoren, wie Selektion, Crossover, Mutation, Inversion, Dominanz und Segregation. Danach ersetzt eine Menge der neuen Chromosome eine Menge der alten Chromosome und es entsteht eine Population. Dies wird wiederholt bis das Stoppkriterium erreicht ist.

3. SPPBO-Schema

Das SPPBO-Schema steht für **S**imple **P**robabilistic **P**opulations-**B**ased **O**ptimization-Schema und wie in [30] beschrieben, besteht es aus 6 Teilen. Diese sind der Suchraum Σ , die Menge der lösungskreierenden Entitäten $\mathcal{E}$, die Menge der Populationen bzw. Archive von Lösungen $\mathcal{P}$, die Heuristik η , die Initialisierungsmethode $\mathcal{I}$ und das Stoppkriterium $\mathcal{S}$. Also kann das SPPBO-Schema beschrieben werden durch das Tupel $(\Sigma, \mathcal{E}, \mathcal{P}, \eta, \mathcal{I}, \mathcal{S})$. Im Folgenden werden die lösungskreierenden Entitäten als SCE (solution creating entities) bezeichnet.

Der Suchraum Σ kann $\Sigma = \{a_1, \dots, a_n\}$ sein, für das TSP, wobei die $a_i, i \in \{1, \dots, N\}$ die Positionen der Städte sind, $\mathbb{R}^n, n \in \mathbb{N}$ für stetige Funktionen, wie der Paraboloidfunktion, wie sie in Tabelle 4 definiert ist oder eine Zusammensetzung von Intervallen über verschiedenen Skalenniveaus, wie es in Abschnitt 6.2.2 für das CSD ist, wo Σ sich aus einem stetigem, einem ordinalen und einem Intervall über den natürlichen Zahlen zusammensetzt. Im SPPBO-Schema kann $\mathcal{E}$ aus 2 Ameisen, wie in ACO [2] oder PACO [17], bestehen oder auch nur einer Ameise, wie in ACO _{$\mathbb{R}$} -VS [31], oder $N \in \mathbb{N}$ Partikeln, wie im Set-Based Discrete PSO [8]. Die Menge der Populationen $\mathcal{P}$ im SPPBO-Schema, wie es in [30] definiert ist, besteht aus zwei Populationen, die jede SCE mitführt, einer Persönlichen P_{pers}^{SCE} und einer Globalen P_{best}^{SCE} . Die persönliche Population p_{pers} besteht aus der besten Lösung, welche die jeweilige SCE bis einschließlich der letzten Iteration des Algorithmus gefunden hat. Die globale Population P_{best}^{SCE} besteht aus der besten Lösung, die alle SCE bis einschließlich der letzten Iteration gefunden haben. Wenn es jedoch nur eine Population, wie in ACO _{$\mathbb{R}$} -VS [31], gibt, dann kann diese $n \in \mathbb{N}$ Lösungen haben, wobei $n = 100$ in [31] gewählt wurde.

Beim Ausführen des SPPBO-Schema werden mit Hilfe der Initialisierungsmethode $\mathcal{I}$ zunächst Lösungen aus dem Suchraum Σ erzeugt, welche die Initiallösungen der Populationen bilden. Sollte es Einschränkungen/Constraints für Σ geben, wie es für das CSD Problem ist, können diese gleich beim Erzeugen der Initiallösungen berücksichtigt werden, sodass nur gültige Lösungen aus Σ generiert werden. Sollte sich für dieses Vorgehen entschieden werden, können zusätzliche Iterationen bei der Initialisierung benötigt werden, neben den Iterationen bei der Ausführung der Lösungssuche des Algorithmus. In diesem Fall wird die Initialisierungsmethode als $\mathcal{I}_{exact}$ bezeichnet. Wenn das Filtern des Suchraumes Σ nach gültigen Lösungen bei gegebenen Einschränkungen während der Initialisierung nicht durchgeführt wird, können die Suchvariablen mit Werten aus Σ so belegt werden, dass keine gültigen Lösungen entstehen. Deswegen wird, sollte der Suchraum Σ nicht gefiltert werden, eine Straffunktion eingeführt, welche sich aus der Zielfunktion und Straftermen zusammensetzt. In diesem Fall wird die Initialisierungsmethode als $\mathcal{I}_{penalty}$ bezeichnet. Bei der Anwendung von ACO _{$\mathbb{R}$} -VS auf das CSD Problem werden beide Vorgehen betrachtet und miteinander verglichen. Algorithmus 2 zeigt nochmal den Ablauf des SPPBO-Schema in Pseudocode.

Algorithm 2 SPPBO

```
1: Execute  $\mathcal{I}$ 
2: repeat
3:   for  $SCE \in \mathcal{E}$  do
4:     CreateSolution(SCE)
5:   for  $P \in \mathcal{P}$  do
6:     UpdatePopulation(P)
7: until  $\mathcal{S}$  fulfilled
```

4. HSPPBO-Schema

Das HSPPBO-Schema steht für **H**ierarchical **S**imple **P**robabilistic **P**opulations-**B**ased **O**ptimization-Schema. Es ist die hierarchische Version des SPBBO-Schema [30]. Das HSPPBO-Schema, wie in [26] beschrieben, besteht aus 11 Teilen, den 6 Teilen aus dem SPPBO-Schema und einer Hierarchie- bzw. Baumstruktur $\mathcal{B}$, einer Funktion *range* $\mathcal{R}$, welche jeder $SCE \in \mathcal{E}$ eine Menge von Populationen $P \in \mathcal{P}$ zuweist, also $range(A) \subseteq \mathcal{P}$, eine Funktion *update* $\mathcal{U}$, welche bestimmt, wie Knoten getauscht werden, wenn ein Kindknoten eine bessere Lösung findet, als der Elternknoten, einem Threshold θ , der bestimmt, wann der Algorithmus auf Veränderungen im SCE-Baum reagiert und eine Funktion *handling* $\mathcal{H}$, welche bestimmt, wie der Algorithmus auf Veränderungen reagiert. Demnach kann das HSPPBO-Schema beschrieben werden durch das Tupel $(\Sigma, \mathcal{E}, \mathcal{B}, \mathcal{R}, \mathcal{U}, \theta, \mathcal{H}, \mathcal{P}, \eta, \mathcal{I}, \mathcal{S})$. In der Ausführung in [26] weist $\mathcal{R}$ jeder $SCE \in \mathcal{E}$ drei Populationen zu. Eine Population $P_{persprev}^{SCE}$ hält die von der SCE als letztes berechnete Lösung des Optimierungsproblems, eine Population $P_{persbest}^{SCE}$ hält die bis dato beste berechnete Lösung des SCE und eine Population $P_{parentbest}^{SEC}$ hält bis dato beste berechnete Lösung der Eltern-SCE. Es gilt also $P_{parentbest}^{SEC} = P_{persbest}^{parent(SCE)}$. Im Fall der Wurzel-SCE sind die letzten beiden Populationen identisch. Damit gibt es implizit eine globale beste Lösung, da die Population $P_{persbest}^{SCE^*}$ der Wurzel SCE^* ausgelesen werden kann.

Das HSPPBO-Schema zeichnet sich dadurch aus, dass die SCE in einer Baumstruktur angeordnet sind. Dadurch wird eine Nachbarschaftstopologie ermöglicht, welche die SCE gemäß der Güte ihrer Lösung ordnet. In [26] wurde sich für eine tertiäre Baumstruktur mit 13 Knoten entschieden, die hier übernommen wird. Wie beschrieben ordnet $\mathcal{R}$ jeder der SCE drei Populationen zu, die alle nur eine Lösung haben. Von diesen Populationen ist für die Funktion $\mathcal{U}$ nur die Population $P_{persbest}^{SCE}$ entscheidend. Stellt der Algorithmus fest, dass $P_{persbest}^{SCE} > P_{persbest}^{parent(SCE)}$ gilt, werden die SCE im Baum getauscht. Die Prüfung, ob dies gilt, wird „top-down“ durchgeführt. Wie in [26] beobachtet, lässt dieses Vorgehen schlechte Lösungen schnell im Baum sich nach unten bewegen, während gute Lösungen hinreichend lange gut bleiben müssen, um im Baum nach oben zu wandern. Wird festgestellt, dass die Anzahl der Vertauschung im Baum im Verhältnis zur Gesamtanzahl an SCE im Baum einen Threshold θ übersteigt, wird die Funktion $\mathcal{H}$ aufgerufen. Nach [26] gilt für den Threshold $\theta \in \{0.1, 0.25, 0.5\}$. Im Code ist $\theta = 0.25$ eingestellt. Die Einstel-

lung wurde beibehalten. Wird der Threshold überschritten, greift $\mathcal{H}$. Nach [26] gibt es drei Einstellungen für $\mathcal{H}$, nämlich $\mathcal{H}_{full}$, $\mathcal{H}_{partial}$ und $\mathcal{H}_{none}$. Ist $\mathcal{H}_{full}$ eingestellt, werden beim überschreiten des Thresholds alle Populationen $P_{persbest}^{SCE}$ gelöscht. Das entspricht einem kompletten Vergessen der SCE und ist äquivalent zum Neustart des Algorithmus. Ist $\mathcal{H}_{partial}$ eingestellt, werden nur die Population $P_{persbest}^{SCE}$ der SCE ab der Tiefe 3 und abwärts gelöscht. Ist $\mathcal{H}_{none}$ eingestellt, gibt es keine Veränderungen an den Populationen. Im Code ist $\mathcal{H}_{partial}$ eingestellt. Die Einstellung wurde beibehalten.

Algorithmus 3 zeigt nochmal den Ablauf des HSPPBO-Schema in Pseudocode.

Algorithm 3 HSPPBO

```

1: Initialize  $\mathcal{B}$ 
2: Execute  $\mathcal{I}$ 
3: repeat
4:    $L \leftarrow L + 1$ 
5:   for  $|\mathcal{E}|$  do
6:     NumberOfSwaps = 0
7:     CreateSolution(SCE)
8:     UpdatePopulation( $P_{persprev}^{SCE}$ )
9:     UpdatePopulation( $P_{persbest}^{SCE}$ )
10:  for  $|\mathcal{E}|$  do
11:    if  $f(P_{persbest}^{parent(SCE)}) < f(P_{persbest}^{SCE})$  then
12:      swap tree position between SCE and parent(SCE)
13:      NumberOfSwaps  $\leftarrow$  NumberOfSwaps + 1
14:  if NumberOfSwaps  $> \lceil \theta \cdot |\mathcal{E}| \rceil$  and  $L_{pause} > 5$  then
15:     $L \leftarrow 0$ 
16:    Execute  $\mathcal{H}$ 
17: until  $\mathcal{S}$  fulfilled

```

5. Kombinieren des HSPPBO-Schemas mit $ACO_{\mathbb{R}}$ -very-simple-mixed-variable

5.1. Hom-HPB- $ACO_{\mathbb{R}}$ -VS-MV

Die einfachste Kombination aus dem HSPPBO-Schema und $ACO_{\mathbb{R}}$ -VS ergibt Homogen Hierarchical Population-Based $ACO_{\mathbb{R}}$ -VS, Hom-HPB- $ACO_{\mathbb{R}}$ -VS. Dabei führt jede SCE, nach der Initialisierung, den $ACO_{\mathbb{R}}$ -VS Algorithmus aus. Das Archiv, welches bei der Ausführung von $ACO_{\mathbb{R}}$ -VS notwendig ist, hat diesmal statt 100 Lösungen nur 4. Diese 4 Lösungen sind die 3 Populationen, die jede der SCE mitführt plus die Population, welche die beste bis dato gefunden Lösung hält, wie in Abschnitt 4 erläutert. Der Mechanismus, welcher das Tauschen von Populationen steuert, wenn eine Eltern-SCE eine schlechtere Lösungen oder eine Kind-SCE eine bessere Lösung findet, wie ebenfalls im Abschnitt 4

dargelegt, bleibt der gleiche. Dies führt zur Bezeichnung Hierarchical Population-Based $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable oder kurz HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV. Was noch nicht aufgegriffen wurde und wovon in [26], über das Tauschen der Knoten im Baum hinaus, kein weiterer Gebrauch gemacht wird, ist, dass die **sec!**s (**sec!**s), abhängig von ihrer Position im Baum, nicht exakt denselben Algorithmus ausführen müssen. Weil aber die erste Iteration der Kombination des HSPPBO-Schemes mit dem $\text{ACO}_{\mathbb{R}}$ -VS-MV Algorithmus dies ebenfalls nicht ausnutzt, wird diese Iteration Homogen-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV bezeichnet oder kurz Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS.

5.2. Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV

$\text{ACO}_{\mathbb{R}}$ -VS hat in seiner Definition im Abschnitt 2.11, Gleichung 22, einen Parameter, der sich anbietet, um, abhängig von der Position der SCE im tertiären Baum, mit der Höhe im Baum zu variieren. Dieser Parameter ist ξ . Es wird die Gleichung 22 aufgegriffen und hier als Gleichung 28 benutzt. Dabei hängt ξ_{depth} von der Tiefe des SCE im Baum ab.

$$\delta_{dim} = \xi_{depth} \sum_{j=1}^k \frac{|s_{j-d} - s_{1-d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (28)$$

Für die Tiefe 0, also die Wurzel, wird $\xi = 0.7$, für die Tiefe 1 $\xi = 1.0$, für die Tiefe 2 $\xi = 1.3$ und ab der Tiefe 3 $\xi = 1.5$ gesetzt.

Die Entscheidung die initialen Werte von ξ so zu wählen, ergibt sich aus der Überlegung, dass je höher im Baum sich die SCE befindet, desto höher auch die Güte der Lösung dieses SCE. Ferner gilt für viele Optimierungsprobleme, dass in der Nähe schlechter Lösungen sich weitere schlechte Lösungen befinden und in der Nähe guter Lösungen sich weitere gute Lösungen befinden. Daher verkleinert ξ das Suchintervall für die Tiefe 0, lässt es unverändert für die Tiefe 1 und vergrößert es ab Tiefe 2. Die Werte für ξ wurden explizit so gewählt, da sie als einfach erschienen und damit im Einklang mit der Idee möglicher Einfachheit hinter $\text{ACO}_{\mathbb{R}}$ -VS sind. Da der Wert für ξ_{depth} von der Tiefe des SCE im tertiären Baum abhängt, führen die **sec!** abhängig von ihrer Position im tertiären Baum verschiedene Variationen von $\text{ACO}_{\mathbb{R}}$ -VS aus. Deswegen wird diese Iteration der Kombination des HSPPBO-Schemas mit dem $\text{ACO}_{\mathbb{R}}$ -VS Algorithmus als Heterogen-HPB- $\text{ACO}_{\mathbb{R}}$ -VS oder kurz Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS bezeichnet.

Jetzt muss jedoch die Generierung des neuen Punktes in Gleichung 23 anpassen werden, da für $\xi > 1.0$ das Intervall größer sein kann als Initialisierungsintervall, was dazu führen kann, dass eine Lösung generiert wird, die nicht im Definitionsraum der Testfunktion/des Problems liegt und damit ein Optimum angenommen werden kann, dass keine gültige Lösung ist. Deswegen wird die Gleichung 23 zur Gleichung 29 geändert, wobei lg_d und rg_d die respektiven linken bzw. rechten Intervallgrenzen der d -ten Dimension der n -dimensionalen Testfunktion sind.

$$[\max(s_d - \delta_d, lg_d), \min(s_d + \delta_d, rg_d)], \quad d \in \{1, \dots, n\}, \quad (29)$$

5.3. Het-HSPPB-ACO_ℝ-VS-MV

Schließlich wird noch die Zufallskomponente in Hetero-HPB-ACO_ℝ-VS eingearbeitet. Auch hier bietet es sich an, abhängig von der Position der SCE im Baum, die Wahrscheinlichkeit mit der die Zufallskomponente verwendet wird, zu verändern. Die Zufallskomponente für die sich entschieden wurde, ist das Verwenden einer zufälligen Lösung aus dem Initialisierungsraum, anstatt der Lösungskonstruktion in Gleichung 29. Diese zufällige Lösung wird in der Tiefe 0 mit der Wahrscheinlichkeit 0% genommen, in der Tiefe 1 mit 10%, in der Tiefe 2 mit 15% und in der Tiefe 3 mit der Wahrscheinlichkeit 20%. Auch hier wurden die Werte für die Wahrscheinlichkeiten explizit so gewählt, da sie als einfach erschienen und damit im Einklang mit der Idee möglicher Einfachheit hinter ACO_ℝ-VS sind. Da in diese Iteration der Kombination des HSPPB-Schemas mit dem ACO_ℝ-VS Algorithmus eine einfache probabilistische Komponente eingebaut wurde, bezeichnen wir diese Iteration als Heterogen-HSPPB-ACO_ℝ-VS oder kurz Het-HSPPB-ACO_ℝ-VS.

6. Testfunktionen

Aufbauend auf meiner Bachelorarbeit [31], wird die Art der Testfunktionen, anhand derer die Güte der hier vorlegten Erweiterungen von ACO_ℝ-VS beurteilt werden, erweitert. Wo ACO_ℝ-VS nur auf stetigen Testfunktionen getestet wurde, werden die Erweiterungen von ACO_ℝ-VS auf stetigen Testfunktionen getestet, auf Testfunktionen mit gemischten Variablen und einem Ingenieur-Benchmark-Problem.

Da in meiner Bachelorarbeit [31] die Testfunktionen in ihren Dimensionen nicht variiert haben, Intervallgrenzen keine Rolle spielten und alle Funktionen über stetigen, zusammenhängenden Definitionsbereichen definiert wurden, brauchte es keiner standardisierten, abstrakten Definition. In dieser Arbeit aber werden stetige Testfunktionen und Testfunktionen mit gemischten Variablen, sowie ein Ingenieur-Benchmark-Problem mit gemischten Variablen, betrachtet. Ferner liegen 5 der Testfunktionen mit gemischten Variablen in drei verschiedenen Dimensionen dar, nämlich $n \in \{2, 5, 10\}$. Deswegen war eine überarbeitete Definition der Testfunktionen notwendig. Die einfachste Definition mit dem neuen Standard ist im Code-Snipped 1 zu sehen.

Code-Snipped 1: Paraboloid 10 Dim stetiger Definitionsbereich

```

1 {
2     "name": "Paraboloid",
3     "comment": "standard paraboloid problem",
4     "type": "simple continuous problems",
5     "dimension": 10,
```

```

6      "optimum_argument": [[0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
7          0.0, 0.0, 0.0, 0.0]],
8
9      "variable_boundaries": {
10          "natural_numbers": [],
11          "discrete": [],
12          "continuous": [[-3, 7]],
13          "ordinal": [],
14          "categorical": []
15      },
16
17      "discrete_values": [],
18
19      "magnitudes_of_categorical_variables": []
20 }

```

Wie im Code-Snipped 1 zu sehen, besteht die neue, abstrakte Definition u.a. aus „variable_boundaries“. Dies ist ein neuer Eintrag, der notwendig geworden ist, da für das Problem im Abschnitt 6.2.2 die Intervallgrenzen zugleich Beschränkungen der Suchintervalle sind. Ein weiterer Grund das JSON-Format zu nehmen, ist, dass alle Einträge als Wörterbücher (Dictionary) ausgelesen werden können, was die Definition von neuen abstrakten Einträgen leicht macht, da die Anordnung, im Vergleich zu einem Feld, bei einem Wörterbuch keine Rolle spielt.

Weitere Einträge, die notwendig wurden, sind „dimension“, „optimum“, „discrete_values“ und „magnitudes_of_categorical_variables“. Da ein weiteres Ziel war, dass, neben der Abstraktion, Automatisierung implementiert werden sollte, wurden die JSON-Dateien so konzipiert, dass statische Werte einer Testfunktion, ohne Nutzereingabe, zum Start des Algorithmus ausgelesen werden konnten. Dabei bezeichnet „dimension“ die Dimension der Testfunktion/des Problems. In Kombination mit den „variable_boundaries“ ergibt dies für den Code-Snipped 1 den Definitionsbereich $[-3, 7]$ ¹⁰. Der Eintrag „optimum_argument“ definiert das zum Optimum gehörige Argument, also die notwendige Belegung der unabhängigen Variablen, um das Optimum der Testfunktion zu erhalten, welches im Eintrag „optimum“ ausgelesen werden kann. Die Intervallgrenzen für den Definitionsbereich einer Testfunktion sind unter „variable_boundaries“ hinterlegt und definieren sowohl die Intervalle, in denen eine Lösung liegen muss, um eine gültige Lösung zu sein, als auch die Intervalle, aus denen Initiallösungen während der Initialisierungsmethode $\mathcal{I}$ gezogen werden, wie diese im Abschnitt 4 definiert wurde. Der Eintrag „discrete_values“ listet alle diskreten/ordinalen Werte auf, welche die Testfunktion annehmen kann.

Der letzte Grund, warum das JSON-Format gewählt wurde, wie im Code-Snipped 1 zu sehen, kann das JSON-Format jede Art von Eintrag aufnehmen und somit jede Form von abstrakter Variable beherbergen und damit einen hohen Abstraktionsgrad liefern, der, für auf dieser Arbeit aufbauende Forschung, notwendig ist.

6.1. Stetige Testfunktionen

Aufbauend auf meiner Bachelorarbeit [31] werden die Erweiterungen von $ACO_{\mathbb{R}}$ -VS auf denselben stetigen Testfunktionen getestet, wie der ursprüngliche Algorithmus $ACO_{\mathbb{R}}$ -VS. Diese sind Testfunktion zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen in Tabelle 4 und Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, in Tabellen 5, 6 und 7.

Die Testfunktionen wurden aus verschiedene Quellen entnommen. Die Branin RCOS-Testfunktion stammt aus [50], die Goldstein and Price-Testfunktion aus [51] und die Griewangk-Testfunktion aus [34]. Die Griewangk-Testfunktion wurde auch in den Arbeiten von *Dorigo und Socha* [44] und *Socha* [43] benutzt, auf denen diese Arbeit aufbaut. Aber dort wurde eine andere Version der Griewangk-Testfunktion benutzt, als die Version in anderen Quellen, nämlich *Molga und Smutnicki* [34] und *Pohlheim* [39], die explizit Testfunktionen auflisten, um Algorithmen zu testen. Da in [44] und [43] keine Optima angegeben wurden, wurde in dieser Arbeit sich für die Version der Griewangk-Testfunktion aus [39] und [34] entscheiden. Die Testfunktionen Hartmann stammen aus [47] und [48] und die Shekel-Testfunktionen aus [49]. Alle anderen Testfunktionen stammen aus [44] und [43]. Bei der Hartmann(6,4)-Testfunktion haben sich die Quellen in ihrer Definition des Funktionsterms im Matrixeintrag a_{15} widersprochen. In [44] und [43] wird $a_{15} = 1.5$ angegeben während in [48] angegeben wird, dass $a_{15} = 1.7$. Da nur in [48] das dazugehörige Optimum angegeben wird, wurde in dieser Arbeit sich für die Angabe aus dieser Quelle entschieden. Um zukünftige Forschung zu erleichtern, werden alle Testfunktionen in dieser Arbeit, genau wie in meiner Bachelorarbeit [31], nicht nur mit Name und Funktionsterm, sondern auch mit Definitionsbereich/Initialisierungsintervall, Optimum und Argument des Optimums angegeben.

In meiner Bachelorarbeit stimmten die berechnete Optima für die Funktionen Hartmann(3,4), Hartmann(6,4), Shekel(4,5), Shekel(4,7) und Shekel(4,10) mit denen in der Literatur nicht überein. Der Fehler konnte gefunden werden. Es wurden die Funktionsgleichungen der Testfunktionen in den Quellen falsch interpretiert, weswegen falsche Ergebnisse entstanden. Für Shekel(4,k;k = 5,7,10) lag der Fehler in der Interpretationen der Gleichung 30. Die Gleichung wurde so interpretiert, wie es in der Gleichung 31 geschrieben ist. Aber der Term ist, wie in Gleichung 32 zu interpretieren.

$$f(x) = - \sum_{i=1}^k \left(\sum_{j=1}^4 (x_j - a_{ji})^2 + c_i \right)^{-1} \quad (30)$$

$$f(x) = - \sum_{i=1}^k \left(\sum_{j=1}^4 ((x_j - a_{ji})^2 + c_i) \right)^{-1} \quad (31)$$

$$f(x) = - \sum_{i=1}^k \left(\sum_{j=1}^4 ((x_j - a_{ji})^2) + c_i \right)^{-1} \quad (32)$$

Die Testfunktionen, welche in den Tabellen 4, 5, 6 und 7 gelistet sind, unterscheiden sich nicht nur darin, für welche Vergleiche sie herangezogen werden, sondern auch welches Abbruchkriterium beim Vergleich genutzt wird. Es werden hier die selben Abbruchkriterien wie in meiner Bachelorarbeit [31], sowie wie sie in *Socha und Dorigo* [44] und in *Socha* [43] verwendet werden. Für Testfunktionen aus Tabelle 4 gilt das Abbruchkriterium $|f - f^*| < \varepsilon$, $\varepsilon = 10^{-10}$ und für Testfunktionen aus den Tabellen 5, 6 und 7 gilt das Abbruchkriterium $|f - f^*| < |\varepsilon_1 f| + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$. Dabei ist ε_1 der relative Fehler und ε_2 der absolute Fehler. In beiden Abbruchkriterien steht f für den momentanen Funktionswert und f^* für das bekannte Optimum. Das zweite Abbruchkriterium wird in [44] und [43] als $|f - f^*| < \varepsilon_1 f + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$ und im, in *Socha* [43] bereitgestellten, Quellcode als $|f - f^*| < \varepsilon_1 f^* + \varepsilon_2$ angegeben, aber es musste zu der jetzigen Form verändert werden, da sonst negative Optima nicht erreicht werden können. So wird z.B. das Optimum $f^* = -3.32237$ der Testfunktion Hartmann(6,4) nicht erreicht, da, für einen an das Optimum hinreichend nahen, Funktionswert $f = -3.32237 + \varepsilon_3$, mit $\varepsilon_3 > 10^{-4}$, gilt $|-3.32237 - (f + \varepsilon_3)| = |\varepsilon_3| = \varepsilon_3 > \varepsilon_1 \cdot (-3.32237) + \varepsilon_2$. Eine weitere Herausforderung des Abbruchkriteriums $|f - f^*| < \varepsilon_1 f + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$, ist die Tatsache, dass 6 der in diesem Abschnitt vorgestellten 20 Testfunktionen Optima haben, die mehr als 4 Nachkommastellen besitzen, weswegen, bei der Benutzung des zweiten Abbruchkriteriums, die Güte eines Algorithmus, der auf diesen 6 Testfunktionen getestet wird, besser ausgewertet wird, als sie ist. Denn bei dem zweiten Abbruchkriterium muss der Algorithmus z.B. für $f^* = -3.32237$ nur $f = -3.3223\dots$ erreichen und gerade die letzte Nachkommastelle oder Nachkommastellen, wenn es mehrere sind, können darüber entscheiden, wie viele Iterationen ein Algorithmus braucht, um das Optimum zu erreichen bzw. ob das Optimum überhaupt gefunden wird oder nur ein Wert, der nahe dem Optimum ist.

Ferner noch auf das Initialisierungsintervall eingegangen werden. Das Initialisierungsintervall ist das Intervall aus dem, bei der Initialisierung des Algorithmus, die ersten $k \in \mathbb{N}$ zufälligen Lösungen gezogen werden aus denen dann andere Lösungen konstruiert werden. Dabei kann $k = 13$ sein, wie bei der Initialisierung des HSPPBO-Schema in Abschnitt 4, $k = 50$, wie bei der Initialisierung von ACO_ℝ in Abschnitt 2.9 oder $k = 100$, wie bei der Initialisierung von ACO_ℝ-VS im Abschnitt 2.11. Auch wenn das Initialisierungsintervall bei ACO_ℝ und ACO_{MV} nach der Initialisierung nicht mehr verwendet wird, ist es trotzdem wichtig, da, wenn es schlecht gewählt wurde, die Initiaillösungen

so liegen, dass die Algorithmusleistung zum negativen oder zum positiven verzerrt wird. Die Initialisierungsintervalle für die Testfunktionen aus den Tabellen 5, 5, 6 und 7 müssen aber nicht näher betrachtet werden, da mit den Initialisierungsintervallen für diese Testfunktionen sich schon in [44] und [43] auseinander gesetzt wurde. Deswegen können sie übernommen werden. Eine offene Forschungsfrage ist, wie Optimierungsalgorithmen auf nicht zusammenhängende Definitionsbereiche reagieren.

Wenn es Nebenbedingungen/Constraints gibt, kann das vollständige Initialisierungsintervall sich vom Initialisierungsintervall, das durch Anwenden der Nebenbedingungen entsteht, stark unterscheiden, wie im Abschnitt 6.2.2. Ein solchen Intervall wird hier als gefiltert bezeichnet.

Genau wie in meiner Bachelorarbeit [31] werden die Initialisierungsintervalle hier als Definitionsbereiche bezeichnet. Das liegt daran, dass die Funktionen, sowohl initial über diesen Intervallen definiert sind, als auch die Intervallgrenzen definieren, was gültige Lösungen sind. Der Suchraum für die Funktionen aus den Tabellen 4, 5, 6 und 7 ist zwar $\Sigma = \mathbb{R}^n$, wobei $n \in \mathbb{N}$ von der jeweiligen Testfunktion abhängt, aber die Beschränkungen $\mathcal{B}$ sind die Definitionsbereiche, die für die verschiedenen Testfunktionen abhängig von der Testfunktion definiert sind, z.B. für die Testfunktion „Paraboloid“ gilt $\mathcal{B}_1 = \dots = \mathcal{B}_n = [-3, 7]$

Schließlich werden noch 6 ausgewählte Testfunktionen in den Abschnitten 6.1.1 bis 6.1.6 genauer betrachtet. Diese Testfunktionen haben sich nicht nur durch besondere Herausforderungen, die sie an einen Optimierungsalgorithmus stellen, heraus getan, sondern auch dadurch, dass die Herausforderung, die sie bieten, singulär ist, sie also den Algorithmus auf nur eine Weise fordern und damit sich eignen den Algorithmus auf eine Stärke oder eine Schwäche zu testen und werden deswegen näher betrachtet. Alle Abbildungen in den Abschnitten 6.1.1 bis 6.1.6 wurden von mir mit den Python-Bibliotheken „numpy“, „matplotlib“, „matplotlib.pyplot“ und „cm“, aus „matplotlib“ erstellt. Es wurden für alle Abbildungen in den Abschnitten 6.1.1 bis 6.1.6 die selbe oder eine ähnliche Bildausrichtung, wie in [34] und die selbe Farbpalette, wie in [24], verwendet, da die Kombination von beiden gute Bilder ergeben hat.

Tabelle 4: Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen

Funktion, Definitionsbereich, Optimum	Formel
Cigar, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$ 10^{10} bei mehreren	$f(x) = x_1^2 + 10^4 \sum_{i=2}^n x_i^2$
Ellipsoid, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n \left(100^{\frac{i-1}{n-1}} x_i \right)^2$
Paraboloid, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$ 10^{10} bei $x_1 = 10^{10}$	$f(x) = \sum_{i=1}^n x_i^2$
Rosenbrock, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [1, 1, 1, \dots]$	$f(x) = \sum_{i=1}^{n-1} 100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2$
Tablet, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = 10^4 x_1^2 + \sum_{i=2}^n x_i^2$

Tabelle 5: erster Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden

Funktion, Definitionsbereich, Optimum	Formel
B_2 , $\vec{x} \in [-100, 100]^n, n = 2$, 0 bei $\vec{x} = [0, 0]$	$f(x) = x_1^2 + 2x_2^2 - \frac{3}{10}\cos(3\pi x_1) - \frac{2}{5}\cos(4\pi x_2) + 7/10$
Branin RCOS, $\vec{x} \in [-5, 15]^n, n = 2$, 0.397887 bei $\vec{x} = [-\pi, 12.275]$ od. $x = [\pi, 2.275]$ od. $x = [9.42478, 2.475]$	$f(x) = (x_2 - \frac{5}{4\pi}x_1^2 + \frac{5}{\pi}x_1 - 6)^2 + 10(1 - \frac{1}{8\pi})\cos(x_1) + 10$
Easom, $\vec{x} \in [-100, 100]^n, n = 2$, 0 bei $\vec{x} = [\pi \cdot k + \pi/2]$, $\pi \cdot k + \pi/2], k \in \mathbb{Z}$	$f(x) = -\cos(x_1) \cos(x_2) e^{-(x_1 - \pi)^2 - (x_2 - \pi)^2}$
De Jong, $\vec{x} \in [-5.12, 5.12]^n, n = 3$, 0 bei $\vec{x} = [0, 0, 0]$	$f(x) = x_1^2 + x_2^2 + x_3^2$
Goldstein, $\vec{x} \in [-2, 2]^n$ and Price, $n = 2$, 3 bei $\vec{x} = [0, 1]$	$f(x) = (1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 13x_1^2 - 14x_2 + 6x_1x_2 + x_2^2))(30 + (2x_1 - 3x_2)^2 \cdot (18 - 32x_1 + 12x_1^2 - 48x_2 - 36x_1x_2 - 27x_2^2))$
Griewangk, $\vec{x} \in [-600, 600]^n$, $n = 10$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n \frac{x_i^2}{4000} - \prod_{i=1}^n \cos(\frac{x_i}{\sqrt{i}}) - 1$
Martin and Gaddy, $\vec{x} \in [-20, 20]^n, n = 2$ 0 bei $\vec{x} = [5, 5]$	$f(x) = (x_1 - x_2)^2 + \left(\frac{x_1 + x_2 - 10}{3}\right)^2$
Paraboloid(6), $\vec{x} \in [-5.12, 5.12]^n$, $n=6$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n x_i^2$
Rosenbrock(2,5), $\vec{x} \in [-5, 5]^n$, $n = 2, 5$; 0 bei $\vec{x} = [1, 1, 1, \dots]$	$f(x) = \sum_{i=1}^{n-1} 100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2$
Zakharov, $\vec{x} \in [-5, 10]^n$, $n = 2, 5$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{j=1}^n x_j^2 + \left(\sum_{j=1}^n \frac{jx_j^2}{2}\right)^2 + \left(\sum_{j=1}^n \frac{jx_j^2}{2}\right)^4$

Bei Funktionen, welche in mehreren Dimensionen getestet wurden, stehen die getesteten Dimensionen in Klammern hinter der Funktion. Dasselbe gilt für Funktionen, die sich in der getesteten Dimension von der Dimension aus Tabelle 4 unterscheiden.

Tabelle 6: zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden

Funktion	Formel
Hartmann(3,4)	$f_{H_{3,4}}(x) = - \sum_{i=1}^4 c_i e^{-\sum_{j=1}^3 a_{ij}(x_j - p_{ij})^2}$ $a_{ij} = \begin{pmatrix} 3.0 & 10.0 & 30.0 \\ 0.1 & 10.0 & 35.0 \\ 3.0 & 10.0 & 30.0 \\ 0.1 & 10.0 & 35.0 \end{pmatrix} \quad c_i = \begin{pmatrix} 1.0 \\ 1.2 \\ 3.0 \\ 3.2 \end{pmatrix}$ $p = \begin{pmatrix} 0.3689 & 0.1170 & 0.2673 \\ 0.4699 & 0.4387 & 0.7470 \\ 0.1091 & 0.8732 & 0.5547 \\ 0.0381 & 0.5743 & 0.8828 \end{pmatrix}$
Hartmann(6,4)	$f_{H_{3,4}}(x) = - \sum_{i=1}^4 c_i e^{-\sum_{j=1}^6 a_{ij}(x_j - p_{ij})^2}$ $a_{ij} = \begin{pmatrix} 10.0 & 3.0 & 17.0 & 3.5 & 1.7 & 8.0 \\ 0.05 & 10.0 & 17.0 & 0.1 & 8.0 & 14.0 \\ 3.0 & 3.5 & 1.7 & 10.0 & 17.0 & 8.0 \\ 17.0 & 8.0 & 0.05 & 10.0 & 0.1 & 14.0 \end{pmatrix} \quad c_i = \begin{pmatrix} 1.0 \\ 1.2 \\ 3.0 \\ 3.2 \end{pmatrix}$ $p = \begin{pmatrix} 0.1312 & 0.1696 & 0.5569 & 0.0124 & 0.8283 & 0.5886 \\ 0.2329 & 0.4135 & 0.8307 & 0.3736 & 0.1004 & 0.9991 \\ 0.2348 & 0.1451 & 0.3522 & 0.2883 & 0.3047 & 0.6650 \\ 0.4047 & 0.8828 & 0.8732 & 0.5743 & 0.1091 & 0.0381 \end{pmatrix}$
Shekel(4,k; k = 5,7,10)	$f(x) = - \sum_{i=1}^k (\sum_{j=1}^4 (x_j - a_{ji})^2 + c_i)^{-1}$ $a_{ij} = \begin{pmatrix} 4.0 & 4.0 & 4.0 & 4.0 \\ 1.0 & 1.0 & 1.0 & 1.0 \\ 8.0 & 8.0 & 8.0 & 8.0 \\ 6.0 & 6.0 & 6.0 & 6.0 \\ 3.0 & 7.0 & 3.0 & 7.0 \\ 2.0 & 9.0 & 2.0 & 9.0 \\ 5.0 & 5.0 & 3.0 & 3.0 \\ 8.0 & 1.0 & 8.0 & 1.0 \\ 6.0 & 2.0 & 6.0 & 2.0 \\ 7.0 & 3.6 & 7.0 & 3.6 \end{pmatrix}^T \quad c_i = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.2 \\ 0.4 \\ 0.4 \\ 0.6 \\ 0.3 \\ 0.7 \\ 0.5 \\ 0.5 \end{pmatrix}$

Tabelle 7: zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden

Funktion, Definitionsbereich	Optimum
Hartmann(3,4), $\vec{x} \in [0, 1]^n, n = 3$	-3.86278 bei $\vec{x} = [0.114614, 0.555649, 0.852547]$
Hartmann(6,4), $\vec{x} \in [0, 1]^n, n = 6$	-3.32237 bei $\vec{x} = [0.20169, 0.150011, 0.476874,$ $0.275332, 0.311652, 0.6573]$
Shekel(4,5) $\vec{x}, \in [0, 1]^n, n = 4$	-10.1532 bei $\vec{x} = [4, 4, 4, 4]$
Shekel(4,7) $\vec{x}, \in [0, 1]^n, n = 4$	-10.4029 bei $\vec{x} = [4, 4, 4, 4]$
Shekel(4,10) $\vec{x}, \in [0, 1]^n, n = 4$	-10.5364 bei $\vec{x} = [4, 4, 4, 4]$

6.1.1. Testfunktion - Paraboloid

Die Paraboloid-Testfunktion ist unimodal und konvex. Sie ist die einfachste aller einfachen Testfunktion und somit insgesamt die einfachste Testfunktion. Sie dient sowohl als Demonstration, dass der Algorithmus, welcher getestet wird, ordnungsgemäß funktioniert, als auch zum Testen der Konvergenzgeschwindigkeit des Algorithmus. Konvergiert ein Algorithmus zu schnell, kann er ein globales Optimum überspringen, konvergiert er zu langsam, erreicht er das globale Optimum nicht in der vorgegebenen Menge an Ressourcen, wobei Ressourcen maximale Ausführungszeit oder maximale Anzahl an Funktionsausführungen sein können. Letzteres ist für die Testfunktion Rosenbrock eingetreten, als die Initialversion des ACO_ℝ-VS Algorithmus aus meiner Bachelorarbeit [31] versucht hat diese zu optimieren. Ferner haben *Merkle und Middendorf* [33] gezeigt, dass die Untersuchung einfacher Testfunktionen ein lohnendes Unterfangen sein kann, da einfache Testfunktionen ein unvorhergesehenes Verhalten des Algorithmus aufdecken können.

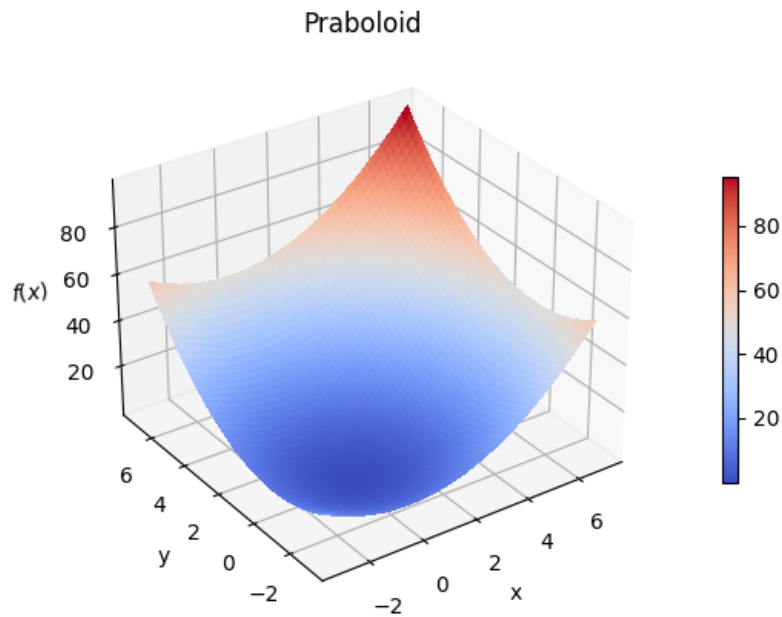


Abbildung 2: Paraboloid - Visualisierung in 3D

6.1.2. Testfunktion - Easom

Wie in [34] beschrieben, ist die Easom-Testfunktion deswegen interessant, weil die Umgebung um das globale Minimum klein ist im Verhältnis zur Größe des gesamten Suchraum. Der Suchraum ist das Quadrat $[-100, 100] \times [-100, 100]$ in der Abbildung 3 links, während das Optimum im Quadrat $[1, 5] \times [1, 5]$ sich befindet, in der Abbildung 3 rechts.

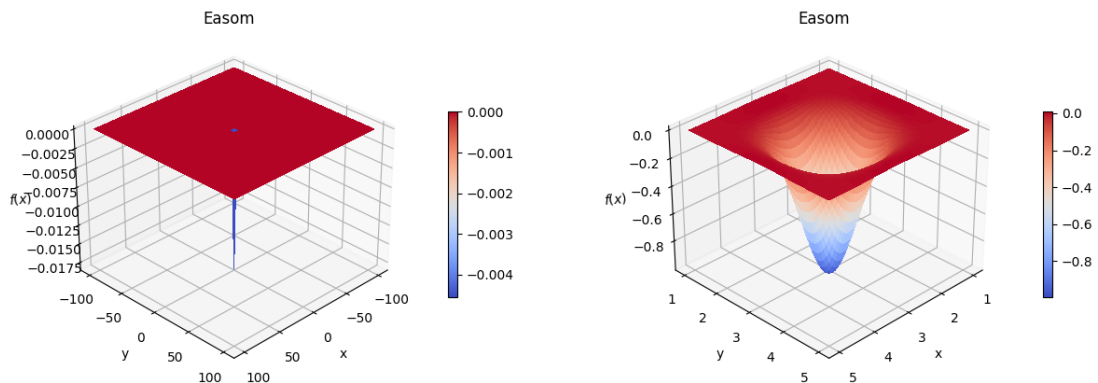


Abbildung 3: Easom-Testfunktion: Vergleich zwischen gesamtem Suchraum (links) und Umgebung um das Optimum (rechts)

6.1.3. Testfunktion - Griewangk

Wie in [34] beschrieben ist die Griewangk-Testfunktion multimodal und hat damit viele lokale Optima, was die Suche nach dem globalen für Algorithmen erschwert. Ferner sind die lokalen Minima zwar gleich verteilt aber weit gestreut, was die Suche nach dem globalen Optima weiter verkompliziert. Die Griewangk-Testfunktion wurde auch deswegen gewählt, da, wie in Abbildung 4 zusehen, sich das Aussehen der Funktion in Abhängigkeit vom Betrachtungsintervall ändert. Im Intervall $[-600, 600]^n$, $n = 2$, was die 2-dimensionale Version des Initialisierungsintervall $[-600, 600]^n$, $n = 10$ ist, welches für die Optimierung verwendet wird, muss der Algorithmus schnell konvergieren, um nicht viele Iterationen mit schlechten Werten zu verbrauchen. Im Intervall $[-10, 10]^n$, $n = 2$, muss der Algorithmus Variabilität zeigen, um die große Menge an lokalen Minima zu meiden. Und schließlich im Intervall $[-4, 4]^n$, $n = 2$ muss der Algorithmus eine aperiodische, zerklüftete Landschaft manövrierern, um das globale Optimum zu finden. Deswegen eignet sich die Griewangk-Testfunktion besonders gut, um das Können eines Algorithmus zu testen.

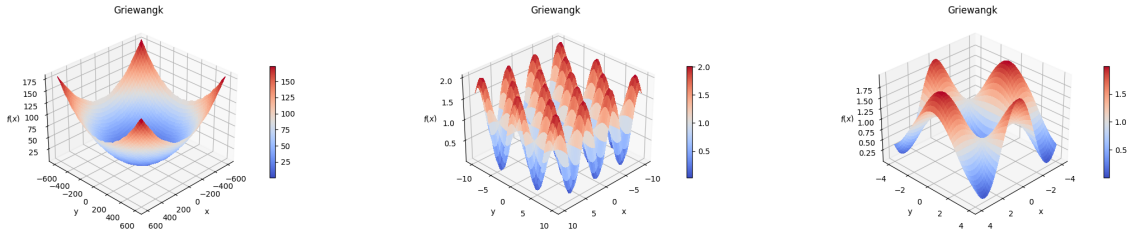


Abbildung 4: Verschiedene Auflösungen der Griewangk-Testfunktion

6.1.4. Testfunktion - Rosenbrock

Wie in [34] beschrieben besteht die Rosenbrock-Testfunktion dadurch, dass ihr Optimum in einem langen, engen Tal liegt. Es ist leicht das Tal zu finden aber es ist sehr schwer zum Optimum zu konvergieren. Die Rosenbrock-Testfunktion ist interessant, da sie sehr flach nahe dem Optimum ist und die Konvergenz langsam vorangeht. Die Rosenbrock-Testfunktion wurde auch aus dem Grund gewählt, weil in meiner Bachelorarbeit [31], unter den Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen, den Testfunktionen aus Tabelle 4, es die einzige Testfunktion war, bei welcher der $ACO_{\mathbb{R}}$ -very-simple nicht der beste Algorithmus war oder zumindest sehr gut. Bei dieser Testfunktion konnte $ACO_{\mathbb{R}}$ -VS das Optimum nicht finden. Deswegen ist diese Testfunktion besonders interessant, um die Variationen von $ACO_{\mathbb{R}}$ -VS zu testen.

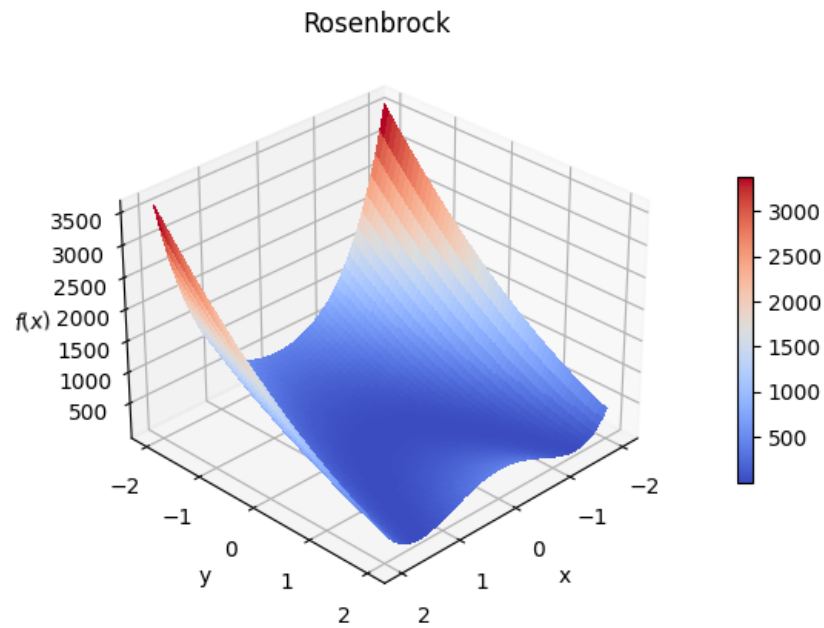


Abbildung 5: Rosenbrock - Visualisierung in 3D

6.1.5. Testfunktion - Hartmann

Die Familie der Hartmann-Testfunktionen ist eine der beiden Familien der komplexen Testfunktionen, die in dieser Arbeit betrachtet werden. Diese Familie zeichnet sich dadurch aus dass sie die leichtere der beiden komplexen Testfunktionfamilien ist. Sie ist multimodal mit 4 lokalen Minima im Fall von Hartmann(3,4), und respektive 6 lokalen Minima im Fall von Hartmann(6,4).

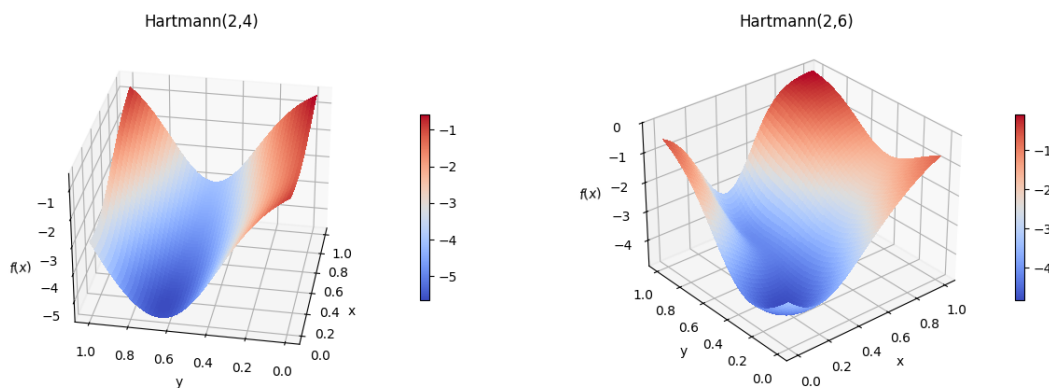


Abbildung 6: Hartmann-Testfunktionen: Hartmann(2,4) (links) und Hartmann(2,6) (rechts)

6.1.6. Testfunktion - Shekel

Die Familie der Shekel-Testfunktionen ist die schwerere der beiden komplexen Testfunktionfamilien, die in dieser Arbeit betrachtet werden. Sie ist multimodal und hat 5 lokale Minima im Fall von Shekel(4,5), 7 lokale Minima im Fall von Shekel(4,7) und respektive 10 lokale Minima im Fall von Shekel(4,10).

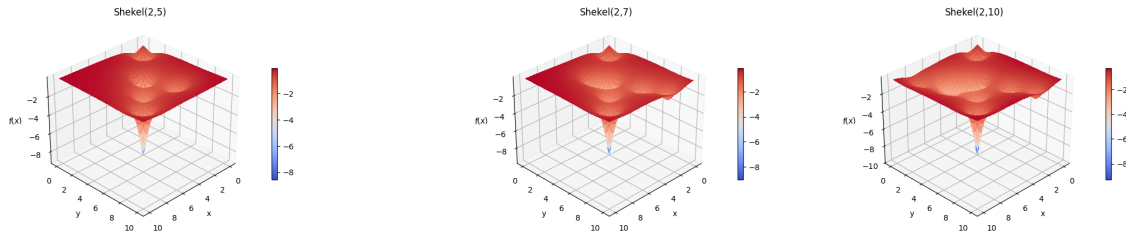


Abbildung 7: Shekel-Testfunktionen: Shekel(2,5) (links), Shekel(2,7) (mittig) und Shekel(2,10) (rechts)

6.2. Testfunktionen mit gemischten Variablen

6.2.1. Konstruierte Testfunktionen

Neue Testfunktionen mit gemischten Variablen wurden erzeugt, indem die stetigen Testfunktionen aus Tabelle 4, außer „Rosenbrock“, genommen wurden und ein Teil ihrer Dimensionen zu diskreten Dimensionen transformiert wurde. Im JSON-Format wird dies wie im Code-Snipped 2 angezeigt.

Code-Snipped 2: Paraboloid 5 Dim gemischte Variablen

```
1 {
2     "name": "Paraboloid_5_11_MV",
3     "comment": "artificial mixed_variable problem",
4     "type": "mixed variable problems",
5     "continuous_dimension": 3,
6     "discrete_dimension": 2,
7     "dimension": 5,
8     "optimum_argument": [[0.0], [0.0], [0.0], [0.0], [0.0]],
9     "optimum": [0.0],
10
11     "variable_boundaries": {
12         "natural_numbers": [],
13         "discrete": [[-3, 7]],
14         "continuous": [[-3, 7]],
15         "ordinal": [],
16         "categorical": []
```

```

17     },
18
19     "discrete_values": [-3.0, -2.0, -1.0, 0.0, 1.0,
20                        2.0, 3.0, 4.0, 5.0, 6.0,
21                        7.0],
22
23     "magnitudes_of_categorical_variables": []
24 }

```

Es wurden die Testfunktionen „Cigar“, „Ellipsoid“, „Paraboloid“ und „Tablet“ aus Tabelle 4 gewählt, da sie sowohl für verschiedene Dimensionen, diskret und stetig, definiert werden konnten, als auch wo die Algorithmen, bis auf wenige Ausnahmen, konvergieren. Im Vergleich werden Kombination aus verschiedenen Anzahlen an Dimensionen und verschiedenen Anzahlen diskreten Punkten untersucht. Jede der Testfunktionen wird mit den Dimensionen $n = \{2, 5, 10\}$ und mit der Anzahl an diskreten Punkten $m \in \{11, 21, 51\}$ getestet. Dabei wird jede der Dimensionen mit allen Anzahlen an diskreten Punkten getestet, also ergeben sich $(2, 11), (2, 21), (2, 51), \dots, (10, 11), (10, 21), (10, 51)$ als Kombinationen pro Testfunktion. Zusätzlich wird jede Funktion mit den beiden Neustartstrategien *fixed* = $n, n \in \mathbb{N}$ und *adaptive* getestet, wobei $n = 0$ gesetzt wurde. Damit ergeben sich 72 konstruierte Testfunktionen, wie in den Tabellen 21 und 22 zu sehen. Testfunktionen, deren Dimension 5 ist, haben als stetige Dimensionen 3 und als diskrete Dimensionen 2, wie im Code-Snipped 2 zu sehen. Die Dimension von Testfunktionen, deren Dimension gerade ist, also 2 oder 10, setzt sich zusammen aus Hälfte diskreter Dimensionen und aus Hälfte stetiger Dimensionen. Das Abbruchkriterium für alle die so konstruierten Testfunktionen ist das selbe, wie für Testfunktionen aus Tabelle 4, nämlich $|f - f^*| < \varepsilon$, $\varepsilon = 10^{-10}$. Es wurde das selbe Abbruchkriterium verwendet, um Vergleichbarkeit zu ermöglichen.

6.2.2. Coil Spring Design Problem (CSD)

Das Coil Spring Design Problem (CSD) kann auf verschiedene Arten definiert werden, wie in [41], [18] oder später, aufbauend auf [41], in [29]. In dieser Arbeit wird die Definition aus [29] verwendet. Im CSD Problem, wie es in [29] definiert wird, wird so nach dem Außendurchmesser D , dem Drahtdurchmesser d und der Windungszahl N einer Schraubenfeder (Coil Spring), wie sie in Abbildungen 8 skizzenhaft dargestellt ist, gesucht, dass der Materialbedarf für die Schraubenfeder minimiert wird, während gleichzeitig die Nebenbedingungen (Constraints) erfüllt werden.

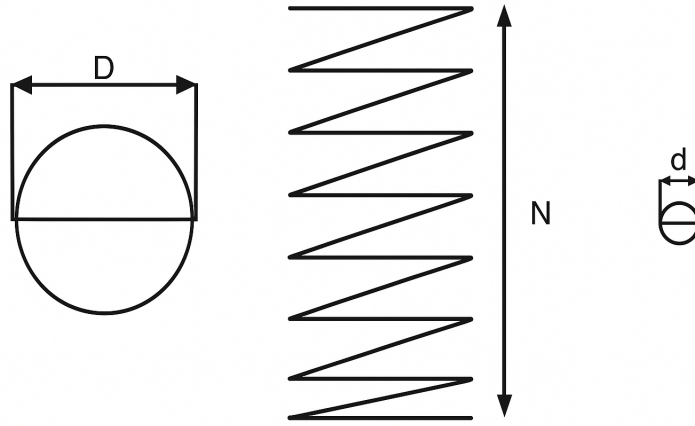


Abbildung 8: Skizze einer Schraubenfeder mit Federdurchmesser (links), Windungszahl (mitte) und Drahtdurchmesser (rechts)“; das Bild wurde von mir mit einem Zeichenprogramm erstellt und dann mit ChatGPT verbessert

Da in *Sandgren* [41] das CSD Problem initial in imperialen Maßeinheiten definiert wurde, verwendeten alle Folgestudien, wie [10], [28], [18], [16], [22] oder [29], ebenfalls imperiale Maßeinheiten, um die Ergebnisse vergleichen zu können. Deswegen werden auch in dieser Arbeit imperiale Maßeinheiten verwendet.

Das Problem zählt zu Problemen mit gemischten Variablen, da D eine stetige und d eine ordinale Variable ist. N ist eine Variable, die nur natürliche Zahlen als Werte annehmen kann.

Mögliche Werte, die der Drahtdurchmesser d annehmen kann, sind in der Tabelle 8 aufgelistet.

Tabelle 8: Standarddrahtdurchmesser für die Schraubenfeder angegeben in Zoll (inch)

0.0090	0.0095	0.0104	0.0118	0.0128	0.0132
0.0140	0.0150	0.0162	0.0173	0.0180	0.0200
0.0140	0.0150	0.0162	0.0173	0.0180	0.0200
0.0470	0.0540	0.0630	0.0720	0.0800	0.0920
0.1050	0.1200	0.1350	0.1480	0.1620	0.1770
0.1920	0.2070	0.2250	0.2440	0.2630	0.2830
0.3070	0.3310	0.3620	0.3940	0.4375	0.5000

Weiter werden in [29] Intervallgrenzen definiert, die in der Tabelle 9 aufgeführt werden. Es fällt auf, dass durch die Definition der Intervallgrenze $d_{min} = 0.2$ Zoll (inch), 31

der 42 möglichen Werte, welche die diskrete Variable d annehmen kann, wegfallen, aber dies ist so in *Sandgren* [41] angegeben und die Folgequellen [10], [28], [18], [16], [22] und [29] verwenden die Intervallgrenze direkt oder verweisen auf *Sandgren* [41]. Keine der Quellen gibt eine Begründung an. Um Vergleichbarkeit zu garantieren, wird $d_{min} = 0.2$ beibehalten.

Tabelle 9: Intervallgrenzen

maximale Arbeitsbelastung	$F_{max} = 1000.0$ Pfund (lb)
zulässige maximale Scherspannung	$S = 189000.0$ Pfund pro Quadratzoll (psi)
maximale freie Länge	$l_{max} = 14.0$ Zoll (inch)
Minstdrahtdurchmesser	$d_{min} = 0.2$ Zoll (inch)
maximaler Außendurchmesser der Feder	$D_{max} = 3.0$ Zoll (inch)
Vorspannkompressionskraft	$F_p = 300.0$ Pfund (lb)
zulässige maximale Durchbiegung unter Vorspannung	$\sigma_{pm} = 6.0$ Zoll (in)
Auslenkung von der Vorlastposition zur Maximallastposition	$\sigma_w = 1.25$ Zoll (in)
Schubmodul des Materials	$G = 11.5 \cdot 10^6$

Die Nebenbedingen umfassen primäre und sekundäre Variablen. Die primären Variablen sind D , d und N . Die sekundären Variablen sind C_f , K , σ_p und l_f . Die sekundären Variablen ergeben sich aus den primären Variablen und den Intervallgrenzen und werden in den Gleichungen 33 bis 36 definiert. Die Definitionsbereiche der primären Variablen können der Tabelle 10 entnommen werden.

Tabelle 10: Definitionsbereiche

D	{3d, 3.0}
d	{ d_{min} , 0.5000}
N	{1, 70}

$$C_f = \frac{4\frac{D}{d} - 1}{4\frac{D}{d} - 4} + \frac{0.615d}{D} \quad (33)$$

$$K = \frac{Gd^4}{8ND^3} \quad (34)$$

$$\sigma_p = \frac{F_p}{K} \quad (35)$$

$$l_f = \frac{F_{max}}{K} + 1.05(N + 2)d \quad (36)$$

Ferner unterliegen die Variablen im CSD 8 Constrains c_i , $i \in \{1, \dots, 8\}$. Diese sind in Tabelle 11 dargelegt.

Tabelle 11: Constrains

1	$c_1 = \frac{8C_f F_{max} D}{\pi d^3} - S \leq 0$
2	$c_2 = l_f - l_{max} \leq 0$
3	$c_3 = d_{min} - d \leq 0$
4	$c_4 = D - D_{max} \leq 0$
5	$c_5 = 3.0 - \frac{D}{d}$
6	$c_6 = \sigma_p - \sigma_{pm} \leq 0$
7	$c_7 = \sigma_p + \frac{F_{max} - F_p}{K} + 1.05(N + 2)d - l_f \leq 0$
8	$c_8 = \sigma_w - \frac{F_{max} - F_p}{K} \leq 0$

Der Materialbedarf und damit die Zielfunktion wird mit Hilfe der Formel 37 berechnet.

$$f(N, D, d) = \frac{\pi^2 D d^2 (N + 2)}{4} \quad (37)$$

Schließlich wird das Abbruchkriterium als $|f - f^*| < |\varepsilon_1 f| + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-6}$ gewählt. Dieses Abbruchkriterium wird gewählt, da davon ausgegangen wird, dass in *Socha* [43] das selbe Kriterium benutzt wurde und es eine Erweiterung des Abbruchkriteriums $|f - f^*| < |\varepsilon_1 f| + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$ ist, welches für den Großteil der stetigen Testfunktionen in dieser Arbeit verwendet wurde. Leider ist in *Socha* [43] das Abbruchkriterium für das CSD Problem nicht angegeben.

6.2.2.1. Vollständiger Raum als Initialisierungsintervall

Wie bereits im Abschnitt 6 beschrieben, hat das Initialisierungsintervall großen Einfluss auf den Erfolg des Algorithmus. So können für das CSD Anfangslösungen aus dem gesamten Raum $\{3 \cdot d_{min}, 3.0\} \times \{d_{min}, 0.5000\} \times \{1, 70\}$ oder dem gefilterten Raum gezogen werden.

Nimmt man den gesamten Raum zum ziehen der Initiallösungen, so ändert sich die Zielfunktion von der Formel 37 zu der Formel 38, wie in [29] definiert.

$$\hat{f} = f \prod_{i=1}^8 \hat{c}_i^3 \quad (38)$$

Dabei werden die $\hat{c}_i$'s in der Formel 39 definiert.

$$\hat{c}_i = \begin{cases} 1 + w_i c_i, & \text{wenn } c_i > 0 \\ 1, & \text{sonst} \end{cases} \quad (39)$$

Die Gewichte w_i , $i \in \{1, \dots, 8\}$ wurden aus [29] entnommen und befinden sich in der Tabelle 12.

Tabelle 12: Gewichte

1	$w_1 = 10^{-5}$
2	$w_2 = 1$
3	$w_3 = 10^2$
4	$w_4 = 1$
5	$w_5 = 10^2$
6	$w_6 = 1$
7	$w_7 = 10^2$
8	$w_8 = 10^2$

6.2.2.2. Gefilterter Raum als Initialisierungsintervall

Benutzt man den gefilterten Raum, kann die Formel 37 ohne Veränderungen beibehalten werden. Abbildung 9 zeigt, wie stark sich der Initialisierungsraum verkleinert und damit die Lösungssuche erleichtert wird, wenn der Initialisierungsraum vor dem Ausführen des Algorithmus nach gültigen Lösungen gefiltert wird. Wie zu sehen verkleinert sich der Initialisierungsraum um mehr als das achtfache. Alle Abbildungen in diesem Abschnitt von mir mit den Python-Bibliotheken „numpy“ und „matplotlib“ erstellt.

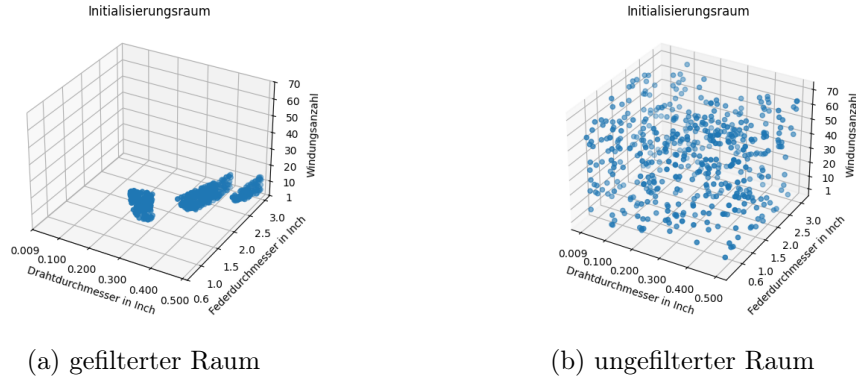


Abbildung 9: gefilterter und ungefilterter Suchraum für das CSD-Problem

Um zu zeigen, wie stark der Initialisierungsraum sich verkleinert, werden die beiden Initialisierungsräume, wie in Abbildung 10, übereinander gelegt.

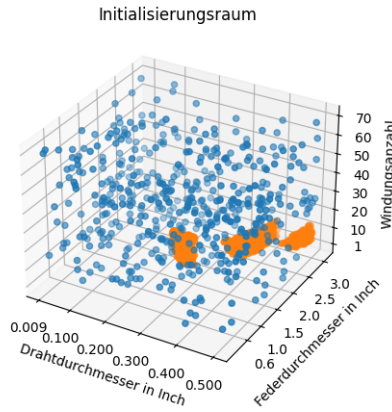


Abbildung 10: gefilterter Raum (orange) und ungefilterter Raum (blau)

6.2.2.3. Resultierende Testkombinationen für das CSD Problem

Wie dargelegt, kann der Suchraum genommen werden ohne Filterung und mit Filterung. Zusätzlich kann die Zielfunktion exakt formuliert werden, wie in Gleichung 37 und mit Straftermen, wie in Gleichung 38. Schließlich werden noch zwei Neustartstrategien, *fixed* = 0 und *adaptvie*, betrachtet. Damit ergeben sich 8 Kombinationen die getestet werden, wie in den Tabellen 24 und 25 zu sehen.

7. Vergleich von $\text{ACO}_{\mathbb{R}}$, ACO_{MV} und den Varianten von $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable

Alle Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV können mit gemischten Variablen umgehen. Aus Platzgründen wurde in den Tabellen das „-MV“ weggelassen.

7.1. Vergleich bzgl. stetiger Testfunktionen

Da in *Socha* [43] ACO_{MV} erst dann eingeführt wird, wenn Testfunktionen mit gemischten Variablen eingeführt werden, kann in diesem Abschnitt nur mit $\text{ACO}_{\mathbb{R}}$ aus [43] verglichen werden. Ferner werden in diesem Abschnitt nur Minimierungstestfunktionen betrachtet, also Testfunktionen, bei denen das globale Minimum gesucht ist, da die Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV nicht mit Testfunktionen umgehen können, bei denen das Optimum außerhalb des Definitionsbereichs liegt, was bei den Maximierungstestfunktionen „Diagonale Plane“ und „Plane“ der Fall ist. Damit entfallen die beiden Maximierungstestfunktionen „Diagonale Plane“ und „Plane“. Alle Testfunktionen wurden 50 mal durchlaufen.

Wie in den Tabellen 13 und 14 zu sehen, ist $\text{ACO}_{\mathbb{R}}$ -VS der beste Algorithmus in 3 von 5 Testfunktionen und sehr gut in Einer. Nur bei der Testfunktion „Rosenbrock“ konvergiert $\text{ACO}_{\mathbb{R}}$ -VS nicht. Die Leistung von $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 sind dagegen schlechter. Die Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-adapt sind nicht aufgeführt, da keine der Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-adapt bei keiner der Testfunktionen aus den Tabellen 13 und 14 konvergierte. Positiv fallen zudem die Algorithmen $\text{ACO}_{\mathbb{R}}$ und CMA-ES auf, da ihre relative Anzahl an Funktionsauswertungen nur wenig schlechter sind, als der Bestwert, außer bei der Testfunktion „Ellipsoid“.

Es folgt, dass unter den Algorithmen, die probabilistisches Lernen einsetzen, $\text{ACO}_{\mathbb{R}}$ -VS der bessere Algorithmus ist, knapp gefolgt von $\text{ACO}_{\mathbb{R}}$ und CMA-ES. Auf der anderen Seite, gemessen daran, dass die Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV, unabhängig der Neustartstrategie, sowohl komplexer als $\text{ACO}_{\mathbb{R}}$ -VS sind, als auch schlechter bzw. viel schlechter, sofern sie konvergieren, schlussfolgern wir, dass die Erweiterung von $\text{ACO}_{\mathbb{R}}$ -VS zu den Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV, bei Testfunktionen, welche zum Vergleich für Algorithmen, die probabilistisches Lernen einsetzen, genommen werden, keinen Mehrwert brachte.

Tabelle 13: Vergleich der Ergebnisse zwischen Algorithmen, die probabilistisches Lernen einsetzen, basierend auf *Socha und Dorigo* [44], und damit auf *Kern et al., 2004* [23]

Testfunktion	(1 + 1)ES	CSA-ES	CMA-ES	IDEA	MBOA	Bestwert
Cigar	1149.9 (2342400)	1508.1 (3072000)	1.9 (3840)	8.7 (17664)	22.6 (46080)	1.0 (2037)
Ellipsoid	456.1 (293700)	760.1 (489500)	6.9 (4450)	11.1 (7120)	96.7 (62300)	1.0 (644)
Paraboloid	1.0 (1370)	1.6 (2192)	1.3 (1781)	5.0 (6850)	48 (65760)	1.0 (1370)
Rosenbrock	*51 (366690)	180 (1294200)	1.0 (7190)	*210 (1509900)	*1100 (7909000)	1.0 (7190)
Tablet	67.4 (118082)	95.3 (166855)	2.5 (4364)	4.3 (7445)	35.2 (61608)	1.0 (1751)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo ein * steht, wurde die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht und dort wo ∞ steht wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Tabelle 14: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}$ -very-simple und den Varianten von $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable, mit Neustartstrategie *fixed* = 0 (NSS-f0), auf Testfunktionen, die zum Vergleich zwischen Algorithmen, die probabilistisches Lernen einsetzen, benutzt werden, basierend auf meiner Bachelorarbeit [31], *Socha und Dorigo* [44] und damit auf *Kern et al., 2004* [23]

Testfunktion	$\text{ACO}_{\mathbb{R}}$	$\text{ACO}_{\mathbb{R}}$ -VS	Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HSPPB- $\text{ACO}_{\mathbb{R}}$ -VS	Bestwert
Cigar	2.6 (5376)	1.0 (2037)	17.6 (35774)	7.6 (15452)	7.3 (14823)	1.0 (2037)
Ellipsoid	18 (11570)	1.0 (644)	17.5 (11279)	13.8 (8912)	15.5 (9957)	1.0 (644)
Paraboloid	1.1 (1570)	1.1 (1483)	3.5 (4834)	3.6 (4953)	3.9 (5326)	1.0 (1370)
Rosenbrock	*1.1 (7909)	∞	∞	∞	∞	1.0 (7190)
Tablet	1.5 (2567)	1.0 (1751)	3.8 (6653)	5.3 (9323)	4.3 (7457)	1.0 (1751)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo ein * steht, wurde die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht und dort wo ∞ steht wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in den Tabellen 15, 16 und 17 zu sehen, im Gegenteil zu den Testfunktionen, die zum Vergleich zwischen Algorithmen, die probabilistisches Lernen einsetzen, benutzt werden, ist es bei Testfunktionen, die benutzt werden, um Algorithmen, die verwandt zu Ameisenalgorithmen sind, zu testen, nicht mehr eindeutig, welcher Algorithmus der beste ist. So sind $\text{ACO}_{\mathbb{R}}$ und $\text{ACO}_{\mathbb{R}}$ -VS die besten Algorithmen jeder in jeweils 3 von 8 Testfunktionen und Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 ist der beste Algorithmus in 2 der 8 Testfunktionen. Ferner sind die Algorithmen $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}$ -VS und Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 bei den Testfunktionen, wo sie nicht der beste Algorithmus sind, wenn man ihre relative Anzahl an Funktionsauswertungen betrachtet, nahe dem Bestwert, außer bei den Testfunktionen „Griewangk“ und „Rosenbrock(5)“, wo $\text{ACO}_{\mathbb{R}}$ -VS viel schlechter ist als $\text{ACO}_{\mathbb{R}}$ und Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 nicht konvergiert. Auf der anderen Seite kann aus der Tabelle 15 entnommen werden, dass die Algorithmen CACO, API und CIAC, dort, wo Ergebnisse in der Literatur berichtet wurden, schlechter sind, als die drei genannten Favoriten. Bei den Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV mit

NSS-adapt sind die Ergebnisse gemischt aber da sie bei 4 der 8 Testfunktionen schlechter sind (relativ bis 107.5 Mal), werden sie auch, insgesamt, als schlechter als $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}\text{-VS}$ und $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0 betrachtet.

Es folgt, dass unter den Algorithmen, die verwandt zu Ameisenalgorithmen sind, $\text{ACO}_{\mathbb{R}}$ der bessere Algorithmus ist, da er nicht nur bei 3 der 8 Testfunktionen der beste Algorithmus ist, sondern auch, dort wo er nicht der beste Algorithmus ist, er, wenn man seine relative Anzahl an Funktionsauswertungen betrachtet, nahe dem Bestwert ist. Besonders hebt sich $\text{ACO}_{\mathbb{R}}$ bei den Funktionen „Griewangk“ und „Rosenbrock(5)“ hervor. Knapp gefolgt wird $\text{ACO}_{\mathbb{R}}$ von $\text{ACO}_{\mathbb{R}}\text{-VS}$ und $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0. Ferner, gemessen daran, dass die Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, unabhängig der Neustartstrategie, abgesehen von $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$, nicht besser als $\text{ACO}_{\mathbb{R}}\text{-VS}$ sind, jedoch komplexer und $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ auch nur, im Durchschnitt, genauso gut ist, wie $\text{ACO}_{\mathbb{R}}\text{-VS}$, aber auch komplexer, schlussfolgern wir, dass die Erweiterung von $\text{ACO}_{\mathbb{R}}\text{-VS}$ zu den Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, bei Testfunktionen, welche zum Vergleich für Algorithmen, die verwandt zu Ameisenalgorithmen sind, genommen werden, keinen Mehrwert brachte.

Tabelle 15: Vergleich der Ergebnisse zwischen Algorithmen, die verwandt zu Ameisenalgorithmen sind, basierend auf *Socha und Dorigo* [44], und damit auf *Kern et al., 2004* [23], Teil 1

Testfunktion	CACO	API	CIAC	Bestwert
B_2	-	-	30.8 (11968)	1.0 (389)
Goldstein and Price	15.6 (5376)	-	68.1 (23424) [56%]	1.0 (344)
Griewangk	36 (50040)	-	36 (50040) [52%]	1.0 (1390)
Martin and Gaddy	5.5 (1725)	-	37.7 (11730) [20%]	1.0 (311)
Praboloid(6)	44.6 (21868)	20.7 (10153)	102.0 (49984)	1.0 (490)
Rosenbrock(2)	8.3 (6806)	12.0 (9840)	14.0 (11480)	1.0 (820)
Rosenbrock(5)	-	-	16.0 (39792) [90%]	1.0 (2487)
Shekel(4,5)	-	-	88.4 (39350)	1.0 (445)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Tabelle 16: Vergleich der Ergebnisse zwischen Algorithmen, die verwandt zu Ameisenalgorithmen sind, mit Neustartstrategie *fixed* = 0 (NSS-f0) für Varianten von ACO_R-very-simple-mixed-variable, basierend auf *Socha und Dorigo* [44], und damit auf *Kern et al., 2004* [23], Teil 2

Testfunktion	ACO _R	ACO _R -VS	Hom-HPB- ACO _R -VS	Het-HPB- ACO _R -VS	Het-HSPPB- ACO _R -VS	Bestwert
B_2	1.4 (544)	1.9 (747)	1.0 (389) [98%]	1.2 (471) [98%]	1.3 (499)	1.0 (389)
Goldstein and Price	1.1 (384)	1.0 (344)	1.0 (354) [94%]	1.3 (443) [96%]	1.1 (390)	1.0 (344)
Griewangk	1.0 (1390) [61%]	12.3 (17062) [44%]	∞	∞	∞	1.0 (1390)
Martin and Gaddy	1.1 (345)	1.1 (356)	1.0 (311)	1.1 (328)	1.2 (364)	1.0 (311)
Praboloid(6)	1.6 (781)	1.0 (490)	3.2 (1544)	3.3 (1622)	3.9 (1918)	1.0 (490)
Rosenbrock(2)	1.0 (820)	1.5 (1195)	1.5 (1237)	1.9 (1522)	1.6 (1351)	1.0 (820)
Rosenbrock(5)	1.0 (2487) [97%]	15.8 (39345) [54%]	∞	27.1 (67340) [2%]	∞	1.0 (2487)
Shekel(4,5)	1.8 (787) [57%]	1.0 (445)	1.2 (517) [34%]	1.3 (593) [42%]	58.1 (25860) [48%]	1.0 (445)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Tabelle 17: Vergleich der Ergebnisse zwischen Algorithmen, die verwandt zu Ameisenalgorithmen sind, mit Neustartstrategie *adaptive* (NSS-adapt) für Varianten von ACO_R-very-simple-mixed-variable, basierend auf *Socha und Dorigo* [44], und damit auf *Kern et al., 2004* [23], Teil 3, adaptive

Testfunktion	ACO _R	ACO _R -VS	Hom-HPB- ACO _R -VS	Het-HPB- ACO _R -VS	Het-HSPPB- ACO _R -VS	Bestwert
B_2	1.4 (544)	1.9 (747)	1.6 (638) {2}	1.8 (683) {2}	3.7 (1458) {6}	1.0 (389)
Goldstein and Price	1.1 (384)	1.0 (344)	1.6 (563) {2}	1.7 (595) {3}	2.4 (827) {4}	1.0 (344)
Griewangk	1.0 (1390) [61%]	12.3 (17062) [44%]	∞	∞	∞	1.0 (1390)
Martin and Gaddy	1.1 (345)	1.1 (356)	1.3 (419) {2}	1.7 (522) {2}	2.1 (665) {3}	1.0 (311)
Praboloid(6)	1.6 (781)	1.0 (490)	22.4 (10977) {22}	44.0 (21546) {55}	107.5 (52675) [52%] {164}	1.0 (490)
Rosenbrock(2)	1.0 (820)	1.5 (1195)	19.7 (16117) {76}	28.8 (23618) {129}	48.7 (39916) [94%] {243}	1.0 (820)
Rosenbrock(5)	1.0 (2487) [97%]	15.8 (39345) [54%]	30.2 (75218) [2%] {174}	15.0 (37414) [2%] {105}	∞	1.0 (2487)
Shekel(4,5)	1.8 (787) [57%]	1.0 (445)	8.0 (3566) {9}	11.0 (4913) {15}	18.8 (8348) {30}	1.0 (445)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in den Tabellen 18, 19 und 20 zu sehen, ähnlich wie bei der Betrachtung von Testfunktionen, die benutzt werden, um Algorithmen, die verwandt zu Ameisenalgorithmen

sind, zu testen, ist das Bild nicht eindeutig. Zwar ist $\text{ACO}_{\mathbb{R}}\text{-VS}$ der beste Algorithmus bei 6 der 15 Testfunktionen aber er ist schlechter bei Zwei, nämlich „Griewangk“ und „Rosenbrock(5)“ und bei den Testfunktionen „Easom“ und „Hartmann(3,4)“ konnte für $\text{ACO}_{\mathbb{R}}\text{-VS}$ keine Angabe gemacht werden, wie in meiner Bachelorarbeit [31] dargelegt. Auf der anderen Seite ist $\text{ACO}_{\mathbb{R}}$ nur bei einer der Testfunktionen der beste Algorithmus aber, wenn man seine relative Anzahl an Funktionsauswertungen betrachtet, ist er bei jeder Testfunktion nahe dem Bestwert. Ähnlich ist auch ECTS, ein Algorithmus der bei 5 der 15 Testfunktionen der beste Algorithmus ist, bei den Testfunktionen, wo er nicht der beste ist, nahe am relativen Bestwert. Jedoch fehlen bei ECTS 4 der 15 Testfunktionen, vor allem die Testfunktion „Griewangk“, welche sich als schwer herausgestellt hat und die Güte von ECTS bzgl. dieser Testfunktion besonders interessant gewesen wäre. Schließlich ist ECTS bei zwei Testfunktionen schlecht, weswegen dieser Algorithmus insgesamt im Vergleich weniger gut abschneidet. Die Güte von $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ ist vergleichbar mit $\text{ACO}_{\mathbb{R}}$, da $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ nur bei zwei der Testfunktionen der beste ist aber, wenn man seine relative Anzahl an Funktionsauswertungen betrachtet, er bei allen übrigen Testfunktionen, bis auf „Griewangk“ und „Rosenbrock“, den Testfunktionen, wo der Algorithmus nicht konvergiert, nahe dem Bestwert ist. Aber da $\text{HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ gerade bei den schweren Testfunktionen nicht konvergiert, wird $\text{ACO}_{\mathbb{R}}$ als den besseren Algorithmus eingestuft.

Es folgt, dass unter den Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, $\text{ACO}_{\mathbb{R}}$ der bessere Algorithmus ist. Er ist zwar nur bei einer Testfunktion der Beste aber die Tatsache, dass, wenn man seine relative Anzahl an Funktionsauswertungen betrachtet, er bei allen anderen Testfunktionen nahe dem Bestwert ist, spricht dafür, dass $\text{ACO}_{\mathbb{R}}$ der beste Algorithmus ist, da er eine hohe Güte testfunktionunabhängig zeigt, was als wichtiger ansehen wurde, als Bestwerte zu erreichen. Da aber Bestwerte trotzdem für einen Algorithmus sprechen, wird $\text{ACO}_{\mathbb{R}}\text{-VS}$ und ECTS ein zweiter Platz zugeteilt. Die Algorithmen teilen sich den zweiten Platz, da sie beide 5 Mal der beste Algorithmus sind und zudem 2 Mal gerade dort die besten sind, wo der jeweils andere Algorithmus schlecht ist, so ist ECTS der beste Algorithmus bei den Testfunktionen „Rosenbrock(2)“ und „Rosenbrock(5)“, wo $\text{ACO}_{\mathbb{R}}\text{-VS}$ eine relative Anzahl an Funktionsauswertungen von 2.5 bzw. 18.4 braucht, während $\text{ACO}_{\mathbb{R}}\text{-VS}$ der beste Algorithmus bei den Testfunktionen „Hartmann(6,4)“ und „Zakharov(5)“ ist, wo ECTS eine relative Anzahl an Funktionsauswertungen von 4.0 bzw. 3.2 braucht. Auf dem dritten Platz wird $\text{Het-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0 platziert, da dieser Algorithmus ein Mal der beste ist, bei allen anderen Testfunktionen, außer „Rosenbrock“, nahe dem Bestwert ist, wenn man seine relative Anzahl an Funktionsauswertungen betrachtet und, im Gegenteil zu den anderen beiden Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0 , bei der Testfunktion „Rosenbrock“, konvergiert.

Es fällt ferner auf, dass je komplexer eine Iteration von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS -adapt wird, desto schlechter wird sie im Durchschnitt. Auch mit NSS-f0 wird $\text{Het-HSPPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ bei „Schekel(4,5)“, „Schekel(4,7)“ und „Schekel(4,10)“ deutlich schlechter als $\text{ACO}_{\mathbb{R}}\text{-VS}$, $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0 und $\text{Het-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$ mit

NSS-f0. Sonst sind die Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0 in ihrer Güte, im Durchschnitt, was ihre relative Anzahl an Funktionsauswertungen angeht, nahe den Bestwerten. Jedoch sind alle Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, unabhängig ihrer Neustartstrategie, komplexer als $\text{ACO}_{\mathbb{R}}\text{-VS}$. Deswegen wird eingestuft, dass die Erweiterung von $\text{ACO}_{\mathbb{R}}\text{-VS}$ zu den Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, bei Testfunktionen, welche zum Vergleich für Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, genommen werden, keinen Mehrwert brachte.

Tabelle 18: Vergleich zwischen $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, Varianten von $\text{ACO}_{\mathbb{R}}\text{-very-simple-mixed-variable}$ und anderen Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, basierend auf *Socha und Dorigo* [44] und damit auf *Kern et al., 2004* [23], Teil 1

Testfunktion	CGA	ECTS	ESA	DE	Bestwert
B_2	1.1 (430)	-	-	-	1.0 (389)
Branin RCOS	2.5 (613)	1.0 (245)	-	-	1.0 (245)
De Jong	2.3 (745)	-	-	1.2 (392)	1.0 (319)
Easom	4.2 (1467)	-	-	-	1.0 (353)
Goldstein	1.8 (416)	1.0 (231)	3.4 (785)	-	1.0 (231)
Griewangk	-	-	-	9.2 (12788)	1.0 (1390)
Hartmann(3,4)	2.5 (581)	2.3 (547)	2.9 (684)	-	1.0 (234)
Hartmann(6,4)	2.5 (939)	4.0 (1516)	7.1 (2671)	-	1.0 (377)
Rosenbrock(2)	2.0 (960)	1.0 (480)	1.7 (820)	1.3 (624)	1.0 (480)
Rosenbrock(5)	1.9 (4070)	1.0 (2142)	2.5 (5355)	-	1.0 (2142)
Shekel(4,5)	1.4 [76%] (610)	1.9 [75%] (854)	2.6 [54%] (1159)	-	1.0 (445)
Shekel(4,7)	1.5 [83%] (680)	2.0 [80%] (884)	2.8 [54%] (1224)	-	1.0 (444)
Shekel(4,10)	1.5 [83%] (650)	2.1 [80%] (910)	2.6 [50%] (1170)	-	1.0 (443)
Zakharov(2)	3.2 (624)	1.0 (195)	81 (15795)	-	1.0 (195)
Zakharov(5)	2.0 (1381)	3.2 (2254)	100.6 (69792)	-	1.0 (694)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht. Bei der Funktion „De Jong“ konnte für $\text{ACO}_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

Tabelle 19: Vergleich zwischen $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}$ -very-simple, Varianten von $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie $fixed = 0$ (NSS-f0) und anderen Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, basierend auf *Socha und Dorigo* [44] und damit auf *Kern et al., 2004* [23], Teil 2

Testfunktion	$\text{ACO}_{\mathbb{R}}$	$\text{ACO}_{\mathbb{R}}$ -VS	Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HSPPB- $\text{ACO}_{\mathbb{R}}$ -VS	Bestwert
B_2	1.4 (544)	1.9 (747)	1.0 [98%] (389)	1.2 [98%] (471)	1.3 (499)	1.0 (389)
Branin RCOS	3.5 (858)	3.0 (747)	1.1 (268)	1.1 (277)	1.6 (392)	1.0 (245)
De Jong	1.2 (~392)	1.0 (319)	1.1 (355)	1.4 (457)	1.1 (340)	1.0 (319)
Easom	2.2 [98%] (777)	-	1.0 (353)	3.9 (1390)	1.3 (460)	1.0 (353)
Goldstein and Price	1.7 (393)	1.5 (344)	1.5 [94%] (354)	1.9 [96%] (443)	1.7 (390)	1.0 (231)
Griewangk	1.0 [61%] (1390)	12.3 [44%] (17062)	∞	∞	∞	1.0 (1390)
Hartmann(3,4)	1.5 (342)	-	1.1 (251)	1.0 (234)	1.4 (323)	1.0 (234)
Hartmann(6,4)	1.9 (722)	1.0 (377)	1.5 [64%] (557)	1.5 [52%] (550)	1.9 [56%] (703)	1.0 (377)
Rosenbrock(2)	1.7 (820)	2.5 (1195)	2.6 (1237)	3.2 (1522)	2.8 (1351)	1.0 (480)
Rosenbrock(5)	1.2 [97%] (2487)	18.4 [54%] (39345)	∞	31.4 [2%] (67340)	∞	1.0 (2142)
Shekel(4,5)	1.8 [57%] (787)	1.0 (445)	1.2 [34%] (517)	1.3 [42%] (593)	58.1 [48%] (25860)	1.0 (445)
Shekel(4,7)	1.8 [79%] (748)	1.0 (444)	1.2 [34%] (545)	1.3 [28%] (583)	82.4 [64%] (36606)	1.0 (444)
Shekel(4,10)	1.6 [81%] (715)	1.0 (443)	1.6 [28%] (541)	3.4 [20%] (1512)	58.6 [72%] (25942)	1.0 (443)
Zakharov(2)	1.5 (293)	1.7 (332)	1.2 (238)	1.0 (201)	1.3 (252)	1.0 (195)
Zakharov(5)	1.7 (727)	1.0 (694)	2.0 (1402)	1.9 (1352)	2.5 (1760)	1.0 (694)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Bei der Funktion „De Jong“ konnte für $\text{ACO}_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

Tabelle 20: Vergleich zwischen $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}$ -very-simple, Varianten von $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie *adaptive* (NSS-adapt) und anderen Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, basierend auf *Socha und Dorigo* [44] und damit auf *Kern et al., 2004* [23], Teil 3

Testfunktion	$\text{ACO}_{\mathbb{R}}$	$\text{ACO}_{\mathbb{R}}$ -VS	Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HSPPB- $\text{ACO}_{\mathbb{R}}$ -VS	Bestwert
B_2	1.4 (544)	1.9 (747)	1.6 (638) {2}	1.8 (683) {2}	3.7 (1458) {6}	1.0 (389)
Branin RCOS	3.5 (858)	3.0 (747)	1.3 (308) {1}	1.5 (373) {2}	2.1 (509) {2}	1.0 (245)
De Jong	1.2 (~392)	1.0 (319)	1.3 (401) {1}	1.4 (457) {2}	2.0 (625) {2}	1.0 (319)
Easom	2.2 (777)	-	3.3 (1173)	3.9 (1390)	6.3 (2207)	1.0 (353)
	[98%]		{10}	{12}	{19}	
Goldstein and Price	1.7 (393)	1.5 (344)	2.4 (563) {2}	2.6 (595) {3}	3.6 (827) {4}	1.0 (231)
Griewangk	1.0 (1390)	12.3 (17062)	∞	∞	∞	1.0 (1390)
	[61%]	[44%]				
Hartmann(3,4)	1.5 (342)	-	1.2 (277) {1}	1.2 (290) {1}	1.5 (340) {1}	1.0 (234)
Hartmann(6,4)	1.9 (722)	1.0 (377)	5.0 (1880) {4}	7.4 (2808) {7}	8.7 (3280)	1.0 (377)
				{10}	{10}	
Rosenbrock(2)	1.7 (820)	2.5 (1195)	19.2 (16117)	28.1 (23618)	47.5 (39916)	1.0 (480)
			{76}	{129}	[94%] {243}	
Rosenbrock(5)	1.2 (2487)	18.4 (39345)	35.1 (75218)	17.5 (37414)	∞	1.0 (2142)
	[97%]	[54%]	[2%] {174}	[2%] {105}		
Shekel(4,5)	1.8 (787)	1.0 (445)	8.0 (3566) {9}	11.0 (4913)	18.8 (8348)	1.0 (445)
	[57%]			{15}	{30}	
Shekel(4,7)	1.8 (748)	1.0 (444)	6.7 (2993) {8}	9.5 (4237)	14.7 (6523)	1.0 (444)
	[79%]			{12}	{25}	
Shekel(4,10)	1.6 (715)	1.0 (443)	9.0 (3976)	15.6 (6913)	24.5 (10869)	1.0 (443)
	[81%]		{10}	{21}	{40}	
Zakharov(2)	1.5 (293)	1.7 (332)	1.2 (233) {1}	1.4 (270) {1}	2.0 (387) {2}	1.0 (195)
Zakharov(5)	1.7 (727)	1.0 (694)	11.3 (7853)	22.0 (15273)	60.0 (41606)	1.0 (694)
			{21}	{53}	[96%] {178}	

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht. Bei der Funktion „De Jong“ konnte für $\text{ACO}_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

7.2. Vergleich bzgl. Testfunktionen mit gemischten Variablen

7.2.1. Vergleich bzgl. konstruierter Testfunktionen mit gemischten Variablen

Für den Vergleich wurden die Funktionen „Cigar“, „Ellipsoid“, „Paraboloid“ und „Tablet“ gewählt, da sie sowohl für verschiedene Dimensionen definiert werden konnten, als auch wo die Algorithmen, bis auf wenige Ausnahmen, konvergierten. Alle Testfunktionen wurden 50 mal durchlaufen.

Wie in den Tabellen 21 und 22 zu sehen, sind die Unterschiede, in der relativen Anzahl an Funktionsauswertungen der Algorithmen, für Algorithmen, mit der selben Neustartstrategie, gering. So ist zwar Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 der beste Algorithmus bei 24 der 36 Testfunktionen aber die anderen beiden Algorithmen Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 und Het-HSPPB- $\text{ACO}_{\mathbb{R}}$ -VS-MV mit NSS-f0 sind, wenn man ihre relativen Anzahlen an Funktionsauswertungen betrachtet, bis auf zwei Ausreißer, nie

schlechter als um den Faktor 2.0. Die relative Anzahl an Funktionsauswertungen bei Algorithmen mit der Neustartstrategie *adaptive* ist stärker gestreut aber auch hier sind die Anzahlen der relativen Funktionsauswertungen der Algorithmen vergleichbar. Was sich unterscheidet, ist der Faktor, mit dem die relativen Anzahlen an Funktionsauswertungen größer werden. Wenn man die Funktionen aus der Tabelle 22 betrachtet, so wächst der Faktor, mit dem die relativen Anzahlen an Funktionsauswertungen größer werden, sofern die Algorithmen konvergieren, mit zunehmender Dimension, stärker, im Vergleich zu den Funktionen aus Tabelle 21. Deswegen wird eingestuft, dass der Entscheidende Unterschied die Neustartstrategie ist und dass *fixed* = 0, *adaptive* überlegen ist.

Tabelle 21: Vergleich zwischen den Varianten von ACO_ℝ-very-simple-mixed-variable mit Neustartstrategie *fixed* = 0 (NSS-f0), basierend auf *Socha und Dorigo* [44], und damit auf *Kern et al., 2004* [23], Teil 1

Testfunktion	Hom-HPB- ACO _ℝ -VS	Het-HPB- ACO _ℝ -VS	Het-HSPPB- ACO _ℝ -VS	Bestwert
Cigar(2,11)	1.0 (219)	1.0 (218)	1.2 (263)	1.0 (218)
Cigar(2,21)	1.1 (269)	1.0 (240)	1.1 (272)	1.0 (240)
Cigar(2,51)	1.1 (280)	1.0 (266)	1.6 (416)	1.0 (266)
Cigar(5,11)	1.6 (2681)	1.0 (1661)	1.3 (2114)	1.0 (1661)
Cigar(5,21)	1.4 (2447)	1.0 (1766)	1.3 (2259)	1.0 (1766)
Cigar(5,51)	1.4 (2845)	1.0 (1993)	1.2 (2357)	1.0 (1993)
Cigar(10,11)	1.5 (8045)	1.0 (5293)	1.1 (5609)	1.0 (5293)
Cigar(10,21)	1.9 (10023)	1.0 (5395)	1.0 (5395)	1.0 (5395)
Cigar(10,51)	2.0 (11927)	1.0 (5900)	1.0 (6033)	1.0 (5900)
Ellipsoid(2,11)	1.0 (204)	1.1 (225)	6.9 (1417)	1.0 (204)
Ellipsoid(2,21)	1.0 (218)	1.1 (230)	2.5 (548)	1.0 (218)
Ellipsoid(2,51)	1.0 (255)	1.1 (263)	1.2 (301)	1.0 (255)
Ellipsoid(5,11)	1.0 (1218)	1.0 (1203)	1.2 (1433)	1.0 (1203)
Ellipsoid(5,21)	2.4 (2933)	1.0 (1247)	1.2 (1485)	1.0 (1247)
Ellipsoid(5,51)	1.1 (1475)	1.0 (1371)	1.1 (1555)	1.0 (1371)
Ellipsoid(10,11)	1.1 (2939) [98%]	1.0 (2692)	1.1 (3054)	1.0 (2692)
Ellipsoid(10,21)	1.1 (3107)	1.0 (2923)	1.1 (3297)	1.0 (2923)
Ellipsoid(10,51)	1.1 (3661)	1.0 (3281)	1.1 (3746)	1.0 (3281)
Paraboloid(2,11)	1.0 (238)	1.3 (300)	1.5 (361)	1.0 (238)
Paraboloid(2,21)	1.0 (241)	1.0 (240)	1.2 (286)	1.0 (240)
Paraboloid(2,51)	1.0 (248)	1.0 (240)	1.2 (286)	1.0 (240)
Paraboloid(5,11)	1.0 (1012) [94%]	1.0 (1020)	1.2 (1175)	1.0 (1012)
Paraboloid(5,21)	1.0 (990)	1.0 (1020)	1.2 (1197)	1.0 (990)
Paraboloid(5,51)	1.0 (1011) [96%]	1.1 (1121)	1.3 (1333)	1.0 1011
Paraboloid(10,11)	1.2 (2848)	1.0 (2326)	1.1 (2624)	1.0 (2326)
Paraboloid(10,21)	1.0 (2396)	1.0 (2505)	1.2 (2813)	1.0 (2396)
Paraboloid(10,51)	1.1 (2419) [98%]	1.0 (2594)	1.2 (2917)	1.0 (2419)
Tablet(2,11)	1.5 (510)	1.0 (344) [92%]	1.5 (519)	1.0 (344)
Tablet(2,21)	1.2 (412)	1.0 (355) [94%]	1.3 (472)	1.0 (355)
Tablet(2,51)	1.0 (535)	5.8 (3093)	1.0 (543)	1.0 (535)
Tablet(5,11)	1.8 (3490)	1.7 (3327) [98%]	1.0 (1919)	1.0 (1919)
Tablet(5,21)	1.1 (1883)	1.0 (1659)	1.2 (1977)	1.0 (1659)
Tablet(5,51)	1.1 (1925)	1.0 (1726) [94%]	1.2 (2006)	1.0 (1726)
Tablet(10,11)	1.0 (3649)	1.4 (4928) [98%]	1.2 (4441)	1.0 (3649)
Tablet(10,21)	1.0 (3861)	1.0 (3975)	1.0 (3928)	1.0 (3861)
Tablet(10,51)	1.3 (5692)	1.3 (5711) [98%]	1.0 (4247)	1.0 (4247)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht. Bei den Algorithmen, welche „adaptive“ als Neustartstrategie angewendet haben, steht die durchschnittliche Anzahl an Neustarts in geschweiften Klammern.

Tabelle 22: Vergleich zwischen den Varianten von $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie *adaptive* (NSS-adapt), basierend auf *Socha und Dorigo* [44], und damit auf *Kern et al., 2004* [23], Teil 1

Testfunktion	Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HSPPB- $\text{ACO}_{\mathbb{R}}$ -VS	Bestwert
Cigar(2,11)	1.4 (304) {1}	1.6 (350) {2}	1.7 (370) {2}	1.0 (218)
Cigar(2,21)	1.4 (338) {1}	1.5 (355) {1}	1.7 (411) {2}	1.0 (240)
Cigar(2,51)	1.5 (403) {2}	1.9 (505) {2}	1.6 (419) {2}	1.0 (266)
Cigar(5,11)	32.6 (54189) [78%] {103}	26.4 (43833) [94%] {109}	22.8 (37837) [98%] {95}	1.0 (1661)
Cigar(5,21)	26.2 (46250) [76%] {85}	22.9 (40479) [90%] {99}	26.5 (46784) [88%] {115}	1.0 (1766)
Cigar(5,51)	29.3 (58299) [60%] {103}	27.4 (54510) [90%] {129}	26.8 (53427) [86%] {128}	1.0 (1993)
Cigar(10,11)	∞	∞	∞	1.0 (5293)
Cigar(10,21)	∞	∞	∞	1.0 (5395)
Cigar(10,51)	∞	∞	∞	1.0 (5900)
Ellipsoid(2,11)	1.5 (297) {1}	1.8 (363) {2}	2.6 (521) {3}	1.0 (204)
Ellipsoid(2,21)	1.5 (330) {1}	1.8 (397) {2}	2.4 (516) {2}	1.0 (218)
Ellipsoid(2,51)	1.4 (362) {1}	1.6 (419) {2}	2.5 (626) {3}	1.0 (255)
Ellipsoid(5,11)	13.0 (15699) {33}	14.6 (17518) {43}	26.2 (31563) [92%] {101}	1.0 (1203)
Ellipsoid(5,21)	13.0 (16222) {33}	15.6 (19462) {48}	31.5 (39315) [88%] {127}	1.0 (1247)
Ellipsoid(5,51)	15.1 (20717) {41}	16.3 (22313) {55}	33.9 (46454) [78%] {147}	1.0 (1371)
Ellipsoid(10,11)	11.4 (30791) [4%] {47}	5.7 (15353) [4%] {27}	∞	1.0 (2692)
Ellipsoid(10,21)	8.6 (25264) [6%] {36}	1.3 (3705) [2%] {8}	∞	1.0 (2923)
Ellipsoid(10,51)	∞	9.2 (30141) [4%] {73}	∞	1.0 (3281)
Paraboloid(2,11)	1.2 (291) {1}	1.7 (412) {2}	1.7 (393) {2}	1.0 (238)
Paraboloid(2,21)	1.4 (325) {1}	1.7 (397) {2}	2.4 (568) {3}	1.0 (240)
Paraboloid(2,51)	1.5 (358) {1}	1.6 (389) 2	2.4 (583) {3}	1.0 (240)
Paraboloid(5,11)	8.8 (8948) {22}	9.9 (9969) {28}	31.2 (31563) [92%] {101}	1.0 (1012)
Paraboloid(5,21)	11.7 (11563) {29}	10.6 (10489) {29}	39.7 (39315) [88%] {127}	1.0 (990)
Paraboloid(5,51)	10.6 (10766) {25}	13.3 (13437) {37}	45.9 (46454) [78%] {147}	1.0 (1011)
Paraboloid(10,11)	28.7 (66703) [18%] {134}	28.1 (65358) [8%] {157}	∞	1.0 (2326)
Paraboloid(10,21)	33.4 (79970) [8%] {164}	35.2 (84295) [12%] {212}	∞	1.0 (2396)
Paraboloid(10,51)	25.6 (61980) [12%] {123}	23.2 (56190) [8%] {145}	∞	1.0 (2419)
Tablet(2,11)	2.1 (727) {3}	2.3 (806) {3}	4.1 (1410) {6}	1.0 (344)
Tablet(2,21)	2.0 (796) {3}	2.6 (909) {4}	4.8 (1700) {8}	1.0 (355)
Tablet(2,51)	1.5 (793) {3}	1.6 (866) {3}	2.9 (1537) {6}	1.0 (535)
Tablet(5,11)	21.4 (41127) [98%] {81}	17.1 (32808) {84}	41.5 (79666) [20%] 261	1.0 (1919)
Tablet(5,21)	20.6 (34135) [98%] {66}	21.6 (35840) [94%] {91}	36.7 (60965) [32%] 189	1.0 (1659)
Tablet(5,51)	26.3 (45325) [92%] {85}	15.2 (26181) [92%] {64}	33.3 (57457) [32%] 177	1.0 (1726)
Tablet(10,11)	2.7 (9984) [2%] {14}	∞	∞	1.0 (3649)
Tablet(10,21)	∞	22.7 (87757) [4%] {203}	∞	1.0 (3861)
Tablet(10,51)	∞	2.7 (11616) [4%] {22}	∞	1.0 (4247)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht. Bei den Algorithmen, welche „adaptive“ als Neustartstrategie angewendet haben, steht die durchschnittliche Anzahl an Neustarts in geschweiften Klammern.

7.2.2. Vergleich bzgl. dem Coil Spring Design Problem (CSD)

Dies ist der letzte Vergleich, den in dieser Arbeit betrachten wird. In Abschnitt 7.1 wurde gezeigt, dass die Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV mit stetigen Testfunktionen umgehen können und im Abschnitt 7.2.1 wurde gezeigt, dass die Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV mit konstruierten Testfunktionen mit gemischten Variablen umgehen können. Jetzt wird untersucht, ob die Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV auch mit einem realen Problem, dem Coil Spring Design Problem, umgehen können. Alle Fälle des Testproblems wurden 50 mal durchlaufen.

Hier wurde von *Socha* [43] im Gegenteil zu anderen Vergleichen wo ganze Zahlen verwendet, die Erfolgsrate mancher Algorithmen in Prozent auf Zehntel genau angegeben, weswegen in den Tabellen in diesem Abschnitt auch die Erfolgsrate von den Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ auf Zehntel genau angegeben werden.

In der Tabelle 23 sind die Ergebnisse von $\text{ACO}_{\mathbb{R}}$, ACO_{MV} und Vergleichsalgorithmen zu sehen. In den Tabellen 24 und 25 sind nochmal die Ergebnisse von $\text{ACO}_{\mathbb{R}}$, ACO_{MV} und dann die Ergebnisse von den Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ zu sehen. Leider fehlt für die Algorithmen „NLIDP“ aus *Sandgren* [41], „GA“ aus *Wu und Chow* [52] und „DE“ aus *Lampinen und Zelinka* [27] die Anzahl an Funktionsauswertungen, welche die Algorithmen gebraucht haben, um das Optimum zu erreichen, weswegen sie auch in *Socha* [43] fehlen, der Arbeit, welche auf ihnen aufbaut. Es wird in allen Arbeiten, entweder nur die Anzahl an verfügbaren Funktionsauswertungen für die Optimumssuche angegeben, in allen Fällen, wo es angegeben wurde, sind es 8000, und der Anteil der Erfolgreichen Durchläufe in Prozent, oder nur der Anteil der erfolgreichen Durchläufe in Prozent. In *Liao et al.* [29], der Folgearbeit zu *Socha* [43], wird aber angegeben, dass in der Tat die vollen 8000 Funktionsauswertungen gebraucht wurden und diese sogar zu wenig waren, da der Anteil der erfolgreichen Durchläufe für ACO_{MV} in *Socha* [43] bei 39.0% und in *Liao et al.* [29] bei 74% liegt. Ferner wird in *Liao et al.* [29] angegeben, dass ACO_{MV} für das CSD Problem im Durchschnitt 9948 Funktionsauswertungen gebraucht hat und bei einer Maximalzahl von 19588 Funktionsauswertungen ACO_{MV} eine Durchgangserfolgsrate von 100% hatte. Aber diese Angaben werden im Fließtext, außerhalb der tabellarischen Vergleiche mit anderen Funktionen, gemacht. Aber wenn von diesen Werten ausgegangen wird, kann festgestellt werden, dass das Ergebnis von 619 Funktionsausführungen in Tabelle 24 gut ist. Eine andere Tatsache, welche die Vermutung untermauert, dass exakt 8000 Funktionsauswertungen von ACO_{MV} gebraucht wurden, ist das für ein anderes Ingenieursproblem, das Pressure Vessel Design Problem, in *Socha* [43], sowohl die maximale zur Verfügung stehende Anzahl an Funktionsauswertungen, dort sind es 10000, als auch die Benötigte, angegeben werden.

Es kann das selbe Ergebnis beobachten, wie bei den konstruierten Testfunktionen mit gemischten Variablen, nämlich, dass die Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-f0 besser sind als die Varianten von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ mit NSS-adapt. Es kann auch weiter beobachten werden, dass, wenn der gesamte Raum, zum ziehen der Initiallösungen, genommen wird, aber dann nur, mit den Constrains konforme, Lösungen bei der Lösungssuche erlaubt werden, also bei dem Fall „penalty-exact“, keine Iteration von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, unabhängig der Neustartstrategie, konvergiert. Es wird davon ausgegangen, dass das daran liegt, dass, ausgehend von nur 13 zufälligen Initiallösungen, es sehr schwer ist, eine mit den Constrains konforme Lösung zu generieren, da die Größe des gesamten Suchraums, aus dem die Initiallösungen gezogen werden, im Vergleich zur Größe des Teils des Suchraums, der mit den Constrains konforme Lösungen enthält, viel größer ist, wie in Abbildung 10 zu sehen.

Tabelle 23: Vergleich der Ergebnisse zwischen $ACO_{\mathbb{R}}$, ACO_{MV} und Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden

Testfunktion	$ACO_{\mathbb{R}}$	ACO_{MV}	NLIDP	GA	DE	Bestwert
exakt-exakt	-	-	-	-	-	-
exakt-penalty	-	-	-	-	-	-
penalty-exakt	-	-	-	-	-	-
penalty-penalty	12.9 (≤ 8000) [82.0%]	12.9 (≤ 8000) [39.0%]	- [100.0%]	- [95.3%]	12.9 (≤ 8000) [95.0%]	1.0 (619)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, auch wenn die Anzahl an Funktionsauswertungen nicht bekannt ist, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Tabelle 24: Vergleich der Ergebnisse zwischen $ACO_{\mathbb{R}}$, ACO_{MV} , den Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie *fixed* = 0 (NSS-f0) und Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden

Testfunktion	$ACO_{\mathbb{R}}$	ACO_{MV}	Hom-HPB- $ACO_{\mathbb{R}}$ -VS	Het-HPB- $ACO_{\mathbb{R}}$ -VS	Het-HSPPB- $ACO_{\mathbb{R}}$ -VS	Bestwert
exakt-exakt	-	-	1.0 (144575) [22.0%] (142931) (1285)	1.1 (152564) [18.0%] (149447) (2367)	1.2 (174319) [46.0%] (158855) (7016)	1.0 (144575)
exakt-penalty	-	-	1.2 (133910) [20.0%] (132922) (0)	1.0 (116594) [28.0%] (111652) (0)	1.0 (112408) [58%] (90549) (0)	1.0 (112408)
penalty-exakt	-	-	∞	∞	∞	-
penalty-penalty	12.9 (≤ 8000) [82.0%]	12.9 (≤ 8000) [39.0%]	1.3 (783) [10.0%] (0) (0)	1.0 (619) [22.0%] (0) (0)	30.0 (18587) [66.0%] (0) (0)	1.0 (619)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, danach sind in Klammern die Anzahl an Iteration angegeben, die notwendig waren, um gefilterte Ausgangslösungen zu erhalten, und die Anzahl an Varianten, um gefilterte Lösungen während der Algorithmenausführung zu erhalten, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Tabelle 25: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$, ACO_{MV} , den Variablen von $\text{ACO}_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie *adaptive* (NSS-adapt) und Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden

Testfunktion	$\text{ACO}_{\mathbb{R}}$	ACO_{MV}	Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HPB- $\text{ACO}_{\mathbb{R}}$ -VS	Het-HSPPB- $\text{ACO}_{\mathbb{R}}$ -VS	Bestwert
exakt-exakt	-	-	6.3 (988021) {9} (979734) (5847)	9.6 (1385402) {14} (1371913) (9940)	9.1 (1313269) {14} (1298758) (11020)	1.0 (144575)
exakt-penalty	-	-	32.0 (3600212) {45} (3591069) (0)	57.7 (6491284) {87} (6475082) (0)	104.9 (11791721) {166} (11764326) (0)	1.0 (112408)
penalty-exakt	-	-	∞	∞	∞	-
penalty-penalty	12.9 (≤ 8000) [82.0%]	12.9 (≤ 8000) [39.0%]	13.5 (8358) {31} (0) (0)	16.8 (10393) {43} (0) (0)	39.7 (24552) [98.0%] {116} (0) (0)	1.0 (619)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben. Bei den Algorithmen, welche „adaptive“ als Neustartstrategie angewendet haben, steht die durchschnittliche Anzahl an Neustarts in geschweiften Klammern, danach sind in Klammern die Anzahl an Iteration angegeben, die notwendig waren, um gefilterte Ausgangslösungen zu erhalten, und die Anzahl an Varianten, um gefilterte Lösungen während der Algorithmenausführung zu erhalten, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Es kann weiter festgestellt werden, dass bei allen Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV, unabhängig der Neustartstrategie, wo die Initiallösungen aus dem gefilterten Raum gezogen wurden, also die Fälle „exact-exact“ und „exact-penalty“, für das generieren der Initiallösungen die meisten Funktionsauswertungen gebraucht wurden. Deswegen wird das Verhältnis untersucht zwischen, für die Initialisierung und für die Lösungssuche verwendeter Funktionsauswertungen, nochmal gesondert. Alle Boxplots in diesem Abschnitt und in der Anlage A wurden von mir mit der Python-Bibliothek „matplotlib.pyplot“ erstellt. In allen Abbildungen werden bei „real“ nur die Funktionsauswertungen von erfolgreichen Durchläufen betrachtet.

Wie in den Abbildungen 11 bis 14 zu sehen, brauchen alle Varianten von $\text{ACO}_{\mathbb{R}}$ -VS-MV, unabhängig der Neustartstrategie, mehr Funktionsauswertungen für das Suchen gefilterter Lösungen als für die Konstruktion neuer Lösungen selbst, auch wenn man die, bei der Lösungssuche zusätzlichen Funktionsauswertungen mitzählt. Das gleiche Verhalten kann man in den Abbildungen 15 bis 18, welche sich im Anhang befinden. Auf der anderen Seite kann beobachtet werden, dass das beste Ergebnis bei dem Fall „penalty-penalty“ erreicht wurde. Ebenso kann beobachtet werden, dass das Hinzufügen von adaptiven Neustarts die Leistung des Algorithmus verschlechtert. Es folgt der Schluss, dass, sowohl das Filtern des Raumes vor der Algorithmusausführung allein keinen Mehrwert brachte, als auch das Filtern des Raum von der Algorithmusausführung verbunden mit dem Filtern von Lösungen während der Algorithmusausführung, keinen Mehrwert brachte.

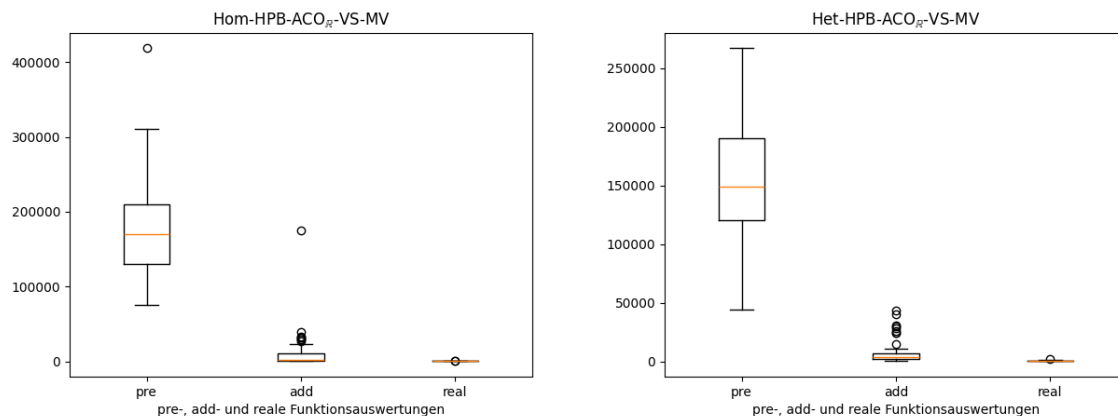


Abbildung 11: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-f0, Teil 1

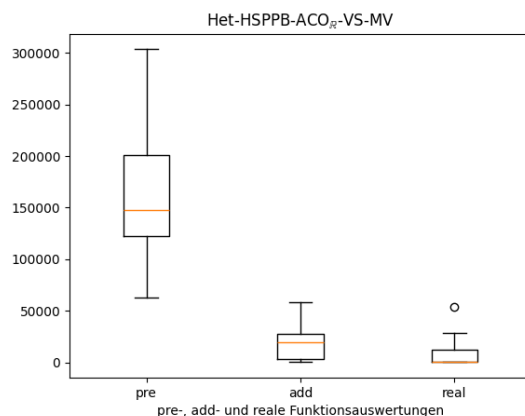


Abbildung 12: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-f0, Teil 2

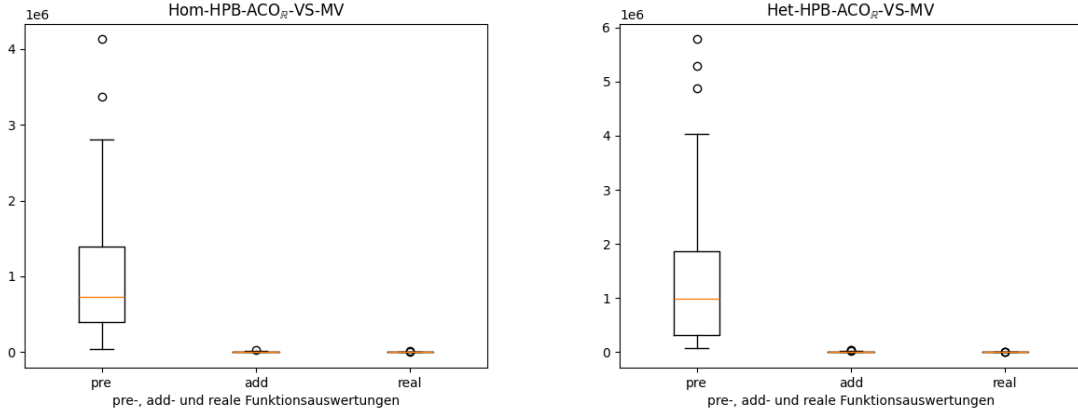


Abbildung 13: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltrern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-adapt, Teil 1

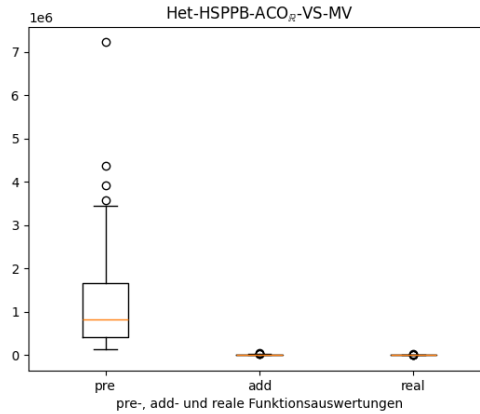


Abbildung 14: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltrern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-adapt, Teil 2

8. Kurzzusammenfassung

Im Vergleich zu ACO_R-very-simple aus meiner Bachelorarbeit [31], schneiden die Varianten von ACO_R-very-simple-mixed-variable bei konstruierten Testfunktionen vergleichbar mit ACO_R-VS ab, oder sie sind schlechter. Es fällt ferner auf, dass das Hinzufügen der Neustartstrategie *adapt* zu schlechteren Ergebnissen geführt hat. Und da das Ziel dieser Arbeit war, herauszufinden, ob eine komplexere Formulierung von ACO_R-VS zu besseren Ergebnissen führt, was, konstruierte Funktionen angeht, nicht geschehen ist, folgt, dass bei Funktionen, die den Testfunktionen in dieser Arbeit ähneln, ACO_R-VS,

$\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, vorzuziehen ist und sollte das Aussehen einer Funktion nicht genau bekannt sein, wenn man jedoch aber davon ausgehen kann, dass sie stetig ist, wird $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}\text{-VS}$ und $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$, vorgezogen.

Auf der anderen Seite ist $\text{Het-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$, mit NSS-f0 und dem Fall „penalty-penalty“, also ohne Filterung des Suchraumes vor oder während der Ausführung des Algorithmus, bei dem Coil Spring Design Problem, 12.9-Mal besser als $\text{ACO}_{\mathbb{R}}$ und ACO_{MV} , und $\text{Hom-HPB-ACO}_{\mathbb{R}}\text{-VS-MV}$, mit NSS-f0 und dem Fall „penalty-penalty“ ist 9.9-Mal besser.

Es folgt also, dass die Vermutung, dass die Betrachtung von einfachen Testfunktionen sich lohnen würde, richtig war. Einfache Testfunktionen haben die Schwächen der Variationen von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ aufgezeigt. Da aber 2 der Variationen von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ besser sind als alle anderen Algorithmen, insbesondere auch ACO_{MV} , ein Algorithmus der speziell für Probleme mit gemischten Variablen entwickelt wurde (relativ bis zu 12.9), folgt, dass weitere Untersuchungen der Variationen von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ eine offene Forschungsfrage sind. Drei weitere Forschungsfragen sind die Untersuchung der Leistung von $\text{ACO}_{\mathbb{R}}\text{-very-simple-mixed-variable}$ selbst, da die ursprüngliche Variation am einfachsten und damit möglicherweise am besten ist, die Untersuchung von den Variationen von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ mit einer Archivgröße von 100, anstatt von 4, da eine Verkleinerung des Lösungsarchivs zu einer schlechteren Leistung der Variationen von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ geführt haben könnte, und die Untersuchung von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ und dessen Varianten auf Testfunktionen, deren Definitionsbereiche/Suchräume nicht zusammenhängend/äquidistant verteilt sind, da die Einfachheit von $\text{ACO}_{\mathbb{R}}\text{-VS-MV}$ und seinen Varianten von der Gleichmäßigkeit mehr profitieren könnten als die Vergleichsalgorithmen und $\text{ACO}_{\mathbb{R}}$, sowie ACO_{MV} .

Literatur

- [1] BILCHEV, G. ; PARMEE, I. C.: The ant colony metaphor for searching continuous design spaces. In: *AISB workshop on evolutionary computing* Springer, 1995, S. 25–39
- [2] BLUM, C. : Ant colony optimization: Introduction and recent trends. In: *Physics of Life reviews* 2 (2005), Nr. 4, S. 353–373
- [3] BONABEAU, E. ; DORIGO, M. ; THERAULAZ, G. : *Swarm intelligence: from natural to artificial systems*. Oxford university press, 1999
- [4] BOSMAN, P. A. ; THIERENS, D. : Continuous iterated density estimation evolutionary algorithms within the IDEA framework. (2000)
- [5] CAMAZINE, S. ; DENEUBOURG, J. ; FRANKS, N. ; SNEYD, J. ; THERAULAZ, G. u. a.: *Self-organization in Biological Systems 2003 Princeton*. 2003
- [6] CHELOUAH, R. ; SIARRY, P. : Enhanced continuous tabu search: An algorithm for optimizing multim minima functions. In: *Meta-heuristics: advances and trends in local search paradigms for optimization*. Springer, 1999, S. 49–61
- [7] CHELOUAH, R. ; SIARRY, P. : A continuous genetic algorithm designed for the global optimization of multimodal functions. In: *Journal of Heuristics* 6 (2000), Nr. 2, S. 191–213
- [8] CHEN, W.-N. ; ZHANG, J. : A set-based discrete PSO for cloud workflow scheduling with user-defined QoS constraints. In: *2012 IEEE International Conference on Systems, Man, and Cybernetics (SMC)* IEEE, 2012, S. 773–778
- [9] CHRISTIAN, B. ; DANIEL, M. : Swarm intelligence introduction and application. In: *Natural Computing Series, Springer* (2008), S. 87–100
- [10] DEB, K. ; GOYAL, M. : A flexible optimization procedure for mechanical component design based on genetic adaptive search. (1998)
- [11] DORIGO, M. ; MANIEZZO, V. ; COLORNI, A. : Ant system: optimization by a colony of cooperating agents. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 26 (1996), Nr. 1, S. 29–41
- [12] DORIGO, M. ; STÜTZLE, T. : Ant colony optimization: overview and recent advances. In: *Handbook of metaheuristics* (2018), S. 311–351
- [13] DRÉO, J. ; SIARRY, P. : A new ant colony algorithm using the heterarchical concept aimed at optimization of multim minima continuous functions. In: *International Workshop on Ant Algorithms* Springer, 2002, S. 216–221
- [14] ENGELBRECHT, A. P.: Fundamentals of Computational Swarm Intelligence. In: *Hoboken, NJ: Wiley* (2005)

- [15] GLOVER, F. : Future paths for integer programming and links to artificial intelligence. In: *Computers & operations research* 13 (1986), Nr. 5, S. 533–549
- [16] GONG, Y. ; WANG, W. ; GONG, S. : A Novel Self-Adaptive Mixed-Variable Multiobjective Ant Colony Optimization Algorithm in Mobile Edge Computing. In: *Security and Communication Networks* 2022 (2022), Nr. 1, S. 4967775
- [17] GUNTSCHE, M. ; MIDDENDORF, M. : A Population Based Approach for ACO. In: *Workshops on Applications of Evolutionary Computing*, Springer Berlin Heidelberg, 2002, S. 72–81
- [18] GUO, C.-x. ; HU, J.-s. ; YE, B. ; CAO, Y.-j. : Swarm intelligence for mixed-variable design optimization. In: *Journal of Zhejiang University-SCIENCE A* 5 (2004), Nr. 7, S. 851–860
- [19] HANSEN, N. ; OSTERMEIER, A. : Completely derandomized self-adaptation in evolution strategies. In: *Evolutionary computation* 9 (2001), Nr. 2, S. 159–195
- [20] HOLLAND, J. H.: Adaptation in natural and artificial systems. In: *Ann Arbor: The University of Michigan Press* 32 (1975)
- [21] HOLLAND, J. H.: Outline for a logical theory of adaptive systems. In: *Journal of the ACM (JACM)* 9 (1962), Nr. 3, S. 297–314
- [22] KANNAN, B. ; KRAMER, S. N.: An augmented Lagrange multiplier based method for mixed integer discrete continuous optimization and its applications to mechanical design. In: *Journal of mechanical design* 116 (1994), Nr. 2, S. 405–411
- [23] KERN, S. ; MÜLLER, S. D. ; HANSEN, N. ; BÜCHE, D. ; OCENASEK, J. ; KOMOUTSAKOS, P. : Learning probability distributions in continuous evolutionary algorithms—a comparative review. In: *Natural Computing* 3 (2004), Nr. 1, S. 77–112
- [24] KIM, J. : *Benchmark functions for bayesian optimization*. 2020
- [25] KIRKPATRICK, S. ; GELATT JR, C. D. ; VECCHI, M. P.: Optimization by simulated annealing. In: *science* 220 (1983), Nr. 4598, S. 671–680
- [26] KUPFER, E. ; LE, H. T. ; ZITT, J. ; LIN, Y.-C. ; MIDDENDORF, M. : A Hierarchical Simple Probabilistic Population-Based Algorithm Applied to the Dynamic TSP. In: *2021 IEEE Symposium Series on Computational Intelligence (SSCI)* IEEE, 2021, S. 1–8
- [27] *Kapitel 8*. In: LAMPINEN, J. ; ZELINKA, I. : *Mechanical engineering design optimization by differential evolution*. GBR : McGraw-Hill Ltd., UK, 1999. – ISBN 0077095065, S. 127–146
- [28] LAMPINEN, J. ; ZELINKA, I. : MIXED INTEGER-DISCRETE-CONTINUOUS OPTIMIZATION BY DIFFERENTIAL EVOLUTION Part 2 : a practical example

- [29] LIAO, T. ; SOCHA, K. ; OCA, M. A. M. ; STÜTZLE, T. ; DORIGO, M. : Ant colony optimization for mixed-variable optimization problems. In: *IEEE Transactions on evolutionary computation* 18 (2013), Nr. 4, S. 503–518
- [30] LIN, Y.-C. ; CLAUSS, M. ; MIDDENDORF, M. : Simple probabilistic population-based optimization. In: *IEEE Transactions on Evolutionary Computation* 20 (2015), Nr. 2, S. 245–262
- [31] MADORSKIY, F. C.: *Vergleich zweier Varianten der Lösungserzeugung von Ameisenalgorithmen zur Funktionsoptimierung*, Universität Leipzig, Bachelorarbeit, 2019
- [32] MARTIN, R. K. ; SWEENEY, D. J. ; DOHERTY, M. E.: The reduced cost branch and bound algorithm for mixed integer programming. In: *Computers & operations research* 12 (1985), Nr. 2, S. 139–149
- [33] MERKLE, D. ; MIDDENDORF, M. : On the behavior of ACO algorithms: Studies on simple problems. In: *Metaheuristics: computer decision-making*. Springer, 2003, S. 465–480
- [34] MOLGA, M. ; SMUTNICKI, C. : Test functions for optimization needs. In: *Test functions for optimization needs* 101 (2005)
- [35] MONMARCHÉ, N. ; VENTURINI, G. ; SLIMANE, M. : On how *Pachycondyla apicalis* ants suggest a new search algorithm. In: *Future generation computer systems* 16 (2000), Nr. 8, S. 937–946
- [36] OCENÁŠEK, J. ; SCHWARZ, J. : Estimation distribution algorithm for mixed continuous-discrete optimization problems. In: *2nd Euro-International Symposium on Computational Intelligence* IOS Press Kosice, 2002, S. 227–232
- [37] OSTERMEIER, A. ; GAWELCZYK, A. ; HANSEN, N. : Step-size adaptation based on non-local use of selection information. In: *International Conference on Parallel Problem Solving from Nature* Springer, 1994, S. 189–198
- [38] PELIKAN, M. ; GOLDBERG, D. E. ; SASTRY, K. : Bayesian optimization algorithm, decision graphs, and Occam’s razor. In: *Proceedings of the 3rd Annual Conference on Genetic and Evolutionary Computation*, 2001, S. 519–526
- [39] POHLHEIM, H. : Examples of objective functions. In: *Retrieved* 4 (2007), Nr. 10, S. 2012
- [40] RECHENBERG, I. : Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biologischen evolution, frommann–holzboog. In: *Stuttgart-Bad Cannstatt* 47 (1973)
- [41] SANDGREN, E. : Nonlinear integer and discrete programming in mechanical design optimization. (1990), S. 223–229

- [42] SIARRY, P. ; BERTHIAU, G. ; DURDIN, F. ; HAUSSY, J. : Enhanced simulated annealing for globally minimizing functions of many-continuous variables. In: *ACM Transactions on Mathematical Software (TOMS)* 23 (1997), Nr. 2, S. 209–228
- [43] SOCHA, K. : *Ant colony optimisation for continuous and mixed-variable domains*. VDM Publishing Saarbrücken, 2009
- [44] SOCHA, K. ; DORIGO, M. : Ant colony optimization for continuous domains. In: *European journal of operational research* 185 (2008), Nr. 3, S. 1155–1173
- [45] STORN, R. ; PRICE, K. : Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. In: *Journal of global optimization* 11 (1997), Nr. 4, S. 341–359
- [46] STÜTZLE, T. ; DORIGO, M. u. a.: ACO algorithms for the traveling salesman problem. In: *Evolutionary algorithms in engineering and computer science* 4 (1999), S. 163–183
- [47] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/hart3.html> (abgerufen am 17.08.2019)
- [48] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/hart6.html> (abgerufen am 17.08.2019)
- [49] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/shekel.html> (abgerufen am 17.08.2019)
- [50] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/branin.html> (abgerufen am 19.08.2019)
- [51] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/goldpr.html> (abgerufen am 19.08.2019)
- [52] WU, S.-J. ; CHOW, P.-T. : Genetic algorithms for nonlinear mixed discrete-integer optimization problems via meta-genetic parameter optimization. In: *Engineering Optimization+ A35* 24 (1995), Nr. 2, S. 137–159

Tabellenverzeichnis

1.	Parametereinstellungen ACO _R -simple	13
2.	Parametereinstellungen ACO _R -VS	13
3.	Parametereinstellungen ACO _R -VS-MV	14
4.	Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen	29
5.	erster Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden	30
6.	zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden	31
7.	zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Testfunktionen adaptiert wurden	32
8.	Standarddrahtdurchmesser für die Schraubenfeder angegeben in Zoll (inch)	38
9.	Intervallgrenzen	39
10.	Definitionsbereiche	39
11.	Constrains	40
12.	Gewichte	41
13.	Vergleich der Ergebnisse zwischen Algorithmen, die probabilistisches Lernen einsetzen, basierend auf <i>Socha und Dorigo</i> [44], und damit auf <i>Kern et al., 2004</i> [23]	44
14.	Vergleich der Ergebnisse zwischen ACO _R , ACO _R -very-simple und den Varianten von ACO _R -very-simple-mixed-variable, mit Neustartstrategie <i>fixed</i> = 0 (NSS-f0), auf Testfunktionen, die zum Vergleich zwischen Algorithmen, die probabilistisches Lernen einsetzen, benutzt werden, basierend auf meiner Bachelorarbeit [31], <i>Socha und Dorigo</i> [44] und damit auf <i>Kern et al., 2004</i> [23]	44
15.	Vergleich der Ergebnisse zwischen Algorithmen, die verwandt zu Ameisenalgorithmen sind, basierend auf <i>Socha und Dorigo</i> [44], und damit auf <i>Kern et al., 2004</i> [23], Teil 1	45
16.	Vergleich der Ergebnisse zwischen Algorithmen, die verwandt zu Ameisenalgorithmen sind, mit Neustartstrategie <i>fixed</i> = 0 (NSS-f0) für Varianten von ACO _R -very-simple-mixed-variable, basierend auf <i>Socha und Dorigo</i> [44], und damit auf <i>Kern et al., 2004</i> [23], Teil 2	46
17.	Vergleich der Ergebnisse zwischen Algorithmen, die verwandt zu Ameisenalgorithmen sind, mit Neustartstrategie <i>adaptive</i> (NSS-adapt) für Varianten von ACO _R -very-simple-mixed-variable, basierend auf <i>Socha und Dorigo</i> [44], und damit auf <i>Kern et al., 2004</i> [23], Teil 3, adaptive	46

18.	Vergleich zwischen $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$ -very-simple, Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable und anderen Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, basierend auf <i>Socha und Dorigo</i> [44] und damit auf <i>Kern et al., 2004</i> [23], Teil 1	48
19.	Vergleich zwischen $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$ -very-simple, Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie <i>fixed</i> = 0 (NSS-f0) und anderen Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, basierend auf <i>Socha und Dorigo</i> [44] und damit auf <i>Kern et al., 2004</i> [23], Teil 2	49
20.	Vergleich zwischen $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$ -very-simple, Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie <i>adaptive</i> (NSS-adapt) und anderen Metaheuristiken, die für stetige Testfunktionen adaptiert wurden, basierend auf <i>Socha und Dorigo</i> [44] und damit auf <i>Kern et al., 2004</i> [23], Teil 3	50
21.	Vergleich zwischen den Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie <i>fixed</i> = 0 (NSS-f0), basierend auf <i>Socha und Dorigo</i> [44], und damit auf <i>Kern et al., 2004</i> [23], Teil 1	51
22.	Vergleich zwischen den Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie <i>adaptive</i> (NSS-adapt), basierend auf <i>Socha und Dorigo</i> [44], und damit auf <i>Kern et al., 2004</i> [23], Teil 1	52
23.	Vergleich der Ergebnisse zwischen $ACO_{\mathbb{R}}$, ACO_{MV} und Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden	54
24.	Vergleich der Ergebnisse zwischen $ACO_{\mathbb{R}}$, ACO_{MV} , den Varianten von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie <i>fixed</i> = 0 (NSS-f0) und Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden	54
25.	Vergleich der Ergebnisse zwischen $ACO_{\mathbb{R}}$, ACO_{MV} , den Variablen von $ACO_{\mathbb{R}}$ -very-simple-mixed-variable mit Neustartstrategie <i>adaptive</i> (NSS-adapt) und Algorithmen, die für Probleme mit gemischten Variablen entwickelt wurden	55

Abbildungsverzeichnis

1.	Experimentaufstellung in Anlehnung an <i>Engelbrecht</i> [14]	3
2.	Paraboloid - Visualisierung in 3D	33
3.	Easom-Testfunktion: Vergleich zwischen gesamtem Suchraum (links) und Umgebung um das Optimum (rechts)	33
4.	Verschiedene Auflösungen der Griewangk-Testfunktion	34
5.	Rosenbrock - Visualisierung in 3D	35
6.	Hartmann-Testfunktionen: Hartmann(2,4) (links) und Hartmann(2,6) (rechts)	35
7.	Shekel-Testfunktionen: Shekel(2,5) (links), Shekel(2,7) (mittig) und Shekel(2,10) (rechts)	36

8.	Skizze einer Schraubenfeder mit Federdurchmesser (links), Windungszahl (mitte) und Drahtdurchmesser (rechts)"; das Bild wurde von mir mit einem Zeichenprogramm erstellt und dann mit ChatGPT verbessert . . .	38
9.	gefilterter und ungefilterter Suchraum für das CSD-Problem	42
10.	gefilterter Raum (orange) und ungefilterter Raum (blau)	42
11.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-f0, Teil 1	56
12.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-f0, Teil 2	56
13.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-adapt, Teil 1	57
14.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-exact“ und NSS-adapt, Teil 2	57
15.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-f0, Teil 1	I
16.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-f0, Teil 2	I
17.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-adapt, Teil 1	I
18.	Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-adapt, Teil 2	II

Anlagen

A. Vergleich der pre-, add- und realen Funktionsauswertungen

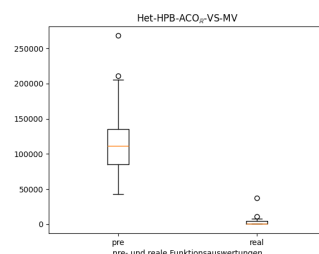
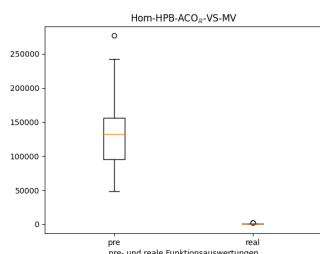


Abbildung 15: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-f0, Teil 1

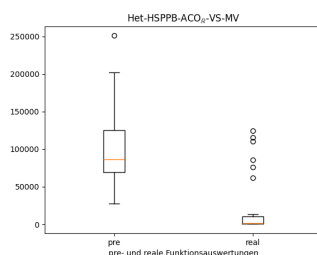


Abbildung 16: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-f0, Teil 2

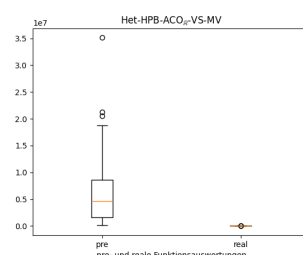
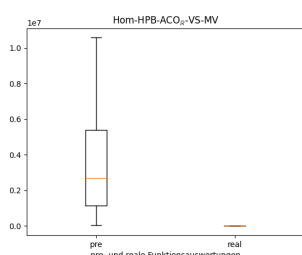


Abbildung 17: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-adapt, Teil 1

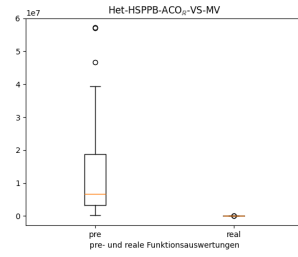


Abbildung 18: Verhältnis zwischen den benötigten Funktionsauswertungen für Vorfiltern, zusätzlichem Filtern und realen Iterationsschritten der Algorithmen für den Fall „exact-penalty“ und NSS-adapt, Teil 2

Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Ort: Leipzig

Datum:

Unterschrift: